

Particle Flow Filter and Differentiable Particle Filter

Haowu Duan

April 15, 2026

Contents

1	The big picture	3
1.1	State-Space Modeling in Financial Markets	3
1.1.1	Filtering	3
1.1.2	Bayesian Inference	3
1.2	Document Roadmap	4
2	Kalman Filter	5
2.1	Overview	5
2.2	Algorithm	5
2.3	Numerical stability analysis for Kalman filter	7
2.4	The limitation and usefulness of Kalman filter	8
3	EKF and UKF	10
3.1	EKF	10
3.2	UKF	11
3.3	EKF/UKF on non-linear models	12
4	Particle-Based Methods	16
4.1	Importance Sampling and Sequential Monte Carlo	16
4.2	Bootstrap Particle Filter	17
4.2.1	Algorithm	17
4.2.2	Particle Degeneracy and Resampling	17
4.3	EDH/LEDH Flow Filter	20
4.3.1	Use Fokker-Planck to reach the target distribution	20
4.3.2	Determine the evolution kernel	21
4.3.3	EDH vs. LEDH: incremental improvements in linearization quality.	22
4.3.4	Algorithm	22
4.3.5	Experiments	24
4.4	Invertible Flow Filter	29
4.4.1	Why do we need Jacobian computation?	29
4.5	Stochastic Flow Filter	32
4.6	Kernel Flow Filter	35
4.6.1	Overview	35
4.6.2	Derivation	35
4.6.3	Matrix Kernel Preventing Collapse	37
4.6.4	Algorithm	37
5	Resampling and Optimal Transport	39
5.1	Two Problems with Resampling	39
5.2	Soft Resampling	39
5.3	Entropy-Regularized OT (Sinkhorn) Resampling	40
5.4	The Derivative Chain through Resampling	42
5.5	Approximate vs Implicit Sinkhorn Backward	44
5.5.1	The Extrapolation Approximation	44
5.5.2	Implicit Differentiation at the Fixed Point	44
5.6	Neural Acceleration	47

6	Parameter Estimation with Differentiable Particle Filters	50
6.1	Markov Chain Monte Carlo	50
6.1.1	Metropolis-Hastings	50
6.1.2	Hamiltonian Monte Carlo	51
6.2	MAP as a Pipeline Smoke Test	52
6.3	Differentiable Resampling and the Gradient Chain	57
6.4	Parameter Prior Selection	58
6.5	Benchmark: Linear Gaussian Model	60
6.5.1	MCMC Diagnostics	60
6.5.2	1-D Model, 1 Parameter	61
6.5.3	Multi-Dimensional Model, 1 Parameter	61
6.6	Gradient Diagnostics on SV2D: Unresolved Nonlinear Failure	61
6.7	Limitations	62

1 The big picture

1.1 State-Space Modeling in Financial Markets

Financial systems are driven by latent factors that shape observable market data. A canonical example is stochastic volatility: the variance of asset returns is not constant but evolves as a hidden process, so what we observe (returns) is a noisy function of a latent state (log-variance) that we cannot directly measure. Similar structure appears in limit order book dynamics and macroeconomic regime shifts. These problems are naturally cast as continuous state-space models (SSMs), which is the focus of this document.

The setup is as following: we observe a time series $\{z_k\}$, $k = 1, 2, \dots, T$. The real driver is a hidden state $\{x_k\}$ that we cannot observe directly. The most general SSM writes:

$$\begin{aligned} x_k &= f(x_{k-1}, w_k) \\ z_k &= h(x_k, u_k) \end{aligned} \tag{1}$$

where w_k and u_k are mutually and temporally uncorrelated noises, f governs state evolution, and h maps states to observations. We only ever see z_k . The state x_k may have the same dimension as z_k or higher — the latter case (partial observation) makes inference harder because information is lost in the mapping h .

A model has two components: a functional form and its parameters. In practice both are unknown. What we do is make an ansatz about the functional form, fit the parameters to data, and treat that ansatz as our best current understanding until a better one comes along. The observations themselves are agnostic to the model choice.

To develop and test numerical methods rigorously, we work in controlled experiments: fix a model and its parameters, generate synthetic data, and check whether our algorithms recover the known ground truth. This is the scope of this project. The two core tasks are **filtering** — estimating the hidden state x_k from observations — and **Bayesian inference** — estimating the model parameters θ from the same observations.

1.1.1 Filtering

Having a good model is only half the battle; we also need a reliable implementation. I use synthetic models first because the truth is known. Given the state evolution f and observation model h , the goal is to estimate the hidden state x_k via the filtering distribution $p(x_k | z_{1:k})$. This is obtained recursively:

$$\begin{aligned} \textbf{Prediction:} \quad p(x_k | z_{1:k-1}) &= \int p(x_k | x_{k-1}) p(x_{k-1} | z_{1:k-1}) dx_{k-1} \\ \textbf{Update:} \quad p(x_k | z_{1:k}) &= \frac{p(z_k | x_k) p(x_k | z_{1:k-1})}{p(z_k | z_{1:k-1})} \end{aligned} \tag{2}$$

In financial applications (and most real-world settings), we do not aim to recover an exact deterministic value of x_k . Instead, the useful quantities are typically the mean m_k and covariance P_k of the filtering distribution. The filtering problem is generally a high-dimensional path integral and is intractable except in special cases (e.g., linear Gaussian SSMs). Different filtering methods (EKF, UKF, particle filters, etc.) use distinct approximations and mathematical frameworks. The filter has one job: approximate the posterior and stay numerically stable.

1.1.2 Bayesian Inference

Once we have a reliable filter, we can use it to estimate the model parameters θ . We keep the functional forms of f and h fixed and treat their parameters as unknowns to be learned. A good θ should make the observed sequence probable. This means maximizing the marginal likelihood $p_\theta(z_{1:T})$, or equivalently minimizing the negative log-likelihood:

$$L_{\text{MLE}}(\theta) = - \sum_{k=1}^T \log p_\theta(z_k | z_{1:k-1}). \tag{3}$$

This is MLE. In a Bayesian setting we also have a prior $p(\theta)$ encoding any hard knowledge about the parameters — for instance, a variance must be positive, or a correlation must lie in $(-1, 1)$. We then minimize:

$$L_{\text{MAP}}(\theta) = - \sum_{k=1}^T \log p_{\theta}(z_k \mid z_{1:k-1}) - \log p(\theta), \quad (4)$$

which gives MAP estimation. MAP is strictly more informative than MLE: the prior rules out parameter regions that are physically unreasonable or numerically dangerous, so the optimizer cannot wander into them.

1.2 Document Roadmap

We cover the two tasks in sequence.

1. First, filtering. We start with the Kalman filter on linear Gaussian models — the only case where the prediction and update integrals are analytically tractable — and use it as a baseline throughout. We then cover the nonlinear extensions: EKF and UKF, which linearize or approximate the model to stay within the Gaussian framework. Next, particle-based methods, which abandon the Gaussian assumption entirely and represent the filtering distribution as a weighted set of samples. Within that family we discuss the Bootstrap Particle Filter (BPF), the EDH and LEDH flow filters (which move particles via a continuous transport rather than reweighting them), and their invertible variants.
2. Second, Bayesian inference. Once the filter is differentiable, we can backpropagate through it to learn θ . We cover MAP estimation as a diagnostic baseline and HMC for full posterior sampling. The central challenge is making the resampling step differentiable so that gradients can flow back to θ . We discuss stop-gradient systematic resampling and soft resampling as reference points, but the main target is OT resampling and its neural-accelerated variant, which provides the richest gradient signal and is the primary method we will develop.

2 Kalman Filter

2.1 Overview

Linear Gaussian models are the easiest state space models. Take the following 1-d dimensional model for example,

$$\begin{aligned} x_k &= f x_{k-1} + w_{k-1}, & w_{k-1} &\sim \mathcal{N}(0, Q) \\ z_k &= h x_k + v_k, & v_k &\sim \mathcal{N}(0, R) \end{aligned} \quad (5)$$

where f and h are constant. Of course to ensure the stability of the sequence, both of them has to be smaller or equal to one. This problem can be exactly solved as we will later demonstrate that the distribution for x_k and z_k will stay as Gaussian through out the process and all quantities are analytically computable. The corresponding algorithm that is derived for linear gaussian models is the Kalman Filter (KF). In real life, we are not so lucky, prediction or observation model could become non-linear,

$$\begin{aligned} f x_{k-1} &\rightarrow \mathbf{f}(x_{k-1}) \\ h x_k &\rightarrow \mathbf{h}(x_k) \end{aligned} \quad (6)$$

here I am using $\mathbf{f}/\mathbf{h}$ to indicate they are generic functions so that they are distinguished from constants f/h . It is also possible that the noises are non-Gaussian, or even non-additive. A good example of the later case is stochastic volatility model,

$$x_k = a x_{k-1} + w_{k-1}, \quad w_{k-1} \sim \mathcal{N}(0, 1) \quad (7)$$

$$z_k = b \exp(x_k/2) \cdot v_k, \quad v_k \sim \mathcal{N}(0, 1) \quad (8)$$

where we have pure multiplicative noise for the observation process, and a, b are just some constant. There is really no one-fits-all filter algorithm for the most generic problems. We will start with additive gaussian noise but non-linear observation function, and understand two basic non-linear generalization of Kalman filter, namely Extended Kalman Filter (EKF) and Unscented Kalman Filter (UKF). EKF and UKF will be very important for later applications.

2.2 Algorithm

Recall the most general framework for the filter,

$$\begin{aligned} \textbf{Prediction:} \quad p(x_k | z_{1:k-1}) &= \int p(x_k | x_{k-1}) p(x_{k-1} | z_{1:k-1}) dx_{k-1} \\ \textbf{Update:} \quad p(x_k | z_{1:k}) &= \frac{p(z_k | x_k) p(x_k | z_{1:k-1})}{p(z_k | z_{1:k-1})} \end{aligned} \quad (9)$$

Because the process are linear gaussian, the distribution $p(x_k | z_{1:k-1})$ and $p(x_k | z_{1:k})$ are Gaussian for all time k . To Characterize Gaussian distributions, all we need is the mean and variance. For the prediction and update processes, we keep track of the mean and variances separately.

- m_t^- : predicted mean (prior), P_t^- : predicted variance (prior)
- m_t : updated mean (posterior), P_t : updated variance (posterior)

For any filter, we need an initial condition,

$$p(x_0) = \mathcal{N}(x_0 | m_0, P_0). \quad (10)$$

A process is *stationary* if its statistical properties — mean, variance — do not change over time. For the linear state evolution $x_k = a x_{k-1} + w_{k-1}$, this happens when $|a| < 1$: the state is repeatedly pulled back toward zero, and after many steps the variance settles to a fixed value. We can find this value by asking: if the variance at step k equals the variance at step $k - 1$ (call it σ^2), what must σ^2 be?

$$\sigma^2 = a^2 \sigma^2 + Q \quad \Rightarrow \quad \sigma^2 = \frac{Q}{1 - a^2}. \quad (11)$$

This is the variance of the stationary distribution. If the process has been running long before we start observing it — the usual assumption — then at time $k = 0$ the state is already drawn from this

equilibrium. The natural initial condition is therefore $m_0 = 0$ and $P_0 = Q/(1 - a^2)$. Starting here avoids a transient period where the filter is far from the true distribution.

When $|a| \geq 1$, there is no equilibrium. The state can drift arbitrarily far from zero and the variance grows without bound over time — the process is *non-stationary*. There is no principled variance to start from, so we express complete uncertainty: $m_0 = 0$ and $P_0 = \kappa$ with κ very large. This is a *diffuse prior* — it says we have no idea where the state is at time zero. As observations arrive, the filter rapidly pulls P_k down toward something sensible, so the exact value of κ does not matter as long as it is large enough to be non-informative.

Initial conditions as a cross-cutting principle. The 1D stationary vs. diffuse distinction above generalises to every model in this report and is worth stating once so later sections can refer back to it. For a multi-dimensional linear state $\mathbf{x}_k = \mathbf{F}\mathbf{x}_{k-1} + \boldsymbol{\omega}_{k-1}$ with stable $\mathbf{F}$ (spectral radius < 1), the stationary covariance is the unique solution of the discrete Lyapunov equation $\boldsymbol{\Sigma}_0 = \mathbf{F}\boldsymbol{\Sigma}_0\mathbf{F}^\top + \mathbf{Q}$, and the natural prior is $\mathbf{x}_0 \sim \mathcal{N}(\mathbf{0}, \boldsymbol{\Sigma}_0)$. We use this for the linear Gaussian benchmarks and for the 1D stochastic volatility model, where $|\alpha| < 1$ gives $\Sigma_0 = \sigma^2/(1 - \alpha^2)$ directly. For tracking models whose dynamics are not contractive — the range-bearing and acoustic-tracking problems — there is no Lyapunov solution and we fall back on a physically motivated prior: a mean $\boldsymbol{\mu}_0$ inside the typical observation footprint and an isotropic covariance $\boldsymbol{\Sigma}_0 = I$ large enough to cover plausible starting positions. On range-bearing, the canonical choice in this report is $\boldsymbol{\mu}_0 = (5, 5)^\top$ with $\boldsymbol{\Sigma}_0 = I_2$, as stated in §3.3; this keeps the initial bearing Jacobian away from the sensor singularity. Individual model sections below give specific numerical values but refer back to this paragraph rather than re-deriving the principle.

Now we compute the update rules for the prediction process, given the mean and variance from the previous update step (m_{k-1}, P_{k-1}) , which characterize $p(x_{k-1} | z_{1:k-1})$. We have, for one realization of the prediction

$$x_k^- = f x_{k-1} + \omega_k \quad (12)$$

It is obvious that,

$$\begin{aligned} \mathbb{E}[x_k^- | 1 : z_{k-1}] &= \mathbb{E}[f x_{k-1} + \omega_k | 1 : z_{k-1}] \\ &= f \mathbb{E}[x_{k-1} | 1 : z_{k-1}] \\ &= f m_{k-1} \end{aligned} \quad (13)$$

and for the variation, we just stick to the definition,

$$\begin{aligned} P_k^- &= \text{Var}[x_k | z_{1:k-1}] = \mathbb{E}[x_k^2 | z_{1:k-1}] - (\mathbb{E}[x_k | z_{1:k-1}])^2 \\ &= f^2 \mathbb{E}[x_{k-1}^2 | z_{1:k-1}] - f^2 (\mathbb{E}[x_{k-1} | z_{1:k-1}])^2 + Q \\ &= f^2 P_{k-1} + Q \end{aligned} \quad (14)$$

where the first contribution is just the prediction noise and the second term is the input noise scale by the determinist part of the evolution. After prediction, we need to do correction based on the new observation z_k . By Bayes' rule:

$$p(x_k | z_{1:k}) = \frac{p(z_k | x_k) p(x_k | z_{1:k-1})}{p(z_k | z_{1:k-1})} \propto p(z_k | x_k) p(x_k | z_{1:k-1}). \quad (15)$$

For the mean and variance, we don't have to need the normalization constant. Let's take log on both side.

$$-\frac{(x_k - m_k)^2}{2P_k} = -\frac{(z_k - h x_k)^2}{2R} - \frac{(x_k - m_k^-)^2}{2P_k^-} \quad (16)$$

from this, it is clear to read of the coefficient of x_k^2 to get,

$$\frac{1}{P_k} = \frac{h^2}{R} + \frac{1}{P_k^-} \quad (17)$$

equivalently,

$$P_k = \frac{R P_k^-}{h^2 P_k^- + R} = (1 - k_k h) P_k^- \quad (18)$$

For simplicity, we define the **Kalman gain**:

$$K_k = \frac{P_k^- h}{h^2 P_k^- + R} = P_k^- h / S_k \quad (19)$$

where we also introduce **innovation covariance** $S_k = \frac{1}{h^2 P_k^- + R}$. After some algebra:

$$m_k = m_k^- + K_k v_k \quad (20)$$

where we denote the **innovation** $v_k = z_k - h m_k^-$, which is the difference between observed data and observation of the predicted mean. Let us summarize the algorithm,

Algorithm 1 Kalman Filter (1-d, additive noise)

Require: Model: $x_k = f x_{k-1} + \omega_{k-1}$, $z_k = h x_k + v_k$, $\omega_{k-1} \sim \mathcal{N}(0, Q)$, $v_k \sim \mathcal{N}(0, R)$

Require: Initial conditions: m_0 , P_0

```

1: for  $k = 1, 2, 3, \dots$  do
2:   Prediction:
3:    $m_k^- \leftarrow f m_{k-1}$ 
4:    $P_k^- \leftarrow f^2 P_{k-1} + Q$ 
5:   Update:
6:    $\nu_k \leftarrow z_k - h m_k^-$ 
7:    $S_k \leftarrow h^2 P_k^- + R$ 
8:    $K_k \leftarrow P_k^- h / S_k$ 
9:    $m_k \leftarrow m_k^- + K_k \nu_k$ 
10:   $P_k \leftarrow (1 - K_k h) P_k^-$ 
11: end for
```

Algorithm 2 Kalman Filter (vector, additive noise)

Require: Model: $\mathbf{x}_k = \mathbf{F} \mathbf{x}_{k-1} + \boldsymbol{\omega}_{k-1}$, $\mathbf{z}_k = \mathbf{H} \mathbf{x}_k + \mathbf{v}_k$, $\boldsymbol{\omega}_{k-1} \sim \mathcal{N}(\mathbf{0}, \mathbf{Q})$, $\mathbf{v}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{R})$

Require: Initial conditions: $\mathbf{m}_0$, $\mathbf{P}_0$

```

1: for  $k = 1, 2, 3, \dots$  do
2:   Prediction:
3:    $\mathbf{m}_k^- \leftarrow \mathbf{F} \mathbf{m}_{k-1}$ 
4:    $\mathbf{P}_k^- \leftarrow \mathbf{F} \mathbf{P}_{k-1} \mathbf{F}^\top + \mathbf{Q}$ 
5:   Update:
6:    $\boldsymbol{\nu}_k \leftarrow \mathbf{z}_k - \mathbf{H} \mathbf{m}_k^-$ 
7:    $\mathbf{S}_k \leftarrow \mathbf{H} \mathbf{P}_k^- \mathbf{H}^\top + \mathbf{R}$ 
8:    $\mathbf{K}_k \leftarrow \mathbf{P}_k^- \mathbf{H}^\top \mathbf{S}_k^{-1}$ 
9:    $\mathbf{m}_k \leftarrow \mathbf{m}_k^- + \mathbf{K}_k \boldsymbol{\nu}_k$ 
10:   $\mathbf{P}_k \leftarrow (\mathbf{I} - \mathbf{K}_k \mathbf{H}) \mathbf{P}_k^-$ 
11: end for
```

2.3 Numerical stability analysis for Kalman filter

If we implement it as it's derived, kalman filter might be numerically unstable. Higher dimension has more failure mode so let us start with 1-d. When the observation noise becomes very small $R \sim 0$, we have $S_k \sim \frac{1}{h^2 P_k^-}$, which leads to,

$$\lim_{R \rightarrow 0} K = \lim_{R \rightarrow 0} \frac{P^- h}{h^2 P^- + R} = \frac{1}{h} \quad (21)$$

With the standard update: $P^+ = (1 - Kh)P^- \approx 0$. But this is very dangerous, because we should always have positive P^+ . With the accumulation of small roundoff errors, $(1 - Kh)$ can become negative. But with Joseph form,

$$P^+ = (I - Kh)P^-(I - Kh) + K^2 R \sim \frac{R}{h^2} \quad (22)$$

Clearly, Joseph form will resolve the issue in 1-d. Joseph form doesn't cure all problems, but it will remove one potential failure in the numerical implementation. We can prove these update formula are mathematically equivalent.

$$\begin{aligned}
\mathbf{P}_k &= (\mathbf{I} - \mathbf{KH}) \mathbf{P}^- (\mathbf{I} - \mathbf{KH})^\top + \mathbf{K} \mathbf{R} \mathbf{K}^\top \\
&= \mathbf{P}^- - \mathbf{KHP}^- - \mathbf{P}^- \mathbf{H}^\top \mathbf{K}^\top + \mathbf{KHP}^- \mathbf{H}^\top \mathbf{K}^\top + \mathbf{K} \mathbf{R} \mathbf{K}^\top \\
&= \mathbf{P}^- - \mathbf{KHP}^- - \mathbf{P}^- \mathbf{H}^\top \mathbf{K}^\top + \mathbf{K} \underbrace{(\mathbf{HP}^- \mathbf{H}^\top + \mathbf{R})}_{=\mathbf{S}} \mathbf{K}^\top.
\end{aligned}$$

Using $\mathbf{K} = \mathbf{P}^- \mathbf{H}^\top \mathbf{S}^{-1} \implies \mathbf{KS} = \mathbf{P}^- \mathbf{H}^\top \implies \mathbf{KSK}^\top = \mathbf{P}^- \mathbf{H}^\top \mathbf{K}^\top$:

$$\begin{aligned}
\mathbf{P}_k &= \mathbf{P}^- - \mathbf{KHP}^- - \mathbf{P}^- \mathbf{H}^\top \mathbf{K}^\top + \mathbf{P}^- \mathbf{H}^\top \mathbf{K}^\top \\
&= \mathbf{P}^- - \mathbf{KHP}^- = (\mathbf{I} - \mathbf{KH}) \mathbf{P}^-.
\end{aligned}$$

Additionally, we will have potential instability when we try to take the inverse of $\mathbf{S}_k^{-1}$. Let's not get into the details of what would cause the failure of $\mathbf{S}_k^{-1}$. But when it fails, it is usually due to that $\mathbf{S}_k$ having large condition number. Let us denote the eigenvalues of $\mathbf{S}_k$ are $\{\lambda_1, \dots, \lambda_d\}$, where d is the dimension of the state space. When $\kappa(\mathbf{S}_k) \gg 1$, small round off error in $\mathbf{S}_k$, which was not exactly and obtained through numerical calculation will leads to huge difference in the inverse, this means that $\mathbf{S}_k^{-1}$ is unreliable.

2.4 The limitation and usefulness of Kalman filter

The derivation above relies entirely on $z_k = h x_k + v_k$ with h constant (or a matrix $\mathbf{H}$ in the vector case). That is what lets us stay Gaussian through the update step and write a closed-form recursion for the mean and variance. The moment the observation function becomes genuinely non-linear, we lose that, and it is not really a fixable problem at the Kalman filter level — there is no principled way to plug a non-linear $h(x)$ into the update equations. People do it anyway by replacing $h(x)$ with some linear surrogate, but at that point we are not running the Kalman filter on a non-linear model, we are running it on a linearized model we picked ourselves, and the answer is only as good as that choice. Since in practice almost every model we care about is non-linear, we really do need a generalization, and in the next section we will discuss two of them: the Extended Kalman Filter (EKF) and the Unscented Kalman Filter (UKF).

None of this makes the Kalman filter useless. On the contrary, being exact on linear Gaussian models is exactly what we want for one important use later on. When we build the differentiable particle filter and run HMC on top of it in Section 6, we need a way to check that the HMC pipeline itself is correct before we trust it on anything harder. The 1-D linear Gaussian model is the obvious test: the Kalman filter gives us the exact marginal log-likelihood, so it gives us the exact posterior over whatever parameter we are estimating. If HMC on the same model with the same data recovers that posterior, we know the pipeline is sound; if it does not, the problem is in HMC, not in the filter. We will come back to this cross-check in Section 6.5 when we benchmark the differentiable pipeline.

Demonstration on a 1D linear Gaussian. To make the “Kalman is exact for linear Gaussian” claim concrete, we run the filter on the exact 1D model that will reappear in the HMC validation in §6.5. The dynamics are $x_k = 0.95 x_{k-1} + w_k$ with $w_k \sim \mathcal{N}(0, 0.25)$ ($B = 0.5$), observation $z_k = x_k + v_k$ with $v_k \sim \mathcal{N}(0, 0.09)$ ($D = 0.3$), prior $\mathcal{N}(0, \Sigma_0)$ with Σ_0 solved from the discrete Lyapunov equation following §2.2, seed 42, $T = 100$ time steps, and 64-bit precision. The Kalman gain converges to its steady-state value $K \approx 0.777$ within the first five time steps, confirming the Riccati recursion has reached its fixed point. Figure 1 shows the resulting trajectory.

We report two metrics throughout. The *root-mean-square error* (RMSE) of the filter's posterior mean against the true state trajectory,

$$\text{RMSE} = \sqrt{\frac{1}{T} \sum_{k=1}^T \|\mathbf{m}_k - \mathbf{x}_k\|^2}, \quad (23)$$

measures how close the filter's point estimate is to the true latent state. It is in the units of the state. The *marginal log-likelihood* $\log p(z_{1:T} \mid \text{model}) = \sum_{k=1}^T \log p(z_k \mid z_{1:k-1})$ measures how well the filter's one-step-ahead predictive distribution absorbs the observation sequence and is the natural score for posterior

calibration, not mean accuracy. RMSE is in latent-state units, log-likelihood is in observation-space units, and the two are not in general comparable across models with different observation functions.

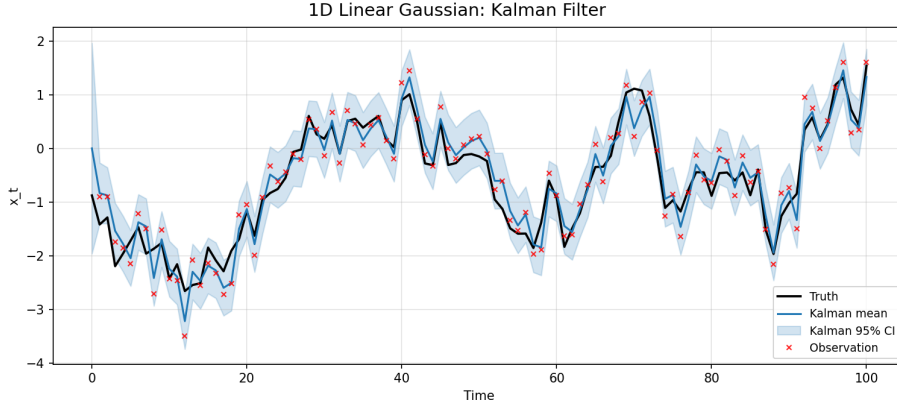


Figure 1: Kalman filter on the 1D linear Gaussian model ($F = 0.95$, $B = 0.5$, $H = 1.0$, $D = 0.3$, $T = 100$, seed 42). True state in black; Kalman posterior mean in blue with the 95% credible band; observations shown as red crosses. RMSE = 0.269, log-likelihood = -104.80 .

The run takes 1.03 seconds on CPU (10.3 ms per time step, incremental peak memory 2.1 MB, GPU unused); for a 1D state the entire filter reduces to two scalar updates per step, so the cost is essentially measurement noise in the timing — a useful reference point against which the particle-filter wall times in Tables 2–5 become meaningful.

What this plot is for. This is not a benchmark of Kalman’s accuracy. The derivation in §2.2 already shows the Kalman filter is analytically exact for linear Gaussian, so there is no approximation error to measure. The RMSE of 0.269 is the sampling-level discrepancy between the filter’s posterior mean and the true state realisation for this particular seed, and is bounded below by the posterior standard deviation the filter itself reports. What the plot is for is *pipeline sanity*: data generation, filter invocation, state sampling from the prior, covariance tracking, Riccati convergence, metric collection, and I/O all run end-to-end without error and produce numbers consistent with the analytic reference. This is the minimum bar every downstream benchmark in §4 and §6 must clear — and the analytic reference itself is the object §6.5 will use to validate HMC: if HMC on this same model recovers the Kalman posterior over the parameters, the differentiable pipeline is sound; if it does not, the fault lies in HMC, not in the filter or the test harness.

3 EKF and UKF

Previously, we discussed why the Kalman filter is not applicable to non-linear models. Here we introduce two principled generalizations, the Extended Kalman Filter (EKF) and the Unscented Kalman Filter (UKF) [1], that can handle non-linearity to some extent. They are both designed to incorporate nonlinear corrections in the observation model, namely, $\mathbf{h}(x)$, with additive noises. In the literature, there are improved versions known as augmented EKF/UKF [1] that can handle non-additive noise but these are beyond the scope of the current document. The only non-additive noise model we will run into is the 1-d Stochastic volatility model, and we will use an approximation to turn it into additive noise model and additional improvement on EKF/UKF is overkill for this model.

3.1 EKF

EKF is a nonlinear improvement on KF, in the sense that we still want to somehow preserve most of the KF algorithm. What non-linearity breaks is the linear structure of the model.

$$\begin{aligned}\hat{\mathbf{F}}\mathbf{x}_{k-1} &\rightarrow \mathbf{f}(\mathbf{x}_{k-1}) \\ \hat{\mathbf{H}}\mathbf{x}_k &\rightarrow \mathbf{h}(\mathbf{x}_{k-1})\end{aligned}\tag{24}$$

but what if we expand the prediction and observation model,

$$\begin{aligned}\mathbf{f}(\mathbf{x} - \mathbf{m}_{k-1}) &\approx \mathbf{f}(\mathbf{m}_{k-1}) + \left. \frac{\partial \mathbf{f}}{\partial \mathbf{x}} \right|_{\mathbf{m}_{k-1}} (\mathbf{x} - \mathbf{m}_{k-1}) + \mathcal{O}((\mathbf{x} - \mathbf{m}_{k-1})^2) \\ \mathbf{h}(\mathbf{x} - \mathbf{m}_k^-) &\approx \mathbf{h}(\mathbf{m}_k^-) + \left. \frac{\partial \mathbf{h}}{\partial \mathbf{x}} \right|_{\mathbf{m}_k^-} (\mathbf{x} - \mathbf{m}_k^-) + \mathcal{O}((\mathbf{x} - \mathbf{m}_{k-1})^2)\end{aligned}\tag{25}$$

by weak nonlinearity, I meant when the second derivative term can be ignored. We would expect such linear approximation to fail when the following two terms are comparable.

$$\left. \frac{\partial \mathbf{h}}{\partial \mathbf{x}} \right|_{\mathbf{m}_k^-} (\mathbf{x} - \mathbf{m}_k^-) \sim \left. \frac{\partial^2 \mathbf{h}}{\partial \mathbf{x}^2} \right|_{\mathbf{m}_k^-} (\mathbf{x} - \mathbf{m}_{k-1})^2\tag{26}$$

When linear approximation holds, we can use the Kalman filter algorithm by adding time dependence to the evolution and observation matrices.

$$\begin{aligned}\mathbf{F}_k &\leftarrow \mathbf{f}'(\mathbf{m}_{k-1}) \\ \mathbf{H}_k &\leftarrow \mathbf{h}'(\mathbf{m}_k^-)\end{aligned}\tag{27}$$

Algorithm 3 Extended Kalman Filter (EKF)

Require: Model: $\mathbf{x}_k = \mathbf{f}(\mathbf{x}_{k-1}) + \boldsymbol{\omega}_{k-1}$, $\mathbf{z}_k = \mathbf{h}(\mathbf{x}_k) + \mathbf{v}_k$, $\boldsymbol{\omega}_{k-1} \sim \mathcal{N}(\mathbf{0}, \mathbf{Q})$, $\mathbf{v}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{R})$

Require: Initial: $\mathbf{m}_0, \mathbf{P}_0$

```

1: for  $k = 1, 2, \dots$  do
2:   Prediction:
3:    $\mathbf{m}_k^- \leftarrow \mathbf{f}(\mathbf{m}_{k-1})$ 
4:    $\mathbf{F} \leftarrow \left. \frac{\partial \mathbf{f}}{\partial \mathbf{x}} \right|_{\mathbf{m}_{k-1}}$ 
5:    $\mathbf{P}_k^- \leftarrow \mathbf{F} \mathbf{P}_{k-1} \mathbf{F}^\top + \mathbf{Q}$ 
6:   Update :
7:    $\hat{\mathbf{z}} \leftarrow \mathbf{h}(\mathbf{m}_k^-)$ 
8:    $\mathbf{H} \leftarrow \left. \frac{\partial \mathbf{h}}{\partial \mathbf{x}} \right|_{\mathbf{m}_k^-}$ 
9:    $\boldsymbol{\nu} \leftarrow \mathbf{z}_k - \hat{\mathbf{z}}$  (innovation)
10:   $\mathbf{S} \leftarrow \mathbf{H} \mathbf{P}_k^- \mathbf{H}^\top + \mathbf{R}$ 
11:   $\mathbf{K} \leftarrow \mathbf{P}_k^- \mathbf{H}^\top \mathbf{S}^{-1}$  (Kalman gain via Cholesky solve)
12:   $\mathbf{m}_k \leftarrow \mathbf{m}_k^- + \mathbf{K} \boldsymbol{\nu}$ 
13:   $\mathbf{P}_k \leftarrow (\mathbf{I} - \mathbf{K} \mathbf{H}) \mathbf{P}_k^- (\mathbf{I} - \mathbf{K} \mathbf{H})^\top + \mathbf{K} \mathbf{R} \mathbf{K}^\top$  (Joseph form)
14: end for
```

3.2 UKF

The EKF approximates the nonlinear model by linearization, which requires computing the Jacobians $\mathbf{F}$ and $\mathbf{H}$. The UKF takes a different route: instead of approximating the *function*, it approximates the *distribution* that results from pushing a Gaussian through the function. The key observation is that the Kalman gain can be written without any reference to $\mathbf{H}$. In the linear case, $\mathbf{K} = \mathbf{P}\mathbf{H}^\top\mathbf{S}^{-1}$ and $\mathbf{z}_k^- = \mathbf{H}\mathbf{x}_k^- + \mathbf{v}_k$ with $\mathbf{v}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{R})$ independent of $\mathbf{x}_k^-$. The cross-covariance between the predicted state and predicted observation is

$$\begin{aligned}\text{Cov}(\mathbf{x}_k^-, \mathbf{z}_k^-) &= \mathbb{E}[(\mathbf{x}_k^- - \mathbf{m}_k^-)(\mathbf{z}_k^- - \mathbf{H}\mathbf{m}_k^-)^\top] \\ &= \mathbb{E}[(\mathbf{x}_k^- - \mathbf{m}_k^-)(\mathbf{H}(\mathbf{x}_k^- - \mathbf{m}_k^-) + \mathbf{v}_k)^\top] \\ &= \mathbb{E}[(\mathbf{x}_k^- - \mathbf{m}_k^-)(\mathbf{x}_k^- - \mathbf{m}_k^-)^\top] \mathbf{H}^\top \\ &= \mathbf{P}_k^- \mathbf{H}^\top,\end{aligned}\tag{28}$$

where the cross term $\mathbb{E}[(\mathbf{x}_k^- - \mathbf{m}_k^-)\mathbf{v}_k^\top] = 0$ by independence. So $\mathbf{P}_k^- \mathbf{H}^\top = \text{Cov}(\mathbf{x}_k^-, \mathbf{z}_k^-) \equiv \mathbf{P}_{xz}$. For the innovation covariance,

$$\begin{aligned}\text{Cov}(\mathbf{z}_k^-, \mathbf{z}_k^-) &= \mathbb{E}[(\mathbf{H}(\mathbf{x}_k^- - \mathbf{m}_k^-) + \mathbf{v}_k)(\mathbf{H}(\mathbf{x}_k^- - \mathbf{m}_k^-) + \mathbf{v}_k)^\top] \\ &= \mathbf{H} \mathbb{E}[(\mathbf{x}_k^- - \mathbf{m}_k^-)(\mathbf{x}_k^- - \mathbf{m}_k^-)^\top] \mathbf{H}^\top + \mathbb{E}[\mathbf{v}_k \mathbf{v}_k^\top] \\ &= \mathbf{H} \mathbf{P}_k^- \mathbf{H}^\top + \mathbf{R} = \mathbf{S},\end{aligned}\tag{29}$$

where the cross terms again vanish by independence. So $\text{Cov}(\mathbf{z}_k^-, \mathbf{z}_k^-) \equiv \mathbf{P}_{zz} = \mathbf{S}$. The Kalman gain therefore takes the equivalent form

$$\mathbf{K} = \mathbf{P}_{xz} \mathbf{S}^{-1},\tag{30}$$

and this expression makes no assumption about linearity and is "blind" to the functional form of the models. If we can estimate $\mathbf{P}_{xz}$ and $\mathbf{S}$ for a nonlinear $\mathbf{h}$, we can run the Kalman update as-is.

The question is how to compute these covariances when $\mathbf{h}$ is nonlinear. EKF evaluates them analytically via the Jacobian and discards higher-order terms. UKF instead draws $2d + 1$ *sigma points* deterministically from the predicted Gaussian $\mathcal{N}(\mathbf{m}_k^-, \mathbf{P}_k^-)$, propagates each through $\mathbf{h}$, and computes the covariances as weighted sums:

$$\hat{\mathbf{z}} = \sum_j W_m^{(j)} \mathbf{h}(\chi_j),\tag{31}$$

$$\mathbf{S} = \sum_j W_c^{(j)} (\mathbf{h}(\chi_j) - \hat{\mathbf{z}})(\mathbf{h}(\chi_j) - \hat{\mathbf{z}})^\top + \mathbf{R},\tag{32}$$

$$\mathbf{P}_{xz} = \sum_j W_c^{(j)} (\chi_j - \mathbf{m}_k^-)(\mathbf{h}(\chi_j) - \hat{\mathbf{z}})^\top.\tag{33}$$

The sigma points are chosen so that their weighted mean and covariance exactly match the Gaussian $\mathcal{N}(\mathbf{m}_k^-, \mathbf{P}_k^-)$, which guarantees the approximation is at least second-order accurate (capturing the curvature of $\mathbf{h}$ that EKF misses). No Jacobian is required. With $\mathbf{P}_{xz}$ and $\mathbf{S}$ in hand, the rest of the update is identical to the Kalman filter.

Here d is the *state dimension*. The number of sigma points $2d + 1$ is fixed by the method — one at the mean, d pairs placed symmetrically along each column of $\sqrt{(d + \lambda)\mathbf{P}}$.

The three tuning parameters α, β, κ all feed into the composite scaling parameter $\lambda = \alpha^2(d + \kappa) - d$. The sigma point displacement is $h = \sqrt{(d + \lambda)\mathbf{P}}$, so λ directly sets the spread. $\alpha \in (0, 1]$ is the primary knob. The typical default $\alpha = 10^{-3}$ with $\kappa = 0$ gives $\lambda \approx -d$, so $(d + \lambda) \approx \alpha^2 d \approx 0$ — the sigma points collapse nearly to the mean and the filter degenerates toward a linearization. κ is minor and usually left at zero.

β enters only in $W_c^{(0)} = W_m^{(0)} + (1 - \alpha^2 + \beta)$, adjusting how much the central sigma point contributes to the covariance. To see why $\beta = 2$ is optimal for Gaussians, consider the simplest case: $d = 1$, $f(x) = x^2$, $x \sim \mathcal{N}(0, P)$. The true variance is $\text{Var}[x^2] = 3P^2 - P^2 = 2P^2$ (Gaussian fourth moment $\mathbb{E}[x^4] = 3P^2$). With $\kappa = 0$, let $h^2 = (1 + \lambda)P$; the three sigma points give $f(\chi_j) \in \{0, h^2, h^2\}$. The UKF mean $\hat{z} = 2W_m^{(1)}h^2 = P$ is already exact. The covariance estimate is

$$\widehat{\text{Var}} = W_c^{(0)}(0 - P)^2 + 2W_c^{(1)}(h^2 - P)^2 = \left(W_c^{(0)} + \frac{\lambda^2}{1 + \lambda}\right) P^2.\tag{34}$$

Setting this equal to $2P^2$ requires $W_c^{(0)} + \lambda^2/(1 + \lambda) = 2$. Substituting $W_c^{(0)} = \lambda/(1 + \lambda) + (1 - \alpha^2 + \beta)$ and $\lambda = \alpha^2 - 1$ (for $d = 1, \kappa = 0$):

$$(\alpha^2 - 1) + (1 - \alpha^2 + \beta) = 2 \implies \beta = 2. \quad (35)$$

The α^2 terms cancel exactly, so $\beta = 2$ holds for any α when $d = 1, \kappa = 0$. For $d > 1$ the cancellation is approximate, but $\beta = 2$ remains a good default.

Algorithm 4 Unscented Kalman Filter (UKF)

Require: Same model as EKF. UKF parameters: $\alpha, \beta, \kappa; \lambda = \alpha^2(d + \kappa) - d$

Require: UKF weights: $W_m^{(0)} = \frac{\lambda}{d + \lambda}, W_c^{(0)} = W_m^{(0)} + 1 - \alpha^2 + \beta, W_m^{(j)} = W_c^{(j)} = \frac{1}{2(d + \lambda)}$ for $j = 1, \dots, 2d$

```

1: for  $k = 1, 2, \dots$  do
2:   Prediction:
3:   Generate  $2d + 1$  sigma points:  $\chi_0 = \mathbf{m}_{k-1}$ 
4:    $\chi_j = \mathbf{m}_{k-1} + \left[ \sqrt{(d + \lambda)\mathbf{P}_{k-1}} \right]_j, \quad j = 1, \dots, d$ 
5:    $\chi_{d+j} = \mathbf{m}_{k-1} - \left[ \sqrt{(d + \lambda)\mathbf{P}_{k-1}} \right]_j, \quad j = 1, \dots, d$ 
6:   Propagate:  $\chi_j^* \leftarrow \mathbf{f}(\chi_j)$  for all  $j$ 
7:    $\mathbf{m}_k^- \leftarrow \sum_{j=0}^{2d} W_m^{(j)} \chi_j^*$ 
8:    $\mathbf{P}_k^- \leftarrow \sum_{j=0}^{2d} W_c^{(j)} (\chi_j^* - \mathbf{m}_k^-)(\chi_j^* - \mathbf{m}_k^-)^\top + \mathbf{Q}$ 
9:   Update (for each particle  $i$ ):
10:  Regenerate sigma points from  $(\mathbf{m}_k^-, \mathbf{P}_k^-)$ 
11:  Propagate through observation:  $\gamma_j \leftarrow \mathbf{h}(\chi_j)$ 
12:   $\hat{\mathbf{z}} \leftarrow \sum_{j=0}^{2d} W_m^{(j)} \gamma_j$ 
13:   $\mathbf{S} \leftarrow \sum_{j=0}^{2d} W_c^{(j)} (\gamma_j - \hat{\mathbf{z}})(\gamma_j - \hat{\mathbf{z}})^\top + \mathbf{R}$ 
14:   $\mathbf{P}_{xz} \leftarrow \sum_{j=0}^{2d} W_c^{(j)} (\chi_j - \mathbf{m}_k^-)(\gamma_j - \hat{\mathbf{z}})^\top$ 
15:   $\mathbf{K} \leftarrow \mathbf{P}_{xz} \mathbf{S}^{-1}$  (Kalman gain via Cholesky solve)
16:   $\mathbf{m}_k \leftarrow \mathbf{m}_k^- + \mathbf{K}(\mathbf{z}_k - \hat{\mathbf{z}})$ 
17:   $\mathbf{P}_k \leftarrow \mathbf{P}_k^- - \mathbf{K} \mathbf{P}_{xz}^\top - \mathbf{P}_{xz} \mathbf{K}^\top + \mathbf{K} \mathbf{S} \mathbf{K}^\top$ 
18: end for
```

3.3 EKF/UKF on non-linear models

We define the Range bearing model and 1-d Stochastic volatility model here. This is the first time they appear and they will also be used in later sections.

Range bearing model. The state $\mathbf{x}_t = (x_t, y_t)^\top \in \mathbb{R}^2$ represents a 2D position. The state evolves linearly and observations are nonlinear range-bearing measurements from a sensor at position $(x_s, y_s)^\top$. The state transition is

$$\mathbf{x}_t = F \mathbf{x}_{t-1} + \mathbf{w}_t, \quad \mathbf{w}_t \sim \mathcal{N}(\mathbf{0}, Q), \quad (36)$$

where $F = I_2$ (constant-position model) and $Q = 0.01 I_2$. The observation model is

$$\mathbf{z}_t = h(\mathbf{x}_t) + \mathbf{v}_t, \quad \mathbf{v}_t \sim \mathcal{N}(\mathbf{0}, R), \quad (37)$$

with the nonlinear measurement function

$$h(\mathbf{x}) = \begin{pmatrix} \sqrt{(x - x_s)^2 + (y - y_s)^2} \\ \text{atan2}(y - y_s, x - x_s) \end{pmatrix}, \quad (38)$$

and diagonal observation noise covariance $R = \text{diag}(\sigma_r^2, \sigma_b^2)$, where $\sigma_r = 0.1$ (range noise std.) and $\sigma_b = 0.1$ rad (bearing noise std.). The sensor is located at the origin $(x_s, y_s) = (0, 0)$. The initial distribution is $\mathbf{x}_0 \sim \mathcal{N}(\boldsymbol{\mu}_0, \Sigma_0)$ with $\boldsymbol{\mu}_0 = (5, 5)^\top$ and $\Sigma_0 = I_2$.

The choice $\boldsymbol{\mu}_0 = (5, 5)^\top$ is not arbitrary: it is the direct application of the cross-cutting initial-condition principle stated in §2.2 to a model with a singular observation Jacobian. The range function $h_1(\mathbf{x}) = \sqrt{(x - x_s)^2 + (y - y_s)^2}$ has a gradient $\partial h_1 / \partial \mathbf{x} = (\mathbf{x} - \mathbf{x}_s) / \|\mathbf{x} - \mathbf{x}_s\|$ that is undefined at the

sensor position $\mathbf{x} = \mathbf{x}_s = (0, 0)$, and the bearing function $h_2(\mathbf{x}) = \text{atan2}(y - y_s, x - x_s)$ has a gradient $\partial h_2 / \partial \mathbf{x} = (-(y - y_s), x - x_s) / \|\mathbf{x} - \mathbf{x}_s\|^2$ that *diverges* at the sensor as $1 / \|\mathbf{x} - \mathbf{x}_s\|^2$. Any prior whose mean sits close to the origin — for example $\boldsymbol{\mu}_0 = (1, 1)^\top$, which places the mean at distance $\sqrt{2} \approx 1.41$ from the sensor — gives a bearing-Jacobian magnitude of order $1/2$ and a range-Jacobian direction that flips sign rapidly across the origin, so the EKF/UKF linearisation at the first time step is poorly conditioned and the particle filters sample many particles into the near-singular region. Moving the prior mean out to $(5, 5)^\top$ (distance $5\sqrt{2} \approx 7.07$ from the sensor) shrinks the bearing-Jacobian magnitude to order $1/50$, keeps $\Sigma_0 = I_2$ well inside the smoothly-observable region, and makes every downstream filter’s first-step linearisation stable.

Stochastic volatility model. The latent state $x_t \in \mathbb{R}$ represents the log-volatility. The state transition is

$$x_t = \alpha x_{t-1} + \sigma w_t, \quad w_t \sim \mathcal{N}(0, 1), \quad (39)$$

with persistence parameter $\alpha = 0.91$ and volatility-of-volatility $\sigma = 1.0$. Since $|\alpha| < 1$, the process is stationary, and we initialize $x_0 \sim \mathcal{N}(0, \sigma^2 / (1 - \alpha^2))$ from its stationary distribution, following the same principle discussed for the 1-D Kalman filter in Section 2.2. The observation model is

$$y_t = \beta \exp\left(\frac{x_t}{2}\right) v_t, \quad v_t \sim \mathcal{N}(0, 1), \quad (40)$$

with scale parameter $\beta = 0.5$. Note that $p(y_t | x_t) = \mathcal{N}(0, \beta^2 e^{x_t})$, so the observation noise is multiplicative and the likelihood is non-Gaussian in x_t . It is straightforward to run EKF and UKF on the range bearing model. Stochastic volatility is harder because the multiplicative noise means the observation is not a function of x_t alone in the additive sense EKF/UKF assume. To make EKF/UKF applicable, we apply a log-squared transform $z_t = \log y_t^2$ to the observation, which yields

$$z_t = \log \beta^2 + x_t + \varepsilon_t, \quad (41)$$

where $\varepsilon_t = \log v_t^2 \sim \log \chi_1^2$ is approximated as $\mathcal{N}(\mu_\varepsilon, \sigma_\varepsilon^2)$ with $\mu_\varepsilon = \psi(1/2) + \log 2 \approx -1.2704$ and $\sigma_\varepsilon^2 = \pi^2/2 \approx 4.935$, where ψ is the digamma function. This linearises the observation in x_t and puts the model into the additive-noise form EKF/UKF require.

Numerical results. All six experiments below share seed 42, $T = 100$ time steps, and 64-bit precision. UKF uses the defaults $\alpha = 10^{-3}$, $\beta = 2$, $\kappa = 0$.

- On the range-bearing model, both EKF (RMSE = 0.133, log-likelihood = +109.90) and UKF (RMSE = 0.133, log-likelihood = +109.89) track the state accurately (Figure 2). The nonlinearity here is mild — h is smooth away from the sensor and the posterior stays well inside a region where the linearisation is accurate — so EKF and UKF produce essentially identical estimates.
- On the raw stochastic volatility model, applying EKF/UKF directly with the nonlinear measurement $y_t = \beta e^{x_t/2} v_t$ gives RMSE = 2.04 for both filters (Figure 3). The Jacobian $\partial h / \partial x_t = (\beta/2) e^{x_t/2} v_t$ depends on the noise realisation v_t , so the EKF linearisation is not even well-defined in the usual additive-noise sense, and both filters mostly track the prior mean rather than responding to observations. Applying the log-squared transform (Figure 4) drops the RMSE to 1.14, nearly halving the latent-state error and recovering meaningful volatility tracking. Log-likelihoods for raw and log-transformed runs are not directly comparable because they are computed in different observation spaces (y_t versus $\log y_t^2$); only the RMSE in latent- x_t space is a fair comparison. EKF and UKF are numerically indistinguishable on the 1D SV model in both forms.

Computational footprint. All six runs execute on CPU; the EKF/UKF machinery has no meaningful GPU workload at $d_x \leq 2$ and $T = 100$. The numbers below are measured on the same host as the Kalman demo in §2.2 and should be read as a reference point for the particle-filter wall times in Tables 2–5, not as an optimisation target.

Table 1: EKF/UKF runtime and memory footprint. Peak memory is the incremental peak during the filter run (measured in Python’s resident set). d_x is the latent state dimension; d_z is the observation dimension. All six runs are CPU-only; no GPU memory consumed.

Experiment	d_x / d_z	Wall (s)	Time/step (ms)	Peak mem (MB)
Range-bearing EKF	2 / 2	1.75	17.5	2.4
Range-bearing UKF	2 / 2	1.88	18.8	2.8
SV raw EKF	1 / 1	1.71	17.1	2.5
SV raw UKF	1 / 1	1.90	18.9	3.0
SV log-transform EKF	1 / 1	1.73	17.3	2.5
SV log-transform UKF	1 / 1	1.95	19.5	3.0

UKF is consistently 5–15% slower than EKF per step. The gap is explained entirely by the sigma-point propagation cost: UKF passes $2d_x + 1 = 5$ sigma points (RB) or 3 sigma points (SV) through the observation function at every step, whereas EKF analytically evaluates a single Jacobian. On these small- d_x problems the absolute cost is trivial (~ 1 – 2 ms per step extra), but the ratio is consistent across models and foreshadows the sigma-point scaling $\mathcal{O}(d_x)$ that makes UKF less attractive as state dimension grows. Peak memory is essentially noise (~ 2 – 3 MB) — the filters hold only the current mean, covariance, and a handful of scratch matrices.

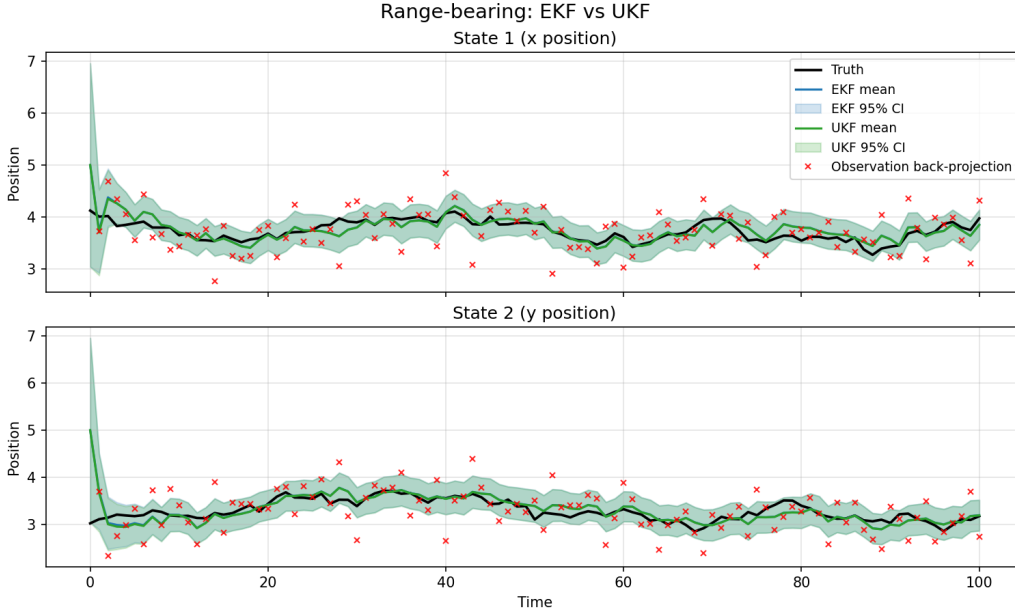


Figure 2: Range-bearing: EKF (blue) and UKF (green) overlaid on the truth trajectory (black), with back-projected observations shown as red crosses. Both filters reach $\text{RMSE} \approx 0.133$ and are numerically indistinguishable.

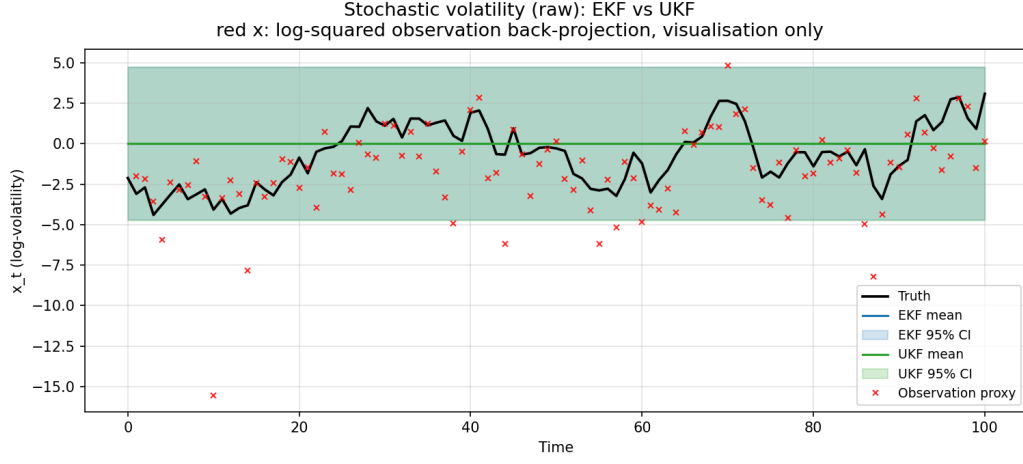


Figure 3: Raw stochastic volatility: EKF (blue) and UKF (green) both mostly track the prior mean rather than the latent volatility. Both filters have $\text{RMSE} = 2.04$ and $\text{log-likelihood} = -151.20$ in the raw observation space.

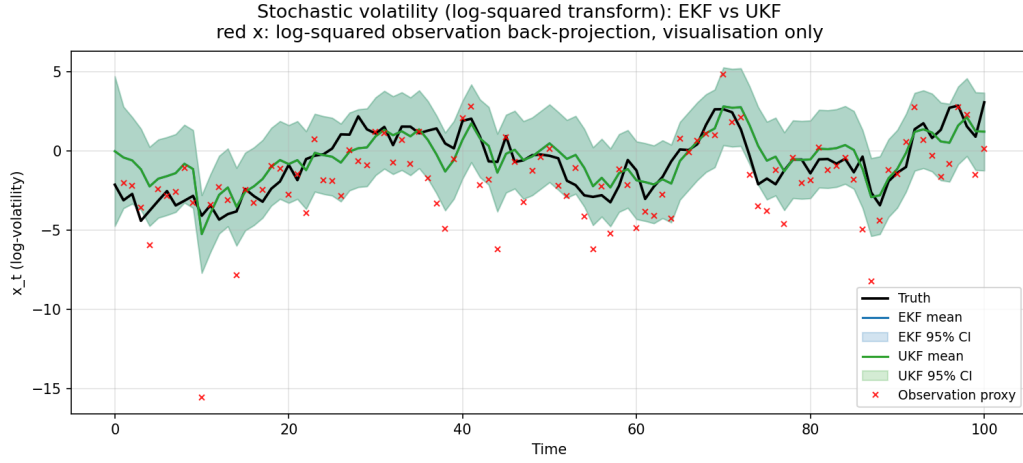


Figure 4: Log-squared stochastic volatility: EKF (blue) and UKF (green) recover meaningful latent-volatility tracking after the transform. Both filters have $\text{RMSE} = 1.14$ and $\text{log-likelihood} = -235.27$ in the transformed observation space.

4 Particle-Based Methods

4.1 Importance Sampling and Sequential Monte Carlo

Start from the filtering recursion.

$$\begin{aligned} \textbf{Prediction:} \quad p(x_k \mid z_{1:k-1}) &= \int p(x_k \mid x_{k-1}) p(x_{k-1} \mid z_{1:k-1}) dx_{k-1} \\ \textbf{Update:} \quad p(x_k \mid z_{1:k}) &= \frac{p(z_k \mid x_k) p(x_k \mid z_{1:k-1})}{p(z_k \mid z_{1:k-1})}, \end{aligned} \quad (42)$$

are intractable in general because the prediction integral has no closed form when f or h are nonlinear or the noise is non-Gaussian, and the normalization constant $p(z_k \mid z_{1:k-1})$ in the update is itself an integral over all of state space. The key idea of particle-based methods is to *represent the filtering distribution by a set of weighted samples* (particles). Given N particles $\{x_k^{(i)}\}_{i=1}^N$ with associated weights $\{w_k^{(i)}\}_{i=1}^N$ satisfying $\sum_i w_k^{(i)} = 1$, we approximate

$$p(x_k \mid z_{1:k}) \approx \sum_{i=1}^N w_k^{(i)} \delta(x_k - x_k^{(i)}). \quad (43)$$

With this empirical representation, any posterior expectation reduces to a weighted sum,

$$\mathbb{E}[g(x_k) \mid z_{1:k}] = \int g(x_k) p(x_k \mid z_{1:k}) dx_k \approx \sum_{i=1}^N w_k^{(i)} g(x_k^{(i)}), \quad (44)$$

so recovering the posterior mean and covariance requires no further integration.

Importance Sampling

The challenge is that we generally cannot sample directly from the target $p(x_k \mid z_{1:k})$. *Importance sampling* (IS) resolves this by drawing from a tractable *proposal distribution* $q(x_k)$ and correcting for the mismatch with weights. For any function g ,

$$\mathbb{E}_p[g(x_k)] = \int g(x_k) \frac{p(x_k \mid z_{1:k})}{q(x_k)} q(x_k) dx_k \approx \sum_{i=1}^N w^{(i)} g(x_k^{(i)}), \quad x_k^{(i)} \sim q(x_k), \quad (45)$$

where the *importance weight* is proportional to the ratio $w^{(i)} \propto p(x_k^{(i)} \mid z_{1:k})/q(x_k^{(i)})$. Since $p(x_k \mid z_{1:k})$ is known only up to a normalizing constant, we use the *self-normalized* estimator with unnormalized weights $\tilde{w}^{(i)} \propto p(x_k^{(i)} \mid z_{1:k})/q(x_k^{(i)})$,

$$w_k^{(i)} = \frac{\tilde{w}_k^{(i)}}{\sum_{j=1}^N \tilde{w}_k^{(j)}}. \quad (46)$$

This is consistent as $N \rightarrow \infty$ for any proposal q with support containing that of p .

Sequential Monte Carlo

Sequential use is where importance sampling becomes a filter. Suppose at time $k-1$ we hold a particle approximation $\{x_{k-1}^{(i)}, w_{k-1}^{(i)}\}$ of $p(x_{k-1} \mid z_{1:k-1})$. Our *only* goal at time k is the posterior $p(x_k \mid z_{1:k})$, whose mean m_k and covariance P_k are the quantities of interest. The intermediate predicted distribution $p(x_k \mid z_{1:k-1})$ and its moments m_k^-, P_k^- are not ends in themselves—they are merely one route to the posterior. This gives us great freedom: we may draw new particles from *any* proposal

$$x_k^{(i)} \sim q(x_k \mid x_{k-1}^{(i)}, z_k), \quad i = 1, \dots, N, \quad (47)$$

provided it has sufficient support, and correct for the mismatch via IS weights. Writing the target as $p(x_k \mid z_{1:k}) \propto p(z_k \mid x_k) p(x_k \mid x_{k-1}^{(i)})$, the unnormalized weight for each particle is

$$\tilde{w}_k^{(i)} = w_{k-1}^{(i)} \cdot \frac{p(z_k \mid x_k^{(i)}) p(x_k^{(i)} \mid x_{k-1}^{(i)})}{q(x_k^{(i)} \mid x_{k-1}^{(i)}, z_k)}, \quad (48)$$

which after normalization gives $p(x_k | z_{1:k}) \approx \sum_i w_k^{(i)} \delta(x_k - x_k^{(i)})$. This is the *Sequential Monte Carlo* (SMC) framework: the prediction and update integrals are handled together in a single IS step, and the two-step predict-then-update decomposition of the Kalman family is not fundamental here—it is just one way to factor the joint density.

The proposal is the design choice. The simplest option $q(x_k | x_{k-1}^{(i)}, z_k) = p(x_k | x_{k-1}^{(i)})$ ignores the current observation entirely, simplifying (48) to $\tilde{w}_k^{(i)} \propto w_{k-1}^{(i)} \cdot p(z_k | x_k^{(i)})$ —this is the Bootstrap Particle Filter. More sophisticated methods either use a smarter proposal that already incorporates z_k (while still relying on weights to correct for the approximation), or abandon importance weights altogether and instead move particles deterministically toward the posterior using a flow constructed from a Gaussian approximation to $p(x_k | z_{1:k})$ —as in the EDH and LEDH flow filters described later.

A well-known practical issue with weighted SMC is *weight degeneracy*: after many steps, the multiplicative weight update concentrates nearly all weight on very few particles. The standard remedy is *resampling*, and we will study its effect by applying Bootstrap Particle Filter on non-linear models.

4.2 Bootstrap Particle Filter

4.2.1 Algorithm

The BPF is the simplest instantiation of SMC: the proposal is the transition prior $q = p(x_k | x_{k-1}^{(i)})$, so each particle is propagated by forward-simulating the dynamics and scored against the new observation. From (48) the weight reduces to $\tilde{w}_k^{(i)} \propto w_{k-1}^{(i)} \cdot p(z_k | x_k^{(i)})$.

Algorithm 5 Bootstrap Particle Filter (BPF)

Require: particles $\{x_{k-1}^{(i)}, w_{k-1}^{(i)}\}_{i=1}^N$, observation z_k

1: **for** $i = 1$ **to** N **do**

2: **Predict:** sample $x_k^{(i)} \sim p(x_k | x_{k-1}^{(i)})$

3: **Weight:** $\tilde{w}_k^{(i)} \leftarrow w_{k-1}^{(i)} \cdot p(z_k | x_k^{(i)})$

4: **end for**

5: **Normalize:** $w_k^{(i)} \leftarrow \tilde{w}_k^{(i)} / \sum_{j=1}^N \tilde{w}_k^{(j)}$ for all i

6: **Estimate:** $m_k = \sum_i w_k^{(i)} x_k^{(i)}$, $P_k = \sum_i w_k^{(i)} (x_k^{(i)} - m_k)(x_k^{(i)} - m_k)^\top$

7: **if** $N_{\text{eff}} = (\sum_i (w_k^{(i)})^2)^{-1} < N/2$ **then**

8: **Resample** (Algorithm 6); set $w_k^{(i)} \leftarrow 1/N$ for all i

9: **end if**

Ensure: $\{x_k^{(i)}, w_k^{(i)}\}_{i=1}^N \approx p(x_k | z_{1:k})$, moments m_k, P_k

4.2.2 Particle Degeneracy and Resampling

The weight update multiplies by a per-particle likelihood, and repeated multiplication concentrates the relative mass on a shrinking number of particles. This is the degeneracy mechanism: not that likelihood factors are below one (they are not bounded for density functions), but that the distribution of relative weights collapses exponentially fast with time. The *effective sample size*

$$N_{\text{eff}} = \frac{1}{\sum_{i=1}^N (w_k^{(i)})^2} \quad (49)$$

quantifies this: it equals N when all weights are uniform and drops toward 1 at full collapse. We trigger resampling when $N_{\text{eff}} < N/2$.

Multinomial resampling. The conceptually simplest fix draws N independent samples from the categorical distribution defined by the current weights,

$$x_k^{(i)} \sim \text{Categorical}(\{w_k^{(j)}\}_{j=1}^N), \quad i = 1, \dots, N, \quad (50)$$

then resets all weights to $1/N$. High-weight particles are duplicated; low-weight ones are discarded. While unbiased, this introduces additional Monte Carlo variance: because each draw is independent, particles with moderate weight may be selected zero times by chance. The cost is $O(N \log N)$ via the inverse-CDF method.

Systematic resampling (the actual implementation). We use systematic resampling, which addresses both drawbacks. A single draw $u \sim \mathcal{U}(0, 1/N)$ places N evenly-spaced points across $[0, 1]$:

$$u^{(i)} = u + \frac{i-1}{N}, \quad i = 1, \dots, N. \quad (51)$$

Particle j is selected for slot i if $u^{(i)}$ falls in $(\sum_{l < j} w_k^{(l)}, \sum_{l \leq j} w_k^{(l)}]$. Because the grid is deterministically spread over $[0, 1]$, every particle with weight $w^{(j)} > 1/N$ is guaranteed to appear at least $\lfloor Nw^{(j)} \rfloor$ times, and the copy count deviates from the ideal $Nw^{(j)}$ by at most 1. This eliminates the random clustering of multinomial resampling, minimizes the variance of the resampling step, and reduces the cost to $O(N)$ via a single linear pass through the cumulative weight sum.

Algorithm 6 Systematic Resampling

Require: weights $\{w^{(i)}\}_{i=1}^N$, positions $\{x^{(i)}\}_{i=1}^N$

- 1: Draw $u \sim \mathcal{U}(0, 1/N)$; set $u^{(i)} = u + (i-1)/N$ for $i = 1, \dots, N$
- 2: Compute cumulative sums $C_j = \sum_{l=1}^j w^{(l)}$
- 3: **for** $i = 1$ **to** N **do**
- 4: Find j_i such that $C_{j_i-1} < u^{(i)} \leq C_{j_i}$; set $\bar{x}^{(i)} \leftarrow x^{(j_i)}$
- 5: **end for**

Ensure: $\{\bar{x}^{(i)}, 1/N\}_{i=1}^N$

After resampling the particle set has uniform weights and represents the same distribution as before, but with diversity restored. The cost is *sample impoverishment*: multiplicity alone now encodes the distribution, with no weight diversity. This motivates the flow-based methods described next, which avoid resampling altogether by moving particles directly toward the posterior. I applied the bootstrap particle filter to the same range-bearing and stochastic-volatility models. Figure 5 shows the posterior mean tracking. Figure 6 shows that the range-bearing ESS collapses within the first few steps, while stochastic volatility stays above threshold longer. Figure 7 shows the resulting weight concentration directly, and Figure 8 shows why the stochastic-volatility run keeps more particle diversity.

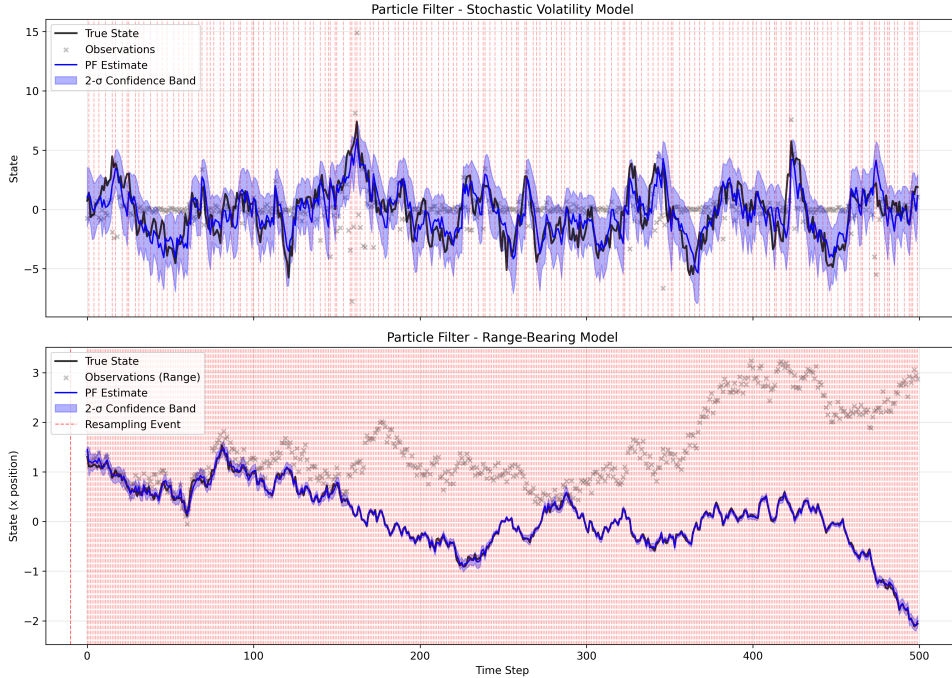


Figure 5: Bootstrap PF tracking on range-bearing and 1D stochastic volatility. The filter mean tracks the true state on both models at $N = 200$.

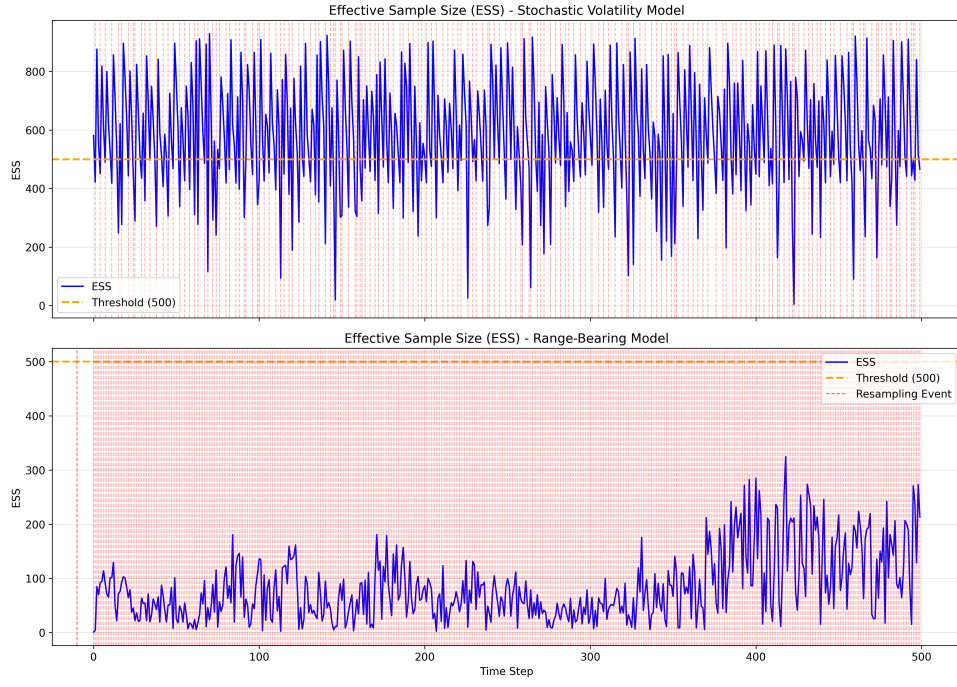


Figure 6: Effective sample size over time for the same two runs. Range-bearing collapses within the first few steps; stochastic volatility stays above threshold longer.

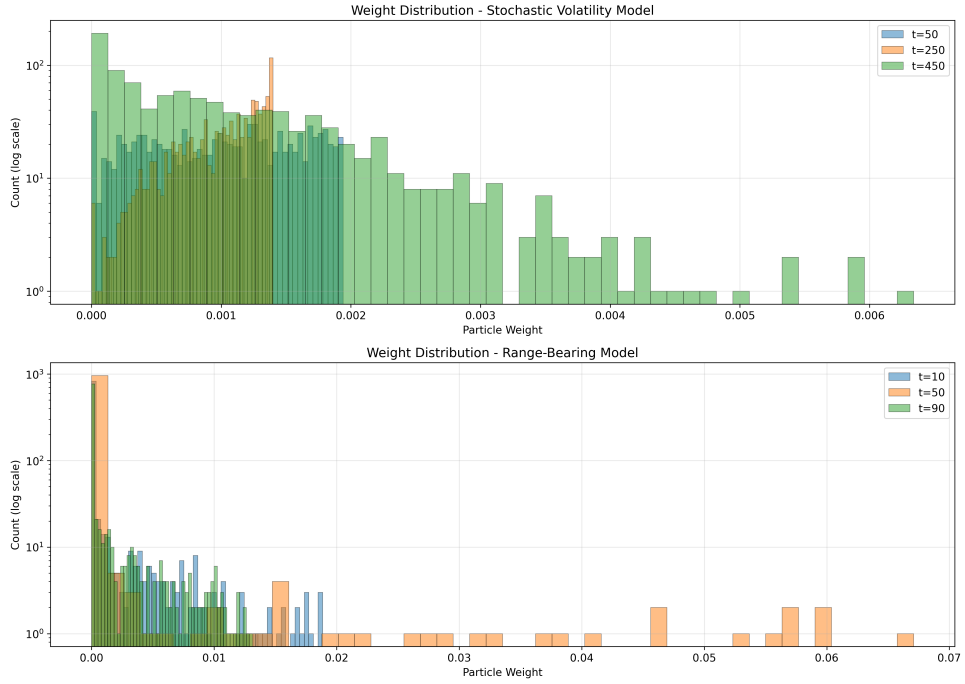


Figure 7: Weight distribution histograms at $k = 1, 25, 100$. The range-bearing cloud is dominated by a single particle within one step.

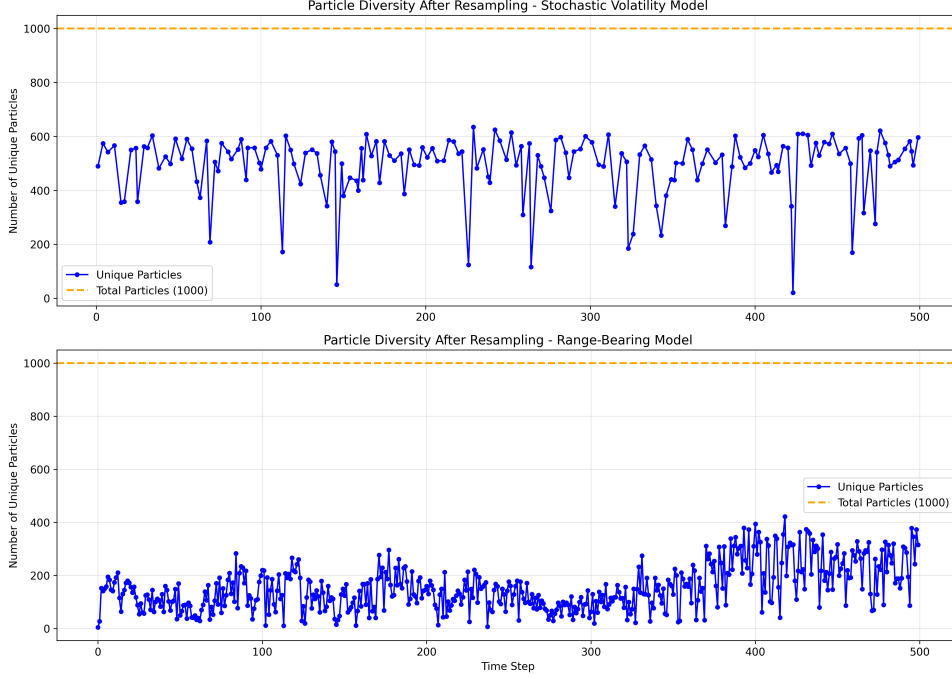


Figure 8: Particle diversity on stochastic volatility. Diversity stays reasonable because the observation is diffuse enough to spread the weight mass.

Empirical takeaway (§4.2). The bootstrap PF does two useful things that EKF/UKF cannot. (i) On the raw stochastic volatility model, where multiplicative-noise observations break every linearisation-based filter at $\text{RMSE} \approx 2.04$, the bootstrap PF evaluates the likelihood directly and reaches $\text{RMSE} = 1.117$ (Table 4), fixing what the Kalman family could not. (ii) On the range-bearing model, the bootstrap PF matches EKF/UKF on RMSE (≈ 0.14) without needing the Jacobian of h at all — useful when the measurement model is a black box.

4.3 EDH/LEDH Flow Filter

I next use Exact Daum–Huang flow and its local variants [2, 3, 4].

4.3.1 Use Fokker-Planck to reach the target distribution

Start from the filtering update.

$$p(x_k \mid z_{1:k}) \propto \underbrace{p(x_k \mid z_{1:k-1})}_{g(x)} \cdot \underbrace{p(z_k \mid x_k)}_{h(x)}. \quad (52)$$

The Bootstrap Particle Filter handles this by reweighting: sample from $g(x)$, then multiply each particle’s weight by $h(x^{(i)})$. That multiplication is precisely what causes weight degeneracy. Flow filters [2, 3] take a different route: instead of adjusting weights, we move the particles. If we can find a deterministic map that transports samples from $g(x)$ into samples from $g(x)h(x)$, we have solved the update step exactly—with no weights and therefore no weight degeneracy.

To do this continuously, introduce a pseudo-time $\lambda \in [0, 1]$ and the *log-homotopy*:

$$\log p(x, \lambda) = \log g(x) + \lambda \log h(x). \quad (53)$$

At $\lambda = 0$, $p(x, 0) = g(x)$ is the predicted distribution; at $\lambda = 1$, $p(x, 1) \propto g(x)h(x)$ is the posterior. We want a flow $f(x, \lambda)$ such that particles obeying

$$\frac{d\mathbf{x}}{d\lambda} = f(\mathbf{x}, \lambda) \quad (54)$$

follow this homotopy path from prior to posterior.

A deterministic flow induces a density evolution governed by the zero-diffusion *Fokker-Planck equation* (also called the continuity equation for probability conservation),

$$\frac{\partial p}{\partial \lambda} = -\nabla \cdot (pf). \quad (55)$$

The homotopy requires $\frac{\partial \log p}{\partial \lambda} = \log h(x)$, which together with the Fokker-Planck equation (dividing by p and using $\nabla \cdot (pf) = p\nabla \cdot f + (\nabla p) \cdot f$) yields the fundamental constraint:

$$\boxed{\log h(x) = -\nabla \cdot f - (\nabla \log p) \cdot f} \quad (56)$$

This is a PDE in the unknown vector field f , and it has infinitely many solutions. Any flow satisfying this equation will correctly transport particles from $g(x)$ to $g(x)h(x)$. The choice of f is a design decision, completely analogous to the choice of proposal in weighted SMC. We just need to pick one that is tractable to compute.

4.3.2 Determine the evolution kernel

The simplest structured choice is an *affine flow*:

$$f(x, \lambda) = A(\lambda)x + b(\lambda), \quad (57)$$

where $A(\lambda)$ and $b(\lambda)$ are a matrix and vector to be determined. A stretches and rotates particles while b translates them. Crucially, they are not free parameters — they must be chosen so that the flow satisfies the Fokker-Planck constraint. For an affine flow $\nabla \cdot f = \text{Tr}(A)$, and the constraint becomes algebraic once we know $\nabla \log p(x, \lambda)$.

This is where we need an assumption. To evaluate $\nabla \log p(x, \lambda)$ in the constraint, we need to know what $p(x, \lambda)$ looks like for intermediate λ . Different assumptions lead to different flows — different schedules for $A(\lambda)$ and $b(\lambda)$ — but they all share the same endpoints: $\lambda = 0$ is the prior $g(x)$ and $\lambda = 1$ is the posterior $g(x)h(x)$, fixed by the homotopy construction. The assumption only determines the path taken between those two endpoints; it does not change the destination. We assume $p(x, \lambda)$ is Gaussian:

$$p(x, \lambda) \sim \mathcal{N}(\bar{\eta}_\lambda, P_\lambda), \quad (58)$$

with precision and mean evolving as

$$P_\lambda^{-1} = P^{-1} + \lambda H^T R^{-1} H, \quad (59)$$

$$\bar{\eta}_\lambda = P_\lambda [P^{-1} \bar{\eta}_0 + \lambda H^T R^{-1} z]. \quad (60)$$

This is the only role of the Gaussian assumption: it gives a tractable expression for $\nabla \log p(x, \lambda) = -P_\lambda^{-1}(x - \bar{\eta}_\lambda)$, which we substitute into the constraint to solve for A and b . A different assumption about the intermediate distribution would yield a different flow schedule, but the particles would still travel between the same prior and posterior.

With the Gaussian form, $\nabla \log p(x, \lambda) = -P_\lambda^{-1}(x - \bar{\eta}_\lambda)$. Differentiating the precision with respect to λ and using the matrix identity $dP_\lambda/d\lambda = -P_\lambda(dP_\lambda^{-1}/d\lambda)P_\lambda$ gives

$$\frac{dP_\lambda}{d\lambda} = -P_\lambda H^T R^{-1} H P_\lambda. \quad (61)$$

Matching this with the covariance evolution under the affine flow, $AP_\lambda + P_\lambda A^T = dP_\lambda/d\lambda$, and solving the resulting Riccati equation yields:

$$\boxed{A(\lambda) = -\frac{1}{2}P_\lambda H^T R^{-1} H = -\frac{1}{2}P H^T (\lambda H P H^T + R)^{-1} H} \quad (62)$$

For $b(\lambda)$, requiring that the mean evolves correctly under the flow gives:

$$\boxed{b(\lambda) = (I + 2\lambda A)[(I + \lambda A)P H^T R^{-1} z + A \bar{\eta}_0]} \quad (63)$$

These two equations define the EDH flow completely. The log-homotopy definition forces λ to run from 0 to 1, so the particle integration over λ is a finite integral solved numerically with small steps ε_j .

4.3.3 EDH vs. LEDH: incremental improvements in linearization quality.

All variants use exactly the same framework: the same log-homotopy, the same affine ansatz, the same Gaussian assumption for $\nabla \log p$. They differ only in *where and when* the Jacobian $H = \partial h / \partial x$ is evaluated. There are three levels of refinement:

- **EDH (fixed global linearization):** H is computed once at the global predicted mean $\bar{\eta}_0$ at the start of the flow ($\lambda = 0$) and held fixed for the entire λ integration. All particles share the same $A(\lambda)$ and $b(\lambda)$ at every step. This is the most efficient variant but the least accurate: as particles move during the flow, the mean $\bar{\eta}(\lambda)$ drifts away from $\bar{\eta}_0$, yet the linearization never corrects for this.
- **EDH (intermediate re-linearization):** At each λ step, H is re-evaluated at the current global mean $\bar{\eta}^{(\lambda)}$, and the nonlinear correction $e = h(\bar{\eta}^{(\lambda)}) - H\bar{\eta}^{(\lambda)}$ is computed. The drift vector b is updated to use $(z - e)$ instead of z , capturing the residual between the true observation function and its local linearization. The wind field is still uniform across particles — all particles share the same A and b at each step — but both are recomputed at each step to track where the cloud has moved. This is strictly richer than the fixed variant: not only does H improve, but b also corrects for the nonlinearity of h at the current operating point.
- **LEDH (local, per-particle linearization) [4]:** $H^{(i)} = \partial h / \partial x|_{x^{(i)}(\lambda)}$ is computed at each particle's own current position at each λ step, giving every particle its own $A^{(i)}(\lambda)$ and $b^{(i)}(\lambda)$. Each particle gets its own local affine drift. The linearization error is controlled by the distance from $x^{(i)}$ to the linearization point—which is now zero. This is the most accurate variant, at the cost of N Jacobian evaluations per λ step.

Each step in this hierarchy adds one more correction to where the linearization is taken—across particles, across pseudo-time, or both. None of them is a different algorithm. They are the same affine flow with progressively better approximations to $\nabla \log p$ plugged in.

4.3.4 Algorithm

All three variants share the same structural role for the global EKF/UKF: it runs in parallel with the particle cloud *solely* to supply the covariance $\mathbf{P}$ and the linearization point needed to compute the flow kernel $A(\lambda), b(\lambda)$. It is never used to produce filter estimates. The reported output—the posterior mean m_k and covariance P_k —is always taken from the empirical particle ensemble:

$$m_k = \frac{1}{N} \sum_i \boldsymbol{\eta}^{(i)}, \quad P_k = \frac{1}{N} \sum_i (\boldsymbol{\eta}^{(i)} - m_k)(\boldsymbol{\eta}^{(i)} - m_k)^\top. \quad (64)$$

To keep the global filter from drifting away from the particle cloud, a *feedback step* resets the EKF/UKF mean to the current empirical mean at the start of each time step before prediction. The EKF/UKF covariance $\mathbf{P}$ is then used as the kernel covariance, but the covariance estimate reported to the user is the empirical one above. The algorithm below is the version I use.

Algorithm 7 EDH Flow Filter (Fixed Global Linearization)

Require: Particles $\{\boldsymbol{\eta}^{(i)}\}_{i=1}^N$, global EKF/UKF state $(\bar{\boldsymbol{\eta}}, \mathbf{P})$ **Require:** Lambda schedule $\{\varepsilon_j\}_{j=1}^{n_\lambda}$

```
1: for  $k = 1, 2, \dots, T$  do
2:   Feedback:  $\bar{\boldsymbol{\eta}} \leftarrow \frac{1}{N} \sum_i \boldsymbol{\eta}^{(i)}$  (reset global mean to empirical mean)
3:   Predict (EKF/UKF):  $(\bar{\boldsymbol{\eta}}_0, \mathbf{P}) \leftarrow \text{EKF/UKF predict}(\bar{\boldsymbol{\eta}}, \mathbf{P})$ 
4:   Predict (particles):  $\boldsymbol{\eta}^{(i)} \sim p(\mathbf{x}_k \mid \boldsymbol{\eta}^{(i)})$  for  $i = 1, \dots, N$ 
5:    $\mathbf{H} \leftarrow \frac{\partial \mathbf{h}}{\partial \mathbf{x}} \Big|_{\bar{\boldsymbol{\eta}}_0}$  (fixed: linearize once at  $\bar{\boldsymbol{\eta}}_0$ , held for all  $\lambda$  steps)
6:    $\lambda \leftarrow 0$ 
7:   for  $j = 1, 2, \dots, n_\lambda$  do
8:      $\lambda \leftarrow \lambda + \varepsilon_j$ 
9:      $\mathbf{A} \leftarrow -\frac{1}{2} \mathbf{P} \mathbf{H}^\top (\lambda \mathbf{H} \mathbf{P} \mathbf{H}^\top + \mathbf{R})^{-1} \mathbf{H}$ 
10:     $\mathbf{b} \leftarrow (\mathbf{I} + 2\lambda \mathbf{A}) [(\mathbf{I} + \lambda \mathbf{A}) \mathbf{P} \mathbf{H}^\top \mathbf{R}^{-1} \mathbf{z}_k + \mathbf{A} \bar{\boldsymbol{\eta}}_0]$ 
11:     $\boldsymbol{\eta}^{(i)} \leftarrow \boldsymbol{\eta}^{(i)} + \varepsilon_j (\mathbf{A} \boldsymbol{\eta}^{(i)} + \mathbf{b})$  for all  $i$ 
12:  end for
13:  Update (EKF/UKF):  $(\bar{\boldsymbol{\eta}}, \mathbf{P}) \leftarrow \text{EKF/UKF update}(\bar{\boldsymbol{\eta}}_0, \mathbf{P}, \mathbf{z}_k)$  (kernel only; not reported)
14:  Output:  $m_k = \frac{1}{N} \sum_i \boldsymbol{\eta}^{(i)}$ ,  $P_k = \frac{1}{N} \sum_i (\boldsymbol{\eta}^{(i)} - m_k)(\boldsymbol{\eta}^{(i)} - m_k)^\top$ 
15: end for
```

Algorithm 8 EDH Flow Filter (Intermediate Re-linearization)

Require: Same setup as Algorithm 7

```
1: for  $k = 1, 2, \dots, T$  do
2:   Feedback, Predict (EKF/UKF), Predict (particles): same as Algorithm 7
3:    $\bar{\boldsymbol{\eta}}^{(\lambda)} \leftarrow \bar{\boldsymbol{\eta}}_0$  (track current global mean through flow)
4:    $\lambda \leftarrow 0$ 
5:   for  $j = 1, 2, \dots, n_\lambda$  do
6:      $\lambda \leftarrow \lambda + \varepsilon_j$ 
7:      $\mathbf{H} \leftarrow \frac{\partial \mathbf{h}}{\partial \mathbf{x}} \Big|_{\bar{\boldsymbol{\eta}}^{(\lambda)}}$  (re-linearize at current global mean  $\bar{\boldsymbol{\eta}}^{(\lambda)}$ )
8:      $\mathbf{e} \leftarrow \mathbf{h}(\bar{\boldsymbol{\eta}}^{(\lambda)}) - \mathbf{H} \bar{\boldsymbol{\eta}}^{(\lambda)}$  (nonlinear correction at current mean)
9:      $\mathbf{A} \leftarrow -\frac{1}{2} \mathbf{P} \mathbf{H}^\top (\lambda \mathbf{H} \mathbf{P} \mathbf{H}^\top + \mathbf{R})^{-1} \mathbf{H}$ 
10:     $\mathbf{b} \leftarrow (\mathbf{I} + 2\lambda \mathbf{A}) [(\mathbf{I} + \lambda \mathbf{A}) \mathbf{P} \mathbf{H}^\top \mathbf{R}^{-1} (\mathbf{z}_k - \mathbf{e}) + \mathbf{A} \bar{\boldsymbol{\eta}}_0]$ 
11:     $\bar{\boldsymbol{\eta}}^{(\lambda)} \leftarrow \bar{\boldsymbol{\eta}}^{(\lambda)} + \varepsilon_j (\mathbf{A} \bar{\boldsymbol{\eta}}^{(\lambda)} + \mathbf{b})$  (advance mean for next re-linearization)
12:     $\boldsymbol{\eta}^{(i)} \leftarrow \boldsymbol{\eta}^{(i)} + \varepsilon_j (\mathbf{A} \boldsymbol{\eta}^{(i)} + \mathbf{b})$  for all  $i$ 
13:  end for
14:  Update (EKF/UKF): same as Algorithm 7 (kernel only; not reported)
15:  Output:  $m_k = \frac{1}{N} \sum_i \boldsymbol{\eta}^{(i)}$ ,  $P_k = \frac{1}{N} \sum_i (\boldsymbol{\eta}^{(i)} - m_k)(\boldsymbol{\eta}^{(i)} - m_k)^\top$ 
16: end for
```

Algorithm 9 LEDH Flow Filter (Local Per-Particle Linearization)

Require: Same setup as Algorithm 7

```
1: for  $k = 1, 2, \dots, T$  do
2:   Feedback, Predict (EKF/UKF), Predict (particles): same as Algorithm 7
3:    $\lambda \leftarrow 0$ 
4:   for  $j = 1, 2, \dots, n_\lambda$  do
5:      $\lambda \leftarrow \lambda + \varepsilon_j$ 
6:     for each particle  $i$  (batched) do
7:        $\mathbf{H}^{(i)} \leftarrow \left. \frac{\partial \mathbf{h}}{\partial \mathbf{x}} \right|_{\boldsymbol{\eta}^{(i)}}$  (re-linearize at each particle's own position)
8:        $\mathbf{e}^{(i)} \leftarrow \mathbf{h}(\boldsymbol{\eta}^{(i)}) - \mathbf{H}^{(i)} \boldsymbol{\eta}^{(i)}$  (nonlinear offset correction)
9:        $\mathbf{A}^{(i)} \leftarrow -\frac{1}{2} \mathbf{P} \mathbf{H}^{(i)\top} (\lambda \mathbf{H}^{(i)} \mathbf{P} \mathbf{H}^{(i)\top} + \mathbf{R})^{-1} \mathbf{H}^{(i)}$ 
10:       $\mathbf{b}^{(i)} \leftarrow (\mathbf{I} + 2\lambda \mathbf{A}^{(i)})[(\mathbf{I} + \lambda \mathbf{A}^{(i)}) \mathbf{P} \mathbf{H}^{(i)\top} \mathbf{R}^{-1} (\mathbf{z}_k - \mathbf{e}^{(i)}) + \mathbf{A}^{(i)} \bar{\boldsymbol{\eta}}_0]$ 
11:       $\boldsymbol{\eta}^{(i)} \leftarrow \boldsymbol{\eta}^{(i)} + \varepsilon_j (\mathbf{A}^{(i)} \boldsymbol{\eta}^{(i)} + \mathbf{b}^{(i)})$ 
12:    end for
13:  end for
14:  Update (EKF/UKF): same as Algorithm 7 (kernel only; not reported)
15:  Output:  $m_k = \frac{1}{N} \sum_i \boldsymbol{\eta}^{(i)}$ ,  $P_k = \frac{1}{N} \sum_i (\boldsymbol{\eta}^{(i)} - m_k)(\boldsymbol{\eta}^{(i)} - m_k)^\top$ 
16: end for
```

4.3.5 Experiments

We run every variant on all three headline models at $N = 200$ particles, $T = 100$ time steps, and $n_\lambda = 29$ flow steps with the geometric ratio $q = 1.2$ unless noted.

Table 2: Range-bearing filter comparison ($N = 200$, $T = 100$, $n_\lambda = 29$). All RMSE values are in position units. The ‘EDH global’ row uses a single global linearisation at $\lambda = 0$ and is the essential control — it diverges, which is what motivates intermediate re-linearisation for every other variant.

Filter	RMSE	Log-lik	Wall (s)	ms/step
Bootstrap PF	0.135	+109.07	30.52	305.24
EDH flow (intermediate relin.)	0.132	+110.13	130.41	1304.1
EDH flow (global, 1-step)	5.276	−17401.29	130.20	1302.0
EDH invertible	0.133	+135.38	138.61	1386.1
LEDH flow	0.132	+110.07	138.65	1386.5
LEDH invertible	0.127	88.78	142.78	1427.8
<i>Reference: EKF</i>	0.133	+109.90	1.75	17.5
<i>Reference: UKF</i>	0.133	+109.89	1.88	18.8

The range-bearing take: once intermediate re-linearisation is turned on, every flow variant sits within $\sim 5\%$ of EKF/UKF on RMSE, at roughly 70–80 $\times$ the wall-time. Global-EDH (one linearisation at $\lambda = 0$, no refresh as the cloud moves) blows up entirely — this is the control that justifies the per-step re-linearisation used by every other variant. LEDH-invertible is best on RMSE but its log-likelihood is noticeably lower than LEDH-flow’s, suggesting the Jacobian correction is trading calibration for mean accuracy on this well-conditioned problem.

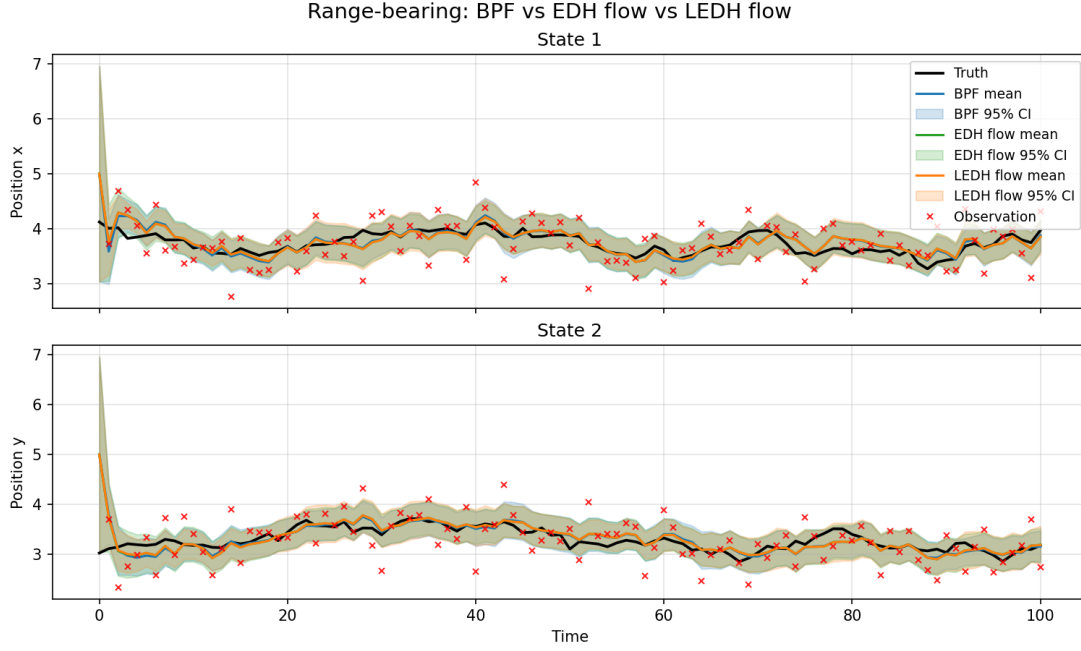


Figure 9: Range-bearing: bootstrap PF (blue), EDH flow (green), and LEDH flow (orange) overlaid on the true trajectory (black), with back-projected observations as red crosses. The three filters are visually indistinguishable on this well-conditioned model — the per-model ranking in Table 2 is driven primarily by wall-time; log-likelihood differs by a few nats but RMSE is tied to three decimal places.

Table 3: Acoustic-tracking filter comparison ($T = 100$ except where noted). The high-dimensional state and weak observation geometry make this the hardest of the three headline models; none of the filters reach the same relative accuracy they do on range-bearing, and the ranking is driven as much by particle count and hyperparameter choice as by algorithm.

Filter	RMSE	Log-lik	Wall (s)	Notes
EKF	340.78	-31166.52	9.70	linearisation fails
UKF	49.54	-5421.08	23.80	
Bootstrap PF ($N = 200$)	55.69	+103.34	8.15	
Bootstrap PF ($N = 10^6, T = 50$)	22.13	+610.82	158.72	heavy particle compute
EDH flow	297.46	-24537.26	36.25	diverges
EDH invertible	253.67	-25126.02	47.94	diverges
LEDH flow	41.37	-101415.40	48.59	best wall-time-to-RMSE
LEDH invertible	106.92	-63810.64	51.80	
Stochastic EDH	45.11	-65192.23	24.79	

The acoustic result is not the clean story I expected. LEDH-flow, not LEDH-invertible, is the best deterministic-flow variant on this model: 41.4 RMSE against 106.9 for the invertible sibling and ~ 255 –297 for the globally-linearised EDH variants. Stochastic EDH at 45.1 is effectively tied with LEDH-flow. Bootstrap PF at a manageable $N = 200$ is cheaper but less accurate (55.7 RMSE in 8.15 s); pushing to $N = 10^6$ (and shortening the trajectory to $T = 50$) reaches 22.1, which I treat as a brute-force reference rather than an operational point. The flow filters do not win wall-time here. They win RMSE at moderate compute.

Table 4: Stochastic volatility, *raw* model ($h(x) = \beta e^{x/2} v$, multiplicative noise; $T = 100$, $N = 200$). Every linearisation-based method fails at ~ 2.0 RMSE; only the bootstrap PF handles the raw multiplicative likelihood.

Filter	RMSE	Log-lik	Wall (s)
Bootstrap PF	1.117	-68.12	3.41
EDH flow	2.059	-156.81	13.17
Kernel (matrix)	2.091	—	101.02
<i>Reference: EKF</i>	2.039	-151.20	1.71
<i>Reference: UKF</i>	2.039	-151.20	1.90

Table 5: Stochastic volatility, *log-squared-transformed* model ($z = \log y^2$; $T = 100$, $N = 200$). The linear transform restores competitiveness to EKF/UKF and the flow family; kernel is slower but still tracks. LEDH-invertible narrowly beats the bootstrap PF on RMSE here, at $6\times$ the wall-time.

Filter	RMSE	Log-lik	Wall (s)
Bootstrap PF	1.153	-235.63	3.75
EDH flow	1.147	-235.47	14.38
LEDH invertible	1.162	-236.25	23.72
Kernel (matrix, $T = 500$)	1.372	—	287.39
<i>Reference: EKF</i>	1.144	-235.27	1.73
<i>Reference: UKF</i>	1.144	-235.27	1.95

The SV takeaway: on the raw model, every filter that relies on a local linearisation of h (EKF, UKF, EDH, LEDH, kernel) fails at the same ~ 2.0 RMSE, and only the bootstrap PF — which evaluates the likelihood directly without touching h ’s Jacobian — tracks the latent volatility. The log-squared transform flattens the ranking: RMSE compresses into a ~ 1.14 – 1.16 band and the wall-time differences become the discriminator. EKF and UKF are again the cheap-and-good option; the flow filters are not wrong on this model, just unnecessary.

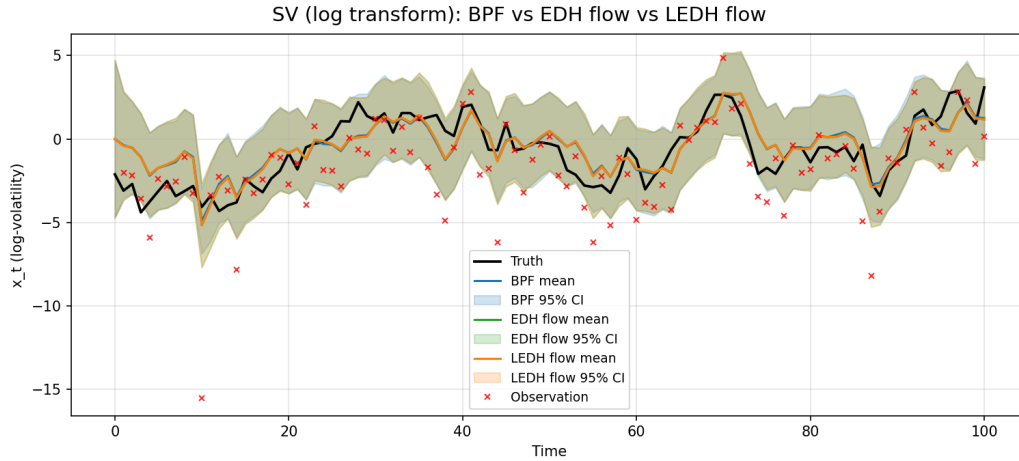


Figure 10: Log-squared stochastic volatility: bootstrap PF (blue), EDH flow (green), and LEDH flow (orange) compared against the true log-volatility (black) with back-projected observations as red crosses. After the log-squared transform, the three filters compress into a narrow band and the differences between them become invisible at this scale; the three flow variants all land in the same narrow RMSE band as the LEDH-invertible entry in Table 5.

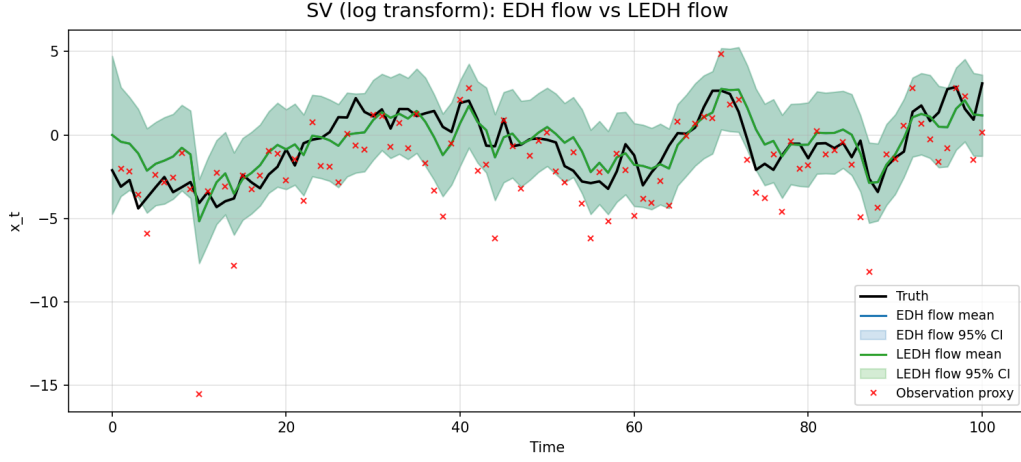


Figure 11: Log-squared stochastic volatility, EDH flow (blue) vs. LEDH flow (green). The pairwise comparison isolates the effect of per-particle re-linearisation (LEDH) against a single global linearisation per time step (EDH) on the transformed model. Per-particle re-linearisation buys essentially nothing here: the two trajectories overlap within plot precision.

Stochastic volatility 2D (filter accuracy). Before the SV2D model reappears in §6.6 as an HMC gradient-failure case, I record the filter-level result first. The three figures below compare bootstrap PF, EDH flow, LEDH flow, and the invertible counterparts of the flow variants on an SV2D trajectory with $N = 200$ particles and $T = 100$ time steps. A clean filter result on this model does not imply HMC will succeed on it — that is the point of §6.6 — but a broken filter result would imply HMC cannot possibly succeed.

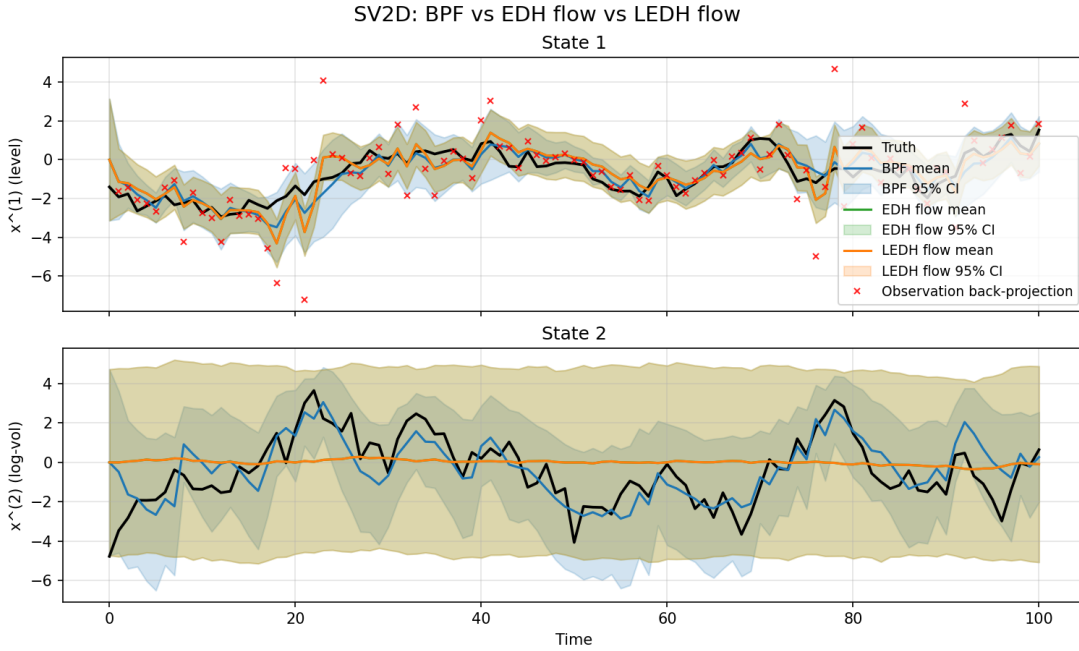


Figure 12: Stochastic volatility 2D: bootstrap PF (blue), EDH flow (green), LEDH flow (orange), overlaid on the true state (black). The first state component $x^{(1)}$ is the linear drift driver; the second $x^{(2)}$ is the log-volatility scale. Back-projected observations (red crosses) are shown only on the first panel because the SV2D observation model has a scalar output $y_t = b x_t^{(1)} + \exp(x_t^{(2)}/2) v_t$ with no per-dimension back-projection of $x^{(2)}$.

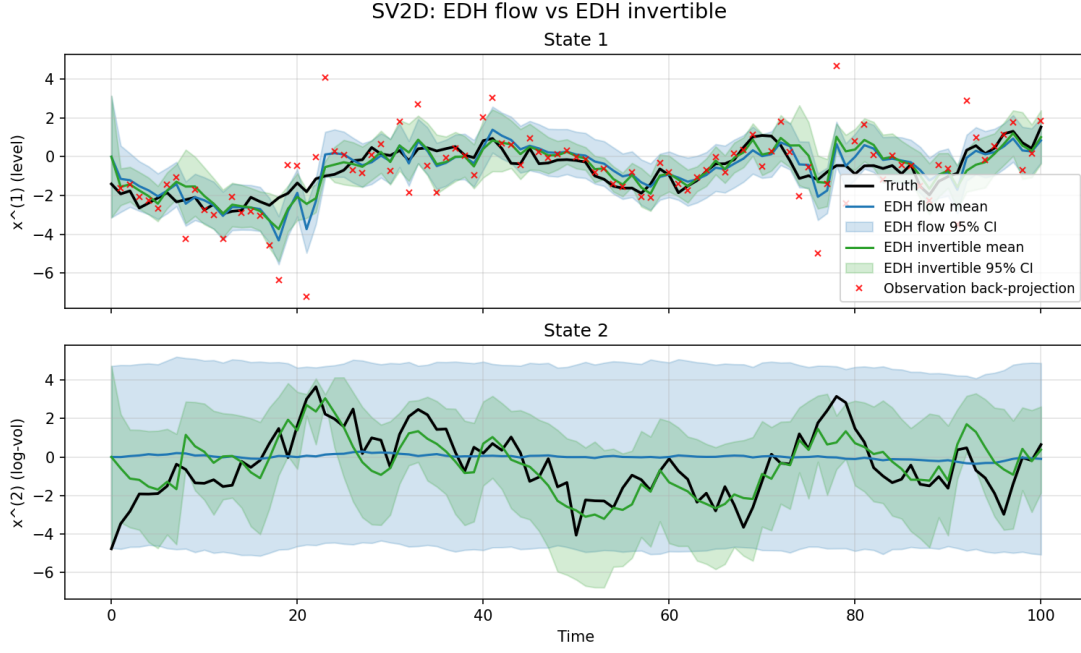


Figure 13: Stochastic volatility 2D, EDH flow (blue) vs. EDH invertible (green). Both share the same global linearisation per time step; only the Jacobian correction differs. On this high-dimensional non-linear target the two variants diverge visibly in places (max mean difference ≈ 3.2 in the log-volatility component), more than on range-bearing where the correction is essentially free.

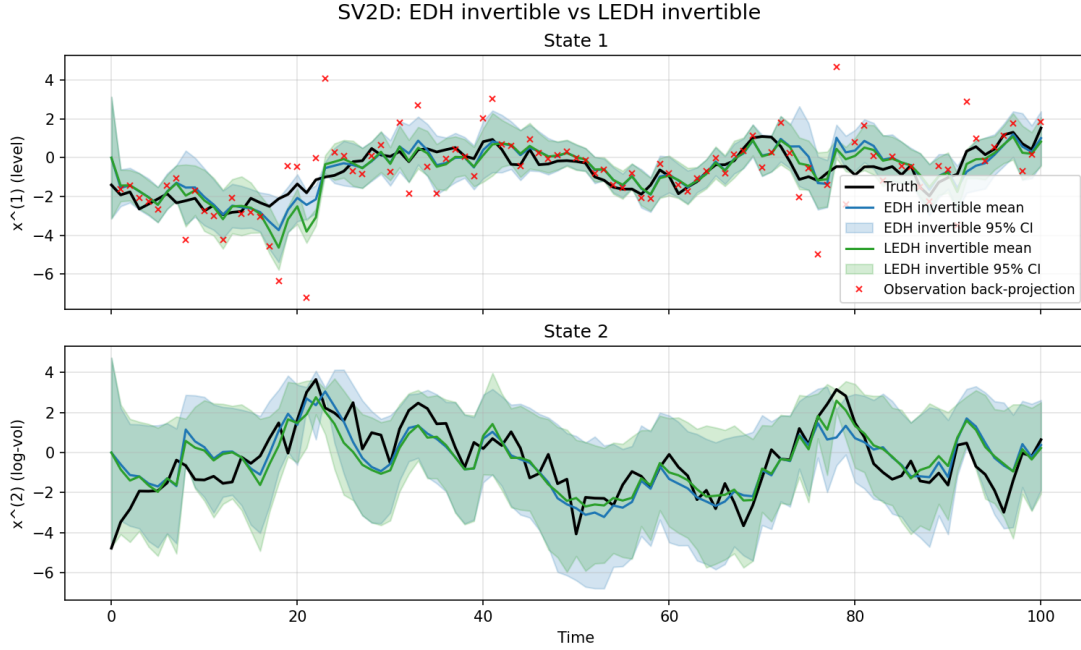


Figure 14: Stochastic volatility 2D, EDH invertible (blue) vs. LEDH invertible (green). Both carry the Jacobian correction; only the linearisation scheme differs (global vs. per-particle). On SV2D the per-particle LEDH variant is *not* consistently better than global-EDH; the two filters cross each other in places and the ranking is not decisive.

4.4 Invertible Flow Filter

4.4.1 Why do we need Jacobian computation?

The invertible flow filter [4] corrects for the volume distortion introduced by discretization. The continuous flow $d\mathbf{x}/d\lambda = f(\mathbf{x}, \lambda)$ is exact by construction: the Fokker-Planck equation guarantees that following the flow moves probability mass correctly from $g(x)$ to $g(x)h(x)$. In practice we integrate with finite steps ε_j , so each λ step applies the discrete affine map

$$\boldsymbol{\eta}^{(i)} \leftarrow (I + \varepsilon_j A_j^{(i)}) \boldsymbol{\eta}^{(i)} + \varepsilon_j \mathbf{b}_j^{(i)}. \quad (65)$$

This is an invertible linear map with Jacobian $\det(I + \varepsilon_j A_j^{(i)})$. Unless $\det(I + \varepsilon_j A) = 1$ for every step (i.e., the map is volume-preserving), the discrete flow changes the volume element and therefore distorts the density. The continuous flow does not have this problem—the exact Fokker-Planck solution implicitly accounts for volume changes—but any finite discretization introduces this error. To correct for it, we re-introduce importance weights. The weight accumulated over all n_λ steps of the flow at time k is

$$\log w_k^{(i)} = \log w_{k-1}^{(i)} + \sum_{j=1}^{n_\lambda} \log |\det(I + \varepsilon_j A_j^{(i)})|, \quad (66)$$

which is then normalized across particles. A positive log-Jacobian means the map expanded the local volume (particles spread out), contributing upward weight; a negative one means compression, contributing downward weight. After the full flow the particle set $\{\boldsymbol{\eta}^{(i)}, w^{(i)}\}$ is a weighted approximation to the posterior, and the usual ESS-triggered resampling is applied.

For the EDH variants (both fixed and intermediate re-linearization), A_j is a global matrix—the same for every particle at step j . The Jacobian factor $\det(I + \varepsilon_j A_j)$ is therefore identical for all particles. Under weight normalization, a common multiplicative factor in all $w^{(i)}$ cancels exactly. The relative weights remain uniform, and no correction is needed—the EDH flow filter without weights is already self-consistent. This is why the unweighted EDH flow (Algorithm 7 and 8) is not simply an approximation that ignores the Jacobian; the Jacobian genuinely does not affect the normalized weights.

For LEDH, $A_j^{(i)}$ is particle-specific: different particles are linearized at different positions, so they are stretched or compressed by different amounts at each step. The Jacobian $\det(I + \varepsilon_j A_j^{(i)})$ now varies across particles and does not cancel under normalization. Ignoring it would let the flow silently distort the distribution in a particle-dependent way. The invertible correction in (66) accounts for exactly this: it reweights each particle by how much the flow expanded or contracted the space around it, restoring consistency between the transported point cloud and the target density.

At each λ step, $A^{(i)} = -\frac{1}{2} \mathbf{P} \mathbf{H}^{(i)\top} (\lambda \mathbf{H}^{(i)} \mathbf{P} \mathbf{H}^{(i)\top} + \mathbf{R})^{-1} \mathbf{H}^{(i)}$ has rank at most d_z (the observation dimension), because it is a product $U W V^\top$ where $U = \mathbf{P} \mathbf{H}^{(i)\top}$ is $d_x \times d_z$. A naive determinant over $d_x \times d_x$ matrices would be expensive. Instead, apply Sylvester’s determinant theorem $\det(I_{d_x} + U V^\top) = \det(I_{d_z} + V^\top U)$ to reduce the computation to a $d_z \times d_z$ determinant:

$$\log |\det(I_{d_x} + \varepsilon_j A^{(i)})| = \log |\det(I_{d_z} - \frac{\varepsilon_j}{2} \mathbf{H}^{(i)} \mathbf{P} \mathbf{H}^{(i)\top} (\lambda \mathbf{H}^{(i)} \mathbf{P} \mathbf{H}^{(i)\top} + \mathbf{R})^{-1})|. \quad (67)$$

When $d_z \ll d_x$ this is a substantial saving. In the common case $d_z = d_x = 1$ it reduces to a scalar logarithm. The accumulated log-weight over all n_λ steps is computed by summing this scalar (or small-matrix determinant) at each step, adding negligible overhead compared to the flow integration itself.

Algorithm 10 EDH Invertible Particle Flow Filter (Intermediate Re-linearization)

Require: Particles $\{\boldsymbol{\eta}^{(i)}, w^{(i)}\}_{i=1}^N$, parallel EKF/UKF, ESS threshold

```

1: for  $k = 1, 2, \dots, T$  do
2:   Predict (EKF/UKF):  $(\bar{\boldsymbol{\eta}}_0, \mathbf{P}) \leftarrow$  EKF/UKF predict
3:   Predict (particles):  $\boldsymbol{\eta}_0^{(i)} \sim p(\mathbf{x}_k \mid \boldsymbol{\eta}^{(i)})$ , store pre-flow particles
4:   Precompute  $\mathbf{R}^{-1}$ 
5:   Initialize  $\lambda \leftarrow 0$ ,  $\boldsymbol{\eta}_1^{(i)} \leftarrow \boldsymbol{\eta}_0^{(i)}$ ,  $\bar{\boldsymbol{\eta}}^{(\lambda)} \leftarrow \bar{\boldsymbol{\eta}}_0$ 
6:   for  $j = 1, 2, \dots, n_\lambda$  do
7:      $\lambda \leftarrow \lambda + \varepsilon_j$ 
8:      $\mathbf{H} \leftarrow \left. \frac{\partial \mathbf{h}}{\partial \mathbf{x}} \right|_{\bar{\boldsymbol{\eta}}^{(\lambda)}}$  (re-linearize at current global mean)
9:      $\mathbf{e} \leftarrow \mathbf{h}(\bar{\boldsymbol{\eta}}^{(\lambda)}) - \mathbf{H}\bar{\boldsymbol{\eta}}^{(\lambda)}$  (nonlinear correction)
10:     $\mathbf{A} \leftarrow -\frac{1}{2}\mathbf{P}\mathbf{H}^\top(\lambda\mathbf{H}\mathbf{P}\mathbf{H}^\top + \mathbf{R})^{-1}\mathbf{H}$ 
11:     $\mathbf{b} \leftarrow (\mathbf{I} + 2\lambda\mathbf{A})[(\mathbf{I} + \lambda\mathbf{A})\mathbf{P}\mathbf{H}^\top\mathbf{R}^{-1}(\mathbf{z}_k - \mathbf{e}) + \mathbf{A}\bar{\boldsymbol{\eta}}_0]$ 
12:     $\bar{\boldsymbol{\eta}}^{(\lambda)} \leftarrow \bar{\boldsymbol{\eta}}^{(\lambda)} + \varepsilon_j(\mathbf{A}\bar{\boldsymbol{\eta}}^{(\lambda)} + \mathbf{b})$  (advance mean for next step)
13:     $\boldsymbol{\eta}_1^{(i)} \leftarrow \boldsymbol{\eta}_1^{(i)} + \varepsilon_j(\mathbf{A}\boldsymbol{\eta}_1^{(i)} + \mathbf{b})$  for all  $i$ 
14:  end for
15:  Weight update:
16:   $\tilde{w}^{(i)} \leftarrow w^{(i)} \cdot \frac{p(\mathbf{z}_k \mid \boldsymbol{\eta}_1^{(i)})}{p(\mathbf{z}_k \mid \boldsymbol{\eta}_0^{(i)})}$  (importance weight correction)
17:   $w^{(i)} \leftarrow \tilde{w}^{(i)} / \sum_j \tilde{w}^{(j)}$  (normalize)
18:   $\boldsymbol{\eta}^{(i)} \leftarrow \boldsymbol{\eta}_1^{(i)}$ 
19:  if  $N_{\text{eff}} = 1 / \sum_i (w^{(i)})^2 < \text{threshold}$  then
20:    Resample  $\{\boldsymbol{\eta}^{(i)}, w^{(i)}\}$  (systematic resampling)
21:  end if
22:  Update (EKF/UKF):  $(\bar{\boldsymbol{\eta}}, \mathbf{P}) \leftarrow$  EKF/UKF update( $\bar{\boldsymbol{\eta}}_0, \mathbf{P}, \mathbf{z}_k$ )
23: end for

```

Algorithm 11 LEDH Invertible Particle Flow Filter (Local Linearization + Jacobian Tracking)

Require: Particles $\{\boldsymbol{\eta}^{(i)}, w^{(i)}\}_{i=1}^N$, per-particle batched EKF/UKF, ESS threshold

Require: Weight clip range C (optional, e.g. $C = 50$)

```
1: for  $k = 1, 2, \dots, T$  do
2:   Predict (batched EKF/UKF):
3:    $(\bar{\boldsymbol{\eta}}_0^{(i)}, \mathbf{P}^{(i)}) \leftarrow$  batched EKF/UKF predict( $\boldsymbol{\eta}^{(i)}, \mathbf{P}^{(i)}$ ) for all  $i$ 
4:   Predict (particles):  $\boldsymbol{\eta}_0^{(i)} \sim p(\mathbf{x}_k | \boldsymbol{\eta}^{(i)})$ 
5:   Precompute  $\mathbf{R}^{-1}$ 
6:   Initialize  $\lambda \leftarrow 0$ ,  $\boldsymbol{\eta}_1^{(i)} \leftarrow \boldsymbol{\eta}_0^{(i)}$ ,  $\log \theta^{(i)} \leftarrow 0$ 
7:   for  $j = 1, 2, \dots, n_\lambda$  do
8:      $\lambda \leftarrow \lambda + \varepsilon_j$ 
9:     for each particle  $i$  (batched) do
10:       $\mathbf{H}^{(i)} \leftarrow \frac{\partial \mathbf{h}}{\partial \mathbf{x}} \Big|_{\boldsymbol{\eta}_1^{(i)}}$  (re-linearize at current position)
11:       $\mathbf{e}^{(i)} \leftarrow \mathbf{h}(\boldsymbol{\eta}_1^{(i)}) - \mathbf{H}^{(i)} \boldsymbol{\eta}_1^{(i)}$ 
12:      Compute  $\mathbf{A}^{(i)}(\lambda), \mathbf{b}^{(i)}(\lambda)$  using  $\mathbf{H}^{(i)}, \mathbf{P}^{(i)}, \bar{\boldsymbol{\eta}}_0^{(i)}$ 
13:      end for
14:       $\boldsymbol{\eta}_1^{(i)} \leftarrow \boldsymbol{\eta}_1^{(i)} + \varepsilon_j (\mathbf{A}^{(i)} \boldsymbol{\eta}_1^{(i)} + \mathbf{b}^{(i)})$  for all  $i$ 
15:       $\mathbf{M}^{(i)} \leftarrow \mathbf{I} + \varepsilon_j \mathbf{A}^{(i)}$  (local Jacobian of flow map)
16:       $\log \theta^{(i)} \leftarrow \log \theta^{(i)} + \log |\det(\mathbf{M}^{(i)})|$  (accumulate log-det)
17:    end for
18:    Weight update with Jacobian correction:
19:     $\log \theta^{(i)} \leftarrow \log \theta^{(i)} - \max_j \log \theta^{(j)}$  (stabilize before exp)
20:     $\theta^{(i)} \leftarrow \exp(\log \theta^{(i)})$ 
21:     $\tilde{w}^{(i)} \leftarrow w^{(i)} \cdot \frac{p(\mathbf{z}_k | \boldsymbol{\eta}_1^{(i)})}{p(\mathbf{z}_k | \boldsymbol{\eta}_0^{(i)})} \cdot \theta^{(i)}$ 
22:    if clip range  $C$  specified then
23:       $\log \tilde{w}^{(i)} \leftarrow \text{clip}(\log \tilde{w}^{(i)}, -C, +C)$  (prevent overflow)
24:    end if
25:     $w^{(i)} \leftarrow \tilde{w}^{(i)} / \sum_j \tilde{w}^{(j)}$ 
26:     $\boldsymbol{\eta}^{(i)} \leftarrow \boldsymbol{\eta}_1^{(i)}$ 
27:    Update (batched EKF/UKF): update per-particle covariances
28:    if  $N_{\text{eff}} < \text{threshold}$  then
29:      Resample  $\{\boldsymbol{\eta}^{(i)}, w^{(i)}\}$ 
30:    end if
31: end for
```

When does the Jacobian correction help? The Jacobian correction is harmless on range-bearing and should help on high-dimensional targets, but on acoustic tracking it is strictly worse than uncorrected LEDH in my runs. I defer the root-cause investigation to future work — the most likely culprits are float32 precision on the per-particle Cholesky and the log-determinant clipping I use for stability.

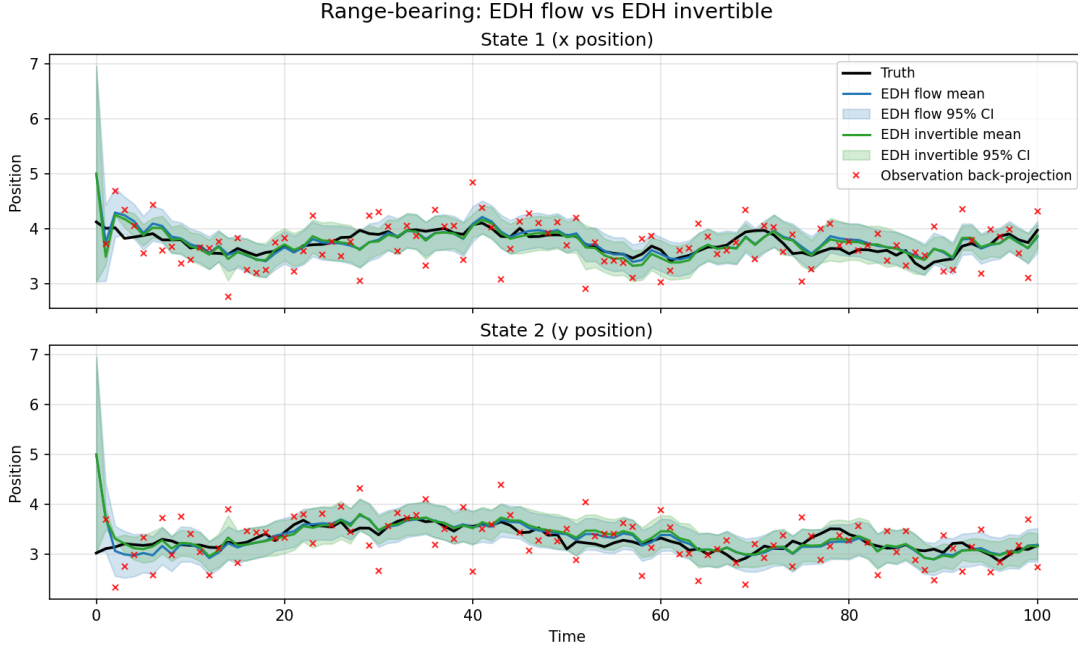


Figure 15: Range-bearing, EDH flow (blue) vs. EDH invertible (green). The two variants are visually indistinguishable: on this well-conditioned 2D model the per-particle log-determinants are close to unity at every flow step, so the Jacobian correction adds negligible information and adds negligible error. This is the control that says the correction is numerically harmless in the easy regime.

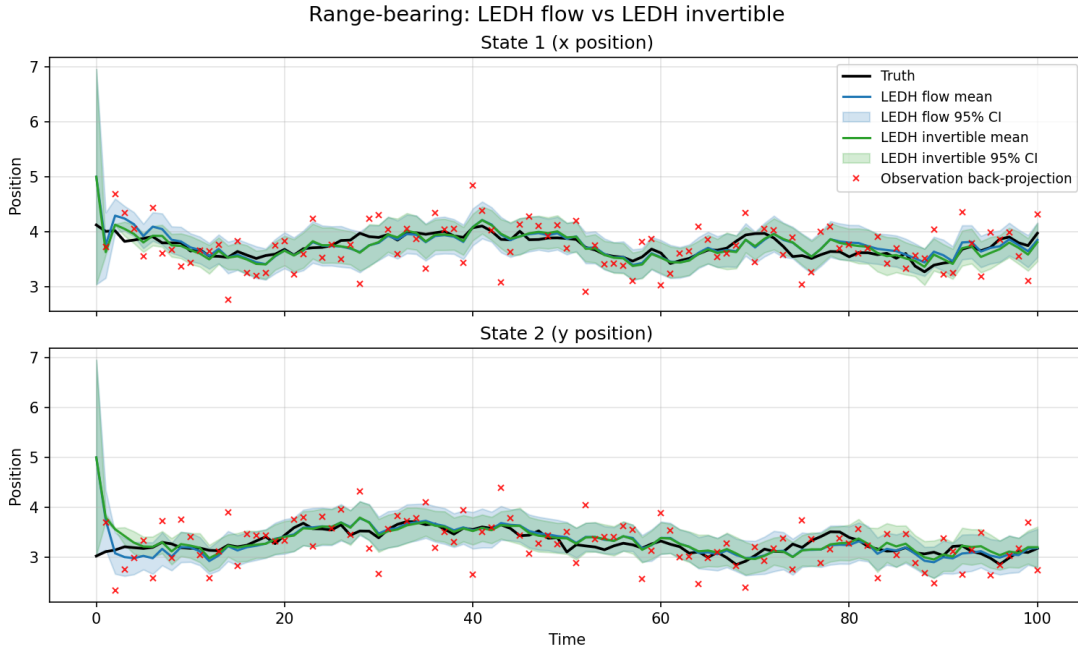


Figure 16: Range-bearing, LEDH flow (blue) vs. LEDH invertible (green). The per-particle-linearisation version of the same comparison. Conclusion is identical: on a well-conditioned low-dimensional model the invertible correction neither helps nor hurts, and the empirical tie holds whether the underlying linearisation is global or local.

4.5 Stochastic Flow Filter

I used the stochastic particle flow filter of [5] as the SDE analogue of deterministic EDH:

$$d\mathbf{x} = [A(\lambda)\mathbf{x} + b(\lambda)] d\lambda + \sqrt{Q} dW_\lambda,$$

Here $Q = \text{diag}(q_1, \dots, q_d)$ is diagonal. The extra noise is not free. The drift needs the score correction from Corollary 2.1 of [5]; otherwise the particles diffuse away from the homotopy density.

Deterministic EDH can re-linearise H at each particle. That is the LEDH idea. Each particle follows its own ODE velocity field, so this is well-defined. The stochastic case is stricter. The drift contains a score term:

$$f_{\text{stoch}}(\mathbf{x}, \lambda) = f_{\text{det}}(\mathbf{x}, \lambda) + \frac{1}{2} Q \nabla_{\mathbf{x}} \log p(\mathbf{x}, \lambda),$$

where $p(\mathbf{x}, \lambda) \propto g(\mathbf{x}) h(\mathbf{x})^\lambda$. The point is not the formula itself. The point is that the score must come from one density shared by all particles. The drift must satisfy

$$\frac{\partial p}{\partial \lambda} = -\nabla \cdot (f p) + \frac{1}{2} \nabla \cdot (Q Q^\top \nabla p),$$

a *global* constraint. There must be one density $p(\mathbf{x}, \lambda)$ whose score drives every particle at the same pseudo-time.

With global linearisation, H is fixed at $\bar{\eta}_0$. Then the homotopy density is Gaussian with precision $M(\lambda) = P_0^{-1} + \lambda H^\top R^{-1} H$, and the score

$$\nabla \log p(\mathbf{x}, \lambda) = -M(\lambda)(\mathbf{x} - m(\lambda))$$

is one linear function of $\mathbf{x}$. This is why the Fokker–Planck argument still goes through.

If I re-linearise at each particle $\mathbf{x}_i$, particle i gets its own H_i , precision $M_i(\lambda)$, mean $m_i(\lambda)$, and score $-M_i(\mathbf{x} - m_i)$. No single density has all of those scores. The particles stop representing one coherent density, so the stochastic-flow guarantee is gone. This is why the stochastic-flow literature uses global linearisation, and why LEDH-style local re-linearisation belongs to deterministic flows. In code, I kept the EDH structure and only changed the flow parameters. For nonlinear observations, I cannot keep updating H during the stochastic flow. The workaround is a local correction term.

The drift formulas assume $h(\mathbf{x}) = H\mathbf{x}$. If the observation is actually linear, computing H once at $\bar{\eta}_0$ is enough.

For nonlinear observations, the Taylor expansion at a point η gives

$$h(\mathbf{x}) \approx h(\eta) + H(\eta)(\mathbf{x} - \eta) = H(\eta)\mathbf{x} + \underbrace{[h(\eta) - H(\eta)\eta]}_e,$$

The residual e is the nonlinear part that the global linear model misses. I borrow this from LEDH but do not re-linearise during the stochastic flow. I compute $e_0 = h(\bar{\eta}_0) - H\bar{\eta}_0$ once and replace z by $z_{\text{corr}} = z - e_0$ in both the deterministic drift and the score correction. For linear observations, $e_0 = 0$.

This correction is essential on nonlinear models. On range-bearing, uncorrected stochastic EDH diverges. The local correction makes the same filter work (Figure 17). It costs one extra evaluation of $h(\bar{\eta}_0)$ per time step. The flow is stiff near $\lambda = 0$. There, $M(\lambda) = P_0^{-1} + \lambda H^\top R^{-1} H$ is still dominated by the anisotropic prior. Near $\lambda = 1$, the measurement term regularises the precision and the stiffness weakens. A fine uniform grid keeps Euler–Maruyama stable, so the stiffness is not fatal if the step size is small enough.

I used a geometric schedule in practice: $\varepsilon_j = \varepsilon_1 q^{j-1}$ with $q = 1.2$. This puts small steps near $\lambda = 0$ and larger steps near $\lambda = 1$. It solved the practical stiffness issue without an optimisation problem. I spent time trying to reproduce the stiffness-mitigation result of [6], but I could not reproduce it. The filter worked. The proposed schedule did not give a useful gain in my runs; a fine uniform grid or the geometric heuristic was enough. I did not try changing the integration scheme. I also followed the pseudocode and evaluated the parameters after updating the step count. Since the first step is the stiffest one, this ordering may explain why I could not reproduce the paper.

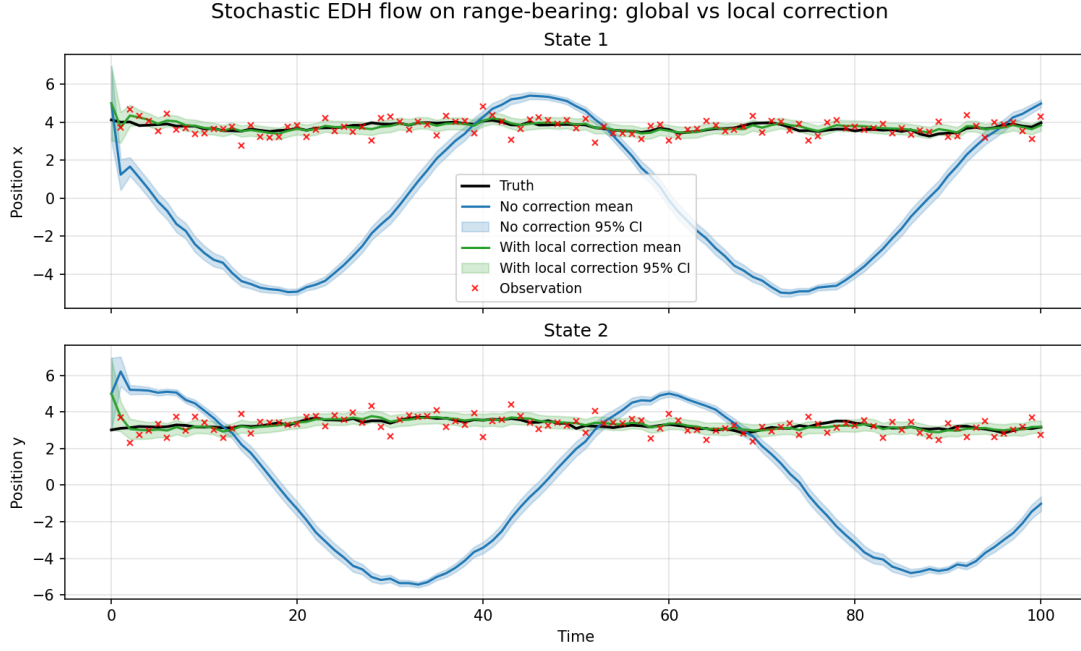


Figure 17: Stochastic EDH flow on the range-bearing model: no correction (blue, $\text{RMSE} \approx 5.28$) versus with local correction $e_0 = h(\bar{\eta}_0) - \mathbf{H}\bar{\eta}_0$ (green, $\text{RMSE} \approx 0.13$). True trajectory in black; observations as red crosses back-projected to position coordinates. The uncorrected run diverges because the global linearisation is stale throughout the flow; adding the local correction makes the same drift equation track the truth within one position unit.

Table 6: Stochastic EDH on the range-bearing model. ‘No correction’ means $e_0 = 0$ and the drift is derived under pure global linearisation; ‘+ local correction’ uses $e_0 = h(\bar{\eta}_0) - \mathbf{H}\bar{\eta}_0$. ‘100 steps’ forces a uniform $n_\lambda = 100$ grid; ‘optimal’ uses the stiffness-mitigation schedule of [6].

Variant	RMSE	Log-lik	Wall (s)
No correction, default n_λ	5.276	-17401.29	129.17
No correction, $n_\lambda = 100$	1.307	-23799.91	104.78
No correction, optimal schedule	1.307	-23780.79	414.63
+ local correction, default n_λ	0.131	+110.06	128.74
+ local correction, $n_\lambda = 100$	0.252	-146.90	106.97
+ local correction, optimal schedule	0.131	+110.10	359.14

Empirical takeaway (§4.5). Three facts from Table 6. (i) Without the local correction e_0 , stochastic flow on the range-bearing model is as bad as the Kalman filter on the same task — the uncorrected drift diverges or tracks the prior mean, because the linearisation is stale throughout the flow. (ii) With the local correction, the default stochastic EDH run works ($\text{RMSE} = 0.131$), but it does *not* beat the deterministic EDH/LEDH flows, which already reach comparable or better RMSE at a fraction of the compute. The $n_\lambda = 100$ variant settles near 0.25. (iii) The ‘optimal’ step-size schedule of [6] did not help in our hands: we could not reproduce the stiffness-mitigation gain, and a fine uniform schedule plus the geometric-ratio heuristic was sufficient.

4.6 Kernel Flow Filter

4.6.1 Overview

Kernel flow replaces the affine drift with an RKHS velocity field [7, 8]. Instead of moving particles along a fixed affine schedule, I move them in the direction that decreases the gap between the particle density and the target as quickly as possible. Define the target posterior $\pi(x) := p(x_t|z_{1:t})$ and the intermediate density $q_\lambda(x)$ at pseudo-time $\lambda \in [0, 1]$. The endpoints are $q_0 = p(x_t|z_{1:t-1})$ and $q_1 = \pi$.

$$\frac{dx_\lambda}{d\lambda} = v_\lambda(x_\lambda) \quad (68)$$

Choose v_λ to decrease the KL divergence as fast as possible:

$$D_{KL}(q_\lambda \parallel \pi) = \int q_\lambda(x) \log \frac{q_\lambda(x)}{\pi(x)} dx \quad (69)$$

For the code, I needed a computable drift. The intuition is simple: move each point along the steepest descent direction for this density mismatch.

4.6.2 Derivation

Deriving Optimal Velocity via Kernel Restriction. Under velocity field v , the density evolves by the Liouville equation:

$$\frac{\partial q_\lambda}{\partial \lambda} = -\nabla_x \cdot (q_\lambda v) \quad (70)$$

Differentiate D_{KL} in pseudo-time:

$$\frac{d}{d\lambda} D_{KL} = \int \frac{\partial q_\lambda}{\partial \lambda} [\log q_\lambda(x) - \log \pi(x) + 1] dx \quad (71)$$

$$= - \int \nabla_x \cdot (q_\lambda v) [\log q_\lambda(x) - \log \pi(x) + 1] dx \quad (72)$$

Using $\nabla_x q_\lambda = q_\lambda \nabla_x \log q_\lambda$ and integrating by parts:

$$\frac{d}{d\lambda} D_{KL} = - \int q_\lambda [\nabla_x \log \pi(x) \cdot v + \nabla_x \cdot v] dx \quad (73)$$

The derivative gives the vector field along the trajectory. To make it computable, I restrict v to functions generated by a kernel $K(x, x')$. Any such function has the form:

$$v(x) = \int \alpha(x') K(x', x) dx' \quad (74)$$

for some coefficient function $\alpha(x')$. I substitute this into the directional derivative.

$$\frac{d}{d\lambda} D_{KL} = - \int q_\lambda(x) \left[\nabla_x \log \pi(x) \cdot \int \alpha(x') K(x', x) dx' + \nabla_x \cdot \int \alpha(x') K(x', x) dx' \right] dx \quad (75)$$

Interchange the integrals and use integration by parts:

$$\frac{d}{d\lambda} D_{KL} = \int \alpha(x) \cdot \phi(x) dx \quad (76)$$

where

$$\phi(x) = - \int q_\lambda(x') [K(x, x') \nabla_{x'} \log \pi(x') + \nabla_{x'} K(x, x')] dx' \quad (77)$$

This gives $\frac{d}{d\lambda} D_{KL} = \langle \alpha, \phi \rangle$ in function space.

To make KL decrease fastest, choose α antiparallel to ϕ . By Cauchy-Schwarz, the minimum is achieved when:

$$\alpha(x) = -\phi(x) \quad (78)$$

The optimal velocity field is therefore:

$$v_\lambda(x) = \int \alpha(x') K(x', x) dx' \quad (79)$$

$$= - \int \phi(x') K(x', x) dx' \quad (80)$$

The reproducing property of the kernel gives:

$$\boxed{v_\lambda(x) = \mathbb{E}_{x' \sim q_\lambda} [K(x, x') \nabla_{x'} \log \pi(x') + \nabla_{x'} K(x, x')]} \quad (81)$$

With particles $\{x_\lambda^{(i)}\}_{i=1}^{N_p} \sim q_\lambda$, the Monte Carlo approximation is:

$$v_\lambda(x) \approx \frac{1}{N_p} \sum_{i=1}^{N_p} \left[K(x_\lambda^{(i)}, x) \nabla \log \pi(x_\lambda^{(i)}) + \nabla K(x_\lambda^{(i)}, x) \right] \quad (82)$$

where the posterior gradient is

$$\nabla \log \pi(x) = \nabla \log p(y_t | x) + \nabla \log p(x | y_{1:t-1}) \quad (83)$$

with the first term given by the likelihood gradient and the second term approximated from the ensemble prior. The update rule is then

$$x_{\lambda+\epsilon}^{(j)} = x_\lambda^{(j)} + \frac{\epsilon}{N_p} \sum_{i=1}^{N_p} \left[K(x_\lambda^{(i)}, x_\lambda^{(j)}) \nabla \log \pi(x_\lambda^{(i)}) + \nabla K(x_\lambda^{(i)}, x_\lambda^{(j)}) \right] \quad (84)$$

All particles keep equal weight $w^{(i)} = 1/N_p$ throughout, so no resampling step is needed. The method only needs a differentiable likelihood $p(y_t | x_t)$; it is not tied to Gaussian observation noise.

Scalar Kernel I use the isotropic Gaussian as the scalar kernel:

$$K(x, x') = \exp \left(-\frac{1}{2\alpha\sigma^2} \|x - x'\|^2 \right), \quad \alpha \sim n_x \quad (85)$$

Its gradient is $\nabla_x K(x, x') = -\frac{1}{\alpha\sigma^2} (x - x') K(x, x')$.

A single scalar $K(x, x')$ weights all components identically.

Matrix-Valued Kernel The matrix-valued kernel measures distance independently per component:

$$\mathbf{K}(x, x') = \text{diag} \left[K^{(1)}(x, x'), \dots, K^{(n_x)}(x, x') \right] \quad (86)$$

where

$$K^{(d)}(x, x') = \exp \left(-\frac{(x^{(d)} - x'^{(d)})^2}{2\alpha(\sigma^{(d)})^2} \right), \quad \alpha = \frac{1}{N_p} \quad (87)$$

The repulsion is component-wise:

$$[\nabla_x \cdot \mathbf{K}(x, x')]^{(d)} = -\frac{x^{(d)} - x'^{(d)}}{\alpha(\sigma^{(d)})^2} K^{(d)}(x, x') \quad (88)$$

Observed dimensions then keep diversity even when unobserved dimensions spread out.

4.6.3 Matrix Kernel Preventing Collapse

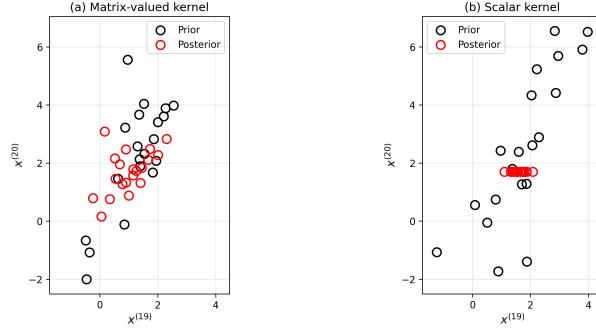


Figure 18: Switching to the matrix kernel prevents x_{20} from collapsing to one point.

4.6.4 Algorithm

Algorithm 12 Kernel Flow Filter — Scalar Kernel

Require: Particles $\{\mathbf{x}^{(i)}\}_{i=1}^N$ with equal weights, scalar bandwidth $\alpha = 1/N$

Require: Localization matrix $\mathbf{C}$ (Schur product mask), pseudo-time steps n_s , step size Δt

- 1: **for** $k = 1, 2, \dots, T$ **do**
 - 2: **Predict:** $\mathbf{x}_0^{(i)} \sim p(\mathbf{x}_k | \mathbf{x}^{(i)})$ for all i
 - 3: Compute ensemble mean $\bar{\mathbf{x}}_0$, localized covariance $\mathbf{B} = \mathbf{C} \circ \hat{\mathbf{P}}_{\text{sample}}$
 - 4: Precompute $\mathbf{B}^{-1}$, $\mathbf{R}^{-1}$, bandwidth $\sigma^2 = \alpha \cdot \text{tr}(\mathbf{B})/d$
 - 5: **for** $s = 1, \dots, n_s$ **do**
 - 6: **for** each particle j (batched) **do**
 - 7: **Posterior gradient:**
 - 8: $\nabla \log p(\mathbf{z}_k | \mathbf{x}^{(j)}) = \mathbf{H}^{(j)\top} \mathbf{R}^{-1}(\mathbf{z}_k - \mathbf{h}(\mathbf{x}^{(j)}))$
 - 9: $\nabla \log p(\mathbf{x}^{(j)}) = -\mathbf{B}^{-1}(\mathbf{x}^{(j)} - \bar{\mathbf{x}}_0)$
 - 10: $\nabla \log \pi(\mathbf{x}^{(j)}) = \nabla \log p(\mathbf{z}_k | \mathbf{x}^{(j)}) + \nabla \log p(\mathbf{x}^{(j)})$
 - 11: **Scalar kernel:** $K(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = \exp\left(-\frac{\|\mathbf{x}^{(i)} - \mathbf{x}^{(j)}\|^2}{2\sigma^2}\right)$
 - 12: **Kernel gradient:** $\nabla_{\mathbf{x}^{(i)}} K = -\frac{1}{\sigma^2}(\mathbf{x}^{(i)} - \mathbf{x}^{(j)})K(\mathbf{x}^{(i)}, \mathbf{x}^{(j)})$
 - 13: **Flow:** $\mathbf{v}^{(j)} = \frac{1}{N} \sum_{i=1}^N [K(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) \nabla \log \pi(\mathbf{x}^{(i)}) + \nabla_{\mathbf{x}^{(i)}} K(\mathbf{x}^{(i)}, \mathbf{x}^{(j)})]$
 - 14: **end for**
 - 15: Precondition: $\mathbf{v}^{(j)} \leftarrow \mathbf{B} \mathbf{v}^{(j)}$
 - 16: $\mathbf{x}^{(j)} \leftarrow \mathbf{x}^{(j)} + \Delta t \cdot \mathbf{v}^{(j)}$ for all j
 - 17: **end for**
 - 18: **end for**
-

Algorithm 13 Kernel Flow Filter — Matrix (Tensor) Kernel

Require: Same setup as scalar kernel. Per-dimension bandwidth: $(\sigma^{(d)})^2 = \alpha \cdot B_{dd}$

```
1: for  $k = 1, 2, \dots, T$  do
2:   Predict and compute  $\bar{\mathbf{x}}_0, \mathbf{B}, \mathbf{B}^{-1}, \mathbf{R}^{-1}$  as in scalar kernel
3:   for  $s = 1, \dots, n_s$  do
4:     for each particle  $j$  (batched) do
5:       Compute  $\nabla \log \pi(\mathbf{x}^{(j)})$  as before
6:       Matrix kernel (diagonal, per-dimension):
7:        $K^{(d)}(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = \exp\left(-\frac{(x_d^{(i)} - x_d^{(j)})^2}{2\alpha(\sigma^{(d)})^2}\right)$  for each dimension  $d$ 
8:        $[\nabla_{\mathbf{x}^{(i)}} \cdot \mathbf{K}]^{(d)} = -\frac{x_d^{(i)} - x_d^{(j)}}{\alpha(\sigma^{(d)})^2} K^{(d)}(\mathbf{x}^{(i)}, \mathbf{x}^{(j)})$ 
9:       Per-dimension flow:
10:       $v_d^{(j)} = \frac{1}{N} \sum_{i=1}^N [K^{(d)}(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) [\nabla \log \pi(\mathbf{x}^{(i)})]_d + [\nabla_{\mathbf{x}^{(i)}} \cdot \mathbf{K}]^{(d)}]$ 
11:     end for
12:     Precondition:  $\mathbf{v}^{(j)} \leftarrow \mathbf{B} \mathbf{v}^{(j)}$ 
13:      $\mathbf{x}^{(j)} \leftarrow \mathbf{x}^{(j)} + \Delta t \cdot \mathbf{v}^{(j)}$  for all  $j$ 
14:   end for
15: end for
```

Empirical takeaway (§4.6). The matrix-valued kernel prevents the state-dimension collapse shown in Figure 18, which is the whole reason to use this filter. The use case is niche but concrete: a low-dimensional, well-conditioned target where the particle budget is severely constrained (on the order of $N \sim 20$) and the user is willing to pay the $O(N^2)$ kernel-matrix cost in exchange for keeping the small ensemble from collapsing.

5 Resampling and Optimal Transport

5.1 Two Problems with Resampling

Before we get into details, one framing up-front: OT resampling’s primary value in this report is *differentiability*, not lower filtering RMSE. Table 8 in §5.4 will show OT paying a 20–180× compute tax on stochastic volatility without beating stop-gradient systematic or soft resampling on RMSE. The motivation for paying that tax only lands in §6, where the gradient surface determines whether HMC can recover the parameter posterior.

Previously, we have discussed the sample impoverishment problem introduced by low quality resampling. Therefore, we need to think about how to improve it. Before and after resampling, we are preserving the distribution. In [9], the author proposed to use optimal transport to do the resampling. An easy way to see this is that, before the resampling, we have particle location and weights at $\{\omega_i, x_i\}$, and after the resampling, we have particle location and weights at $\{\tilde{\omega}_i, \tilde{x}_i\}$. If we think x_i are basis vectors and ω_i are the coefficients, we are dealing with a linear transformation problem. The transformation constructs an equal-weight approximation that reduces resampling variance and better preserves the structure of the distribution.

To put it sharply, we want to make sure $\{\omega_i, x_i\}$ and $\{\tilde{\omega}_i, \tilde{x}_i\}$ represent the same distribution with $\tilde{\omega}_i \equiv \frac{1}{N}$, where N is the total number of particles. There are two problems on resampling that we need to address separately:

Problem 1: Sampling noise. OT resampling is deterministic instead of stochastic, which means it avoids the sampling noise introduced by systematic resampling, therefore removing its contribution to the variance of the particle cloud, namely increasing the quality of covariance estimation. But the classical OT formulation relies on solving a discrete Linear Programming (LP) problem to find the optimal transport plan. LP solvers like the Simplex algorithm or Interior Point methods walk along edges of the polytope or navigate through its interior. They do not need derivatives of the optimal solution with respect to inputs—they only need the cost matrix (particle distances) and constraints (matching weights). This is where the second problem comes in.

Problem 2: Non-differentiability. The LP-based OT plan is not differentiable with respect to the inputs. For our purpose—building a differentiable particle filter whose log-likelihood can be differentiated with respect to model parameters—this is a showstopper. The key innovation of [10] is to introduce entropy regularization so that the optimal coupling P^* becomes differentiable as a function of the cost matrix, therefore turning the whole resampling operation differentiable in principle. The regularized transport cost is

$$W_{2,\epsilon}^2(\alpha_N, \beta_N) = \min_{P \in S(a,b)} \sum_{i,j=1}^N p_{i,j} \left(c_{i,j} + \epsilon \log \frac{p_{i,j}}{a_i b_j} \right) \quad (89)$$

The practical issue is computing the gradient faithfully through the iterative Sinkhorn solver that finds P^* . We will see in the following subsections that the regularized OT map is differentiable in principle, but getting an accurate backward pass requires care. This is our focus for the rest of this section.

By adding the entropy term $\epsilon \log \frac{p_{i,j}}{a_i b_j}$ instead of $\epsilon \log p_{i,j}$, the authors of [10] made it possible to recover the transport plan from the dual variables alone. This objective is the entropic optimal transport problem, equivalent up to constants under fixed marginals to $\langle \mathbf{C}, \mathbf{P} \rangle - \epsilon \mathbf{H}(\mathbf{P})$. The dual recovery step is $\mathcal{O}(N)$ per entry, compared to $\mathcal{O}(N^2)$ for the primal LP of [9], which opens the possibility for modeling more complicated systems with large numbers of particles.

5.2 Soft Resampling

Soft resampling [11] is a compromise between fully stochastic systematic resampling and fully deterministic OT resampling. The idea is to blend the particle weights with a uniform distribution before selection, then apply an importance correction after. The algorithm is given below.

Algorithm 14 Soft Resampling

Require: Particles $\{\mathbf{x}^{(i)}, w^{(i)}\}_{i=1}^N$ (weights normalized: $\sum_i w^{(i)} = 1$)**Require:** Trade-off parameter $\alpha \in [0, 1]$ ($\alpha = 1$: hard resampling, $\alpha = 0$: pure importance sampling)1: **Mixture distribution:**2: $q^{(i)} \leftarrow \alpha w^{(i)} + (1 - \alpha)/N$ for $i = 1, \dots, N$ (q is strictly positive)3: **Systematic sampling from q :**4: Draw $u \sim \mathcal{U}[0, 1/N]$ 5: $u^{(i)} \leftarrow u + (i - 1)/N$ for $i = 1, \dots, N$ 6: Select indices j_i via cumulative sum of q : $\sum_{m=1}^{j_i-1} q^{(m)} < u^{(i)} \leq \sum_{m=1}^{j_i} q^{(m)}$ 7: $\tilde{\mathbf{x}}^{(i)} \leftarrow \mathbf{x}^{(j_i)}$ for $i = 1, \dots, N$ 8: **Importance weight correction:**9: $\tilde{w}^{(i)} \leftarrow w^{(j_i)}/q^{(j_i)}$ for $i = 1, \dots, N$ 10: $\tilde{w}^{(i)} \leftarrow \tilde{w}^{(i)} / \sum_j \tilde{w}^{(j)}$ (normalize)11: **return** $\{\tilde{\mathbf{x}}^{(i)}, \tilde{w}^{(i)}\}_{i=1}^N$

The critical observation is that soft resampling is only *partially* differentiable. The selection step (line 6) is still discrete—the cumulative-sum lookup picks an integer index j_i , and the gradient does not flow through that choice. Only the importance weight correction $w^{(j_i)}/q^{(j_i)}$ on line 9 is differentiable through w . The selection gradient—how θ affects *which* particle gets picked—is still zero. We will make this precise in Section 5.4.

5.3 Entropy-Regularized OT (Sinkhorn) Resampling

The entropy-regularized OT resampling problem is a direct application of the Monge-Kantorovich framework. To set up the problem, recall that in the Monge-Kantorovich problem we have two probability vectors $\mathbf{a} \in \mathbb{R}_+^N$ and $\mathbf{b} \in \mathbb{R}_+^N$ and a cost matrix $\mathbf{C} \in \mathbb{R}^{N \times N}$. The goal is to find the optimal coupling matrix $\mathbf{P} \in \mathbb{R}_+^{N \times N}$, where P_{ij} describes the flow of mass from bin i to bin j , minimizing total transport cost:

$$\min_{\mathbf{P} \in \mathcal{U}(\mathbf{a}, \mathbf{b})} \langle \mathbf{C}, \mathbf{P} \rangle, \quad \mathcal{U}(\mathbf{a}, \mathbf{b}) = \{ \mathbf{P} \in \mathbb{R}_+^{N \times N} : \mathbf{P} \mathbf{1}_N = \mathbf{a}, \mathbf{P}^\top \mathbf{1}_N = \mathbf{b} \}. \quad (90)$$

In the resampling context, we set $\mathbf{a} = \mathbf{w}$ (particle weights as the source marginal), $\mathbf{b} = \frac{1}{N} \mathbf{1}_N$ (the uniform distribution as the target marginal), and $C_{ij} = \|\tilde{\mathbf{x}}^{(i)} - \tilde{\mathbf{x}}^{(j)}\|^2/2$ as the squared-distance cost between normalized particle positions. Solving this exactly via linear programming is computationally expensive and, more importantly, non-differentiable with respect to the inputs.

Convention used in the algorithm below. With the marginals as above, the optimal coupling $\mathbf{P}^*$ has row sums $\mathbf{w}$ (source) and column sums $\frac{1}{N} \mathbf{1}_N$ (target). The resampled particles, following Reich [9], are $\hat{\mathbf{x}}^{(i)} = (1/b_i) \sum_j P_{ji}^* \mathbf{x}^{(j)} = N \sum_j P_{ji}^* \mathbf{x}^{(j)}$, so each new particle is a convex combination of the old ones with column-wise weights of $\mathbf{P}^*$. In the implementation we build a transport matrix T such that $\hat{\mathbf{x}}^{(i)} = \sum_j T_{ij} \mathbf{x}^{(j)}$, which means $T_{ij} = N P_{ji}^*$: the code's T is N times the *transpose* of the coupling $\mathbf{P}^*$. The ‘normalise columns / apply source weights’ block in the OT resampling algorithm below is the step that converts $\mathbf{P}^*$ into this T . We flag the convention explicitly because row-vs-column and source-vs-target are easy to swap silently when translating between the Reich formula and the Sinkhorn output.

The key insight from [12] is that adding a discrete entropy regularizer to the objective,

$$\min_{\mathbf{P} \in \mathcal{U}(\mathbf{a}, \mathbf{b})} \langle \mathbf{C}, \mathbf{P} \rangle - \epsilon \mathbf{H}(\mathbf{P}), \quad \mathbf{H}(\mathbf{P}) = - \sum_{i,j} P_{ij} (\log P_{ij} - 1), \quad (91)$$

makes the problem ϵ -strongly convex, yielding a unique solution that is differentiable with respect to $\mathbf{a}$, $\mathbf{b}$, and $\mathbf{C}$. By the KKT conditions, the optimal $\mathbf{P}^*$ must have the Gibbs-kernel form:

$$P_{ij}^* = u_i K_{ij} v_j, \quad K_{ij} = \exp\left(-\frac{C_{ij}}{\epsilon}\right), \quad (92)$$

where $\mathbf{u} \in \mathbb{R}^N$ and $\mathbf{v} \in \mathbb{R}^N$ are two unknown scaling vectors. To determine them, we substitute this form back into the two marginal constraints $\mathbf{P}^* \mathbf{1}_N = \mathbf{a}$ and $(\mathbf{P}^*)^\top \mathbf{1}_N = \mathbf{b}$, obtaining the equivalent fixed-point conditions:

$$\mathbf{u} \odot (\mathbf{K} \mathbf{v}) = \mathbf{a}, \quad \mathbf{v} \odot (\mathbf{K}^\top \mathbf{u}) = \mathbf{b}, \quad (93)$$

where $\odot$ and $\oslash$ denote element-wise multiplication and division. Rearranging, we can solve for $\mathbf{u}$ and $\mathbf{v}$ alternately:

$$\mathbf{u}^{(\ell)} = \mathbf{a} \oslash (\mathbf{K} \mathbf{v}^{(\ell-1)}), \quad \mathbf{v}^{(\ell)} = \mathbf{b} \oslash (\mathbf{K}^\top \mathbf{u}^{(\ell)}), \quad (94)$$

initialized at $\mathbf{v}^{(0)} = \mathbf{1}_N$. Upon convergence, the optimal coupling is recovered as $\mathbf{P}^{(L)} = \text{diag}(\mathbf{u}^{(L)}) \cdot \mathbf{K} \cdot \text{diag}(\mathbf{v}^{(L)})$. This is the vanilla Sinkhorn algorithm.

The practical difficulty is numerical: since $K_{ij} = \exp(-C_{ij}/\epsilon)$, when particle positions are spread out or ϵ is small, the entries of $\mathbf{K}$ become extremely small, and the repeated divisions in (94) cause the scaling variables $\mathbf{u}$ and $\mathbf{v}$ to grow in the exponential domain, producing infinite or non-finite values. This is the well-known numeric instability of the vanilla Sinkhorn. The standard remedy is to absorb the scaling into dual potentials $\alpha = \epsilon \log \mathbf{u}$ and $\beta = \epsilon \log \mathbf{v}$, so that the coupling takes the form

$$T_{ij} = \exp\left(\frac{\alpha_i + \beta_j - C_{ij}}{\epsilon}\right), \quad (95)$$

and the marginal-matching updates (94) turn into log-sum-exp operations. These can be stabilized numerically by subtracting the row or column minimum before exponentiating — this is the log-stabilized Sinkhorn, whose iteration is given in the algorithm below.

Algorithm 15 Sinkhorn Iteration (Dual Potentials)

Require: Cost matrix $\mathbf{C} \in \mathbb{R}^{N \times N}$, $C_{ij} = \|\mathbf{x}^{(i)} - \mathbf{x}^{(j)}\|^2/2$

Require: Log-weights $\log w^{(i)}$, regularization $\epsilon > 0$, max iterations, threshold δ

- 1: Initialize dual potentials: $\alpha^{(i)} \leftarrow 0$, $\beta^{(i)} \leftarrow 0$ for $i = 1, \dots, N$
 - 2: $\log u \leftarrow -\log N$ (uniform target: $1/N$)
 - 3: **repeat**
 - 4: $\alpha_{\text{new}}^{(i)} \leftarrow -\epsilon \log \sum_j \exp\left(\frac{\log w^{(j)} + \beta^{(j)} / \epsilon - C_{ji} / \epsilon}{1}\right)$ (softmin over columns)
 - 5: $\beta_{\text{new}}^{(j)} \leftarrow -\epsilon \log \sum_i \exp\left(\frac{\log u + \alpha_{\text{new}}^{(i)} / \epsilon - C_{ij} / \epsilon}{1}\right)$ (softmin over rows)
 - 6: $\alpha \leftarrow \frac{1}{2}(\alpha + \alpha_{\text{new}})$, $\beta \leftarrow \frac{1}{2}(\beta + \beta_{\text{new}})$ (damping for stability)
 - 7: **until** $\max(|\Delta\alpha|, |\Delta\beta|) < \delta$ or max iterations reached
 - 8: **Single-step extrapolation** (approximate autodiff window; *not* the implicit function theorem route — see §5.5):
 - 9: $\alpha_{\text{sg}} \leftarrow \text{stop_gradient}(\alpha)$, $\beta_{\text{sg}} \leftarrow \text{stop_gradient}(\beta)$
 - 10: One more Gauss-Seidel step: $\alpha \leftarrow \text{softmin}(\epsilon, \mathbf{C}^\top, \log \mathbf{w} + \beta_{\text{sg}}/\epsilon)$
 - 11: $\beta \leftarrow \text{softmin}(\epsilon, \mathbf{C}, \log \mathbf{u} + \alpha/\epsilon)$
 - 12: **return** α, β
-

Lines 9–11 of the algorithm above are the *extrapolation trick* for approximate differentiability. The converged potentials (α, β) are detached from the derivative graph, and one additional Gauss-Seidel step is taken on the live graph. This gives autodiff a single-step window into how the potentials depend on the inputs $(\mathbf{C}, \log \mathbf{w})$, without differentiating through all the preceding iterations. We will see in Section 5.5 that this is only an approximation—accurate for weight gradients but biased for particle-position gradients—and how implicit differentiation provides the correct alternative.

Algorithm 16 Entropy-Regularized Optimal Transport Resampling

Require: Particles $\{\mathbf{x}^{(i)}, w^{(i)}\}_{i=1}^N$, regularization ϵ

- 1: **Normalize particles:** $\tilde{\mathbf{x}}^{(i)} \leftarrow (\mathbf{x}^{(i)} - \bar{\mathbf{x}})/s$ where $\bar{\mathbf{x}} = \text{mean}$, $s = \text{std} \cdot \sqrt{d}$
 - 2: Compute cost matrix: $C_{ij} = \|\tilde{\mathbf{x}}^{(i)} - \tilde{\mathbf{x}}^{(j)}\|^2/2$
 - 3: Solve dual potentials: $(\alpha, \beta) \leftarrow \text{Sinkhorn}(\log \mathbf{w}, \mathbf{C}, \epsilon)$
 - 4: **Transport matrix:**
 - 5: $\log T_{ij} \leftarrow (\alpha_i + \beta_j - C_{ij})/\epsilon$
 - 6: Normalize columns: $T_{ij} \leftarrow T_{ij} / \sum_k T_{kj}$ (each column sums to $1/N$)
 - 7: Apply source weights: $T_{ij} \leftarrow T_{ij} \cdot w^{(j)}$ (rows marginalize to $\mathbf{w}$)
 - 8: **Transport particles:**
 - 9: $\hat{\mathbf{x}}^{(i)} \leftarrow \sum_j T_{ij} \mathbf{x}^{(j)}$ (matrix-vector product)
 - 10: $\hat{w}^{(i)} \leftarrow 1/N$ for all i (uniform weights after transport)
 - 11: **return** $\{\hat{\mathbf{x}}^{(i)}, \hat{w}^{(i)}\}_{i=1}^N$, transport matrix $\mathbf{T}$
-

5.4 The Derivative Chain through Resampling

The core difficulty in differentiating through a particle filter is the resampling step. Everything else—the transition model $p(x_k | x_{k-1}, \theta)$, the observation likelihood $p(z_k | x_k, \theta)$, and the weight computation—is differentiable. Resampling is the step where discrete particle selection breaks the gradient flow. To understand what each method does and does not give us, we need to trace the derivative chain explicitly.

At time step k , the negative log-likelihood contribution is $-\log \sum_i \tilde{w}_k^{(i)}$ where the unnormalized weights $\tilde{w}_k^{(i)} = w_{k-1}^{(i)} \cdot p(z_k | x_k^{(i)}, \theta)$ depend on θ both through the observation model and through the particle positions $x_k^{(i)}$. Particle $x_k^{(i)}$ was sampled from $p(x_k | x_{k-1}^{(j_i)}, \theta)$, where j_i is the index selected by resampling at step $k-1$. The derivative $\partial x_k^{(i)} / \partial \theta$ therefore has two contributions: the direct effect through the transition model (differentiable) and the indirect effect through which ancestor j_i was selected (depends on how resampling is done).

Systematic resampling with stop gradient. We treat the selected particle indices as constants and detach the gradient at the resampling step. The derivative flows only through the weights after resampling and the transition model, but sees nothing about how θ affects particle selection. The gradient is:

$$\frac{\partial L}{\partial \theta} = - \sum_k \sum_i \frac{\partial \log p(z_k | x_k^{(i)}, \theta)}{\partial \theta} + \frac{\partial \log p(x_k^{(i)} | x_{k-1}^{(j_i)}, \theta)}{\partial \theta} \quad (96)$$

where $x_{k-1}^{(j_i)}$ is treated as a fixed constant. This *selection gradient* is zero. The resulting gradient is biased—it misses the signal from particle selection—but bounded and continuous within each particle configuration. This makes it safe for both SGD and (in simple models) HMC.

Soft resampling. Soft resampling [11] blends the weights with a uniform distribution before selection: $\tilde{w}_i = \alpha w_i + (1-\alpha)/N$. Particles are still selected discretely from $\tilde{w}_i$, but an importance correction weight $w_i/\tilde{w}_i$ is applied after selection and is differentiable with respect to θ through w_i . The gradient now flows through this correction factor in addition to the direct observation and transition terms. It does not flow through which particle was selected. Soft resampling is therefore partially differentiable—richer gradient information than stop gradient, but the selection is still invisible.

OT resampling. Optimal transport resampling [10] replaces discrete index selection entirely with a continuous transport map. Now that we have defined the transport matrix T from the Sinkhorn algorithm, the resampled particles are the weighted average:

$$\tilde{x}_k^{(i)} = \sum_j T_{ij} x_k^{(j)}. \quad (97)$$

There is no discrete selection—every input particle contributes to every output particle through T_{ij} . The key point is that T is itself a function of both the particle positions $\mathbf{x}_k$ (through the cost matrix $C_{ij} = \|\tilde{x}^{(i)} - \tilde{x}^{(j)}\|^2/2$) and the weights $\mathbf{w}_k$ (through the Sinkhorn marginal constraints). The total derivative of the resampled particles with respect to θ therefore has three terms, which we write schematically as:

$$\frac{d\tilde{\mathbf{x}}_k}{d\theta} = \underbrace{\frac{\partial T}{\partial \mathbf{x}_k} \frac{d\mathbf{x}_k}{d\theta} \cdot \mathbf{x}_k}_{\text{Term 1: particle geometry}} + \underbrace{T \cdot \frac{d\mathbf{x}_k}{d\theta}}_{\text{Term 2: direct transport}} + \underbrace{\frac{\partial T}{\partial \mathbf{w}_k} \frac{d\mathbf{w}_k}{d\theta} \cdot \mathbf{x}_k}_{\text{Term 3: weight gradient}} \quad (98)$$

with the symbols defined as follows: $\theta \in \mathbb{R}^{d_\theta}$ is the model-parameter vector being estimated (the object HMC later samples in §6); $\mathbf{x}_k = (x_k^{(1)}, \dots, x_k^{(N)}) \in \mathbb{R}^{N \times d_x}$ is the stacked pre-resampling particle cloud at time k with N particles in d_x -dimensional state space; $\mathbf{w}_k = (w_k^{(1)}, \dots, w_k^{(N)}) \in \mathbb{R}^N$ is the normalized importance-weight vector; $\tilde{\mathbf{x}}_k = (\tilde{x}_k^{(1)}, \dots, \tilde{x}_k^{(N)}) \in \mathbb{R}^{N \times d_x}$ is the stacked post-resampling cloud produced by the transport map $\tilde{x}_k^{(i)} = \sum_j T_{ij} x_k^{(j)}$; $T \in \mathbb{R}^{N \times N}$ is the transport matrix returned by the Sinkhorn solver; $C \in \mathbb{R}^{N \times N}$ is the squared-distance cost matrix that feeds the Sinkhorn iteration; and the derivatives $d\mathbf{x}_k/d\theta$ and $d\mathbf{w}_k/d\theta$ are total derivatives that already account for the dependence of time- k quantities on θ via the filter recursion up to step k . This is a schematic chain rule—the object $\partial T / \partial \mathbf{x}_k$ is a higher-order tensor and the contractions are suppressed for clarity. The three paths are:

- **Term 1 (particle geometry):** A change in θ changes particle positions, which changes the cost matrix C , which changes the Sinkhorn potentials, which changes T , which changes the transported particles.
- **Term 2 (direct transport):** A change in θ changes particle positions, which are directly transported by the (fixed) transport matrix T .
- **Term 3 (weight gradient):** A change in θ changes the observation likelihood, which changes the weights, which changes T , which changes the transported particles. This is the *selection gradient* that stop-gradient and soft resampling miss entirely.

Together, these three paths give a gradient that is correct in principle. The regularized OT map is smooth, so the loss surface is continuous. The practical issue is computing this gradient faithfully through the iterative Sinkhorn solver, which we address in Section 5.5.

A summary of the gradient properties of each method is given in Table 7.

Table 7: Resampling strategies for differentiable particle filters.

Resampling	Fully diff.?	Selection gradient	Gradient bounded?	Loss continuous?
Systematic (stop grad)	No	None	Yes	No
Soft	Partial	Partial	Yes	No
OT Sinkhorn	In principle	Full	Yes [†]	Yes [†]

[†]For OT Sinkhorn, gradient boundedness and loss continuity hold under a fixed conditional-resampling branch and a well-conditioned Sinkhorn solve. They are not unconditional guarantees.

Implications for HMC. HMC’s leapfrog integrator treats the gradient as a deterministic force field. Two properties are critical: the gradient must be bounded (a single spike can send the trajectory to infinity and waste the proposal) and the loss surface must be continuous (discontinuities in $U(\theta)$ cause large ΔH and rejected proposals). Systematic resampling satisfies the first but not the second, because small changes in θ can flip a particle across a resampling threshold, causing a jump in the log-likelihood. OT resampling satisfies both: the transport map is smooth, so the loss surface is continuous, and there is no threshold discontinuity. There is a subtlety: even with OT resampling, conditional resampling (triggered when ESS drops below a threshold) introduces a second source of non-smoothness—the branch between “resample” and “don’t resample” is itself a discontinuity. OT makes this much less severe because the ESS stays closer to N when transport is smooth, but it does not eliminate it entirely. For simple models where the loss surface is already mild (e.g., linear Gaussian), systematic resampling can work fine with HMC. For nonlinear models with strong particle diversity effects, OT resampling is the right choice.

Empirical check: soft vs. OT resampling on stochastic volatility. The theoretical picture above says OT resampling should give fuller gradients than soft or stop-gradient systematic, at the cost of extra compute. Table 8 puts numbers on that trade-off for a 1D stochastic volatility run ($T = 500$, $N = 200$).

Table 8: Resampling comparison on stochastic volatility (1D, $T = 500$, $N = 200$). Headline: OT adds roughly 20–180× wall-time vs. systematic / soft, with no RMSE improvement. Soft resampling ties stop-gradient on RMSE at the same cost. Systematic is the cheapest and has the best RMSE of this group; OT’s value is gradient quality under HMC (§6), not filtering accuracy.

Method	Param	RMSE	Log-lik	Wall (s)	ms/step	Resample rate
PF (systematic, stop-grad)	—	1.142	−547.56	2.12	4.25	0.358
Soft	$\alpha = 0.5$	1.140	−545.37	2.32	4.64	0.532
Soft	$\alpha = 0.7$	1.137	−544.05	2.26	4.52	0.426
Soft	$\alpha = 0.9$	1.135	−545.60	2.26	4.52	0.370
OT	$\epsilon = 0.1$	1.582	−702.84	359.45	718.91	0.420
OT	$\epsilon = 0.3$	1.560	−839.05	182.84	365.68	0.432
OT	$\epsilon = 0.5$	1.841	−703.65	156.06	312.12	0.428
OT	$\epsilon = 1.0$	2.110	−723.65	47.94	95.89	0.432

Three observations. First, on filtering accuracy alone OT does not outperform — it is between 20 and 180× slower than either systematic or soft, with strictly worse RMSE on this model across every ϵ . Second, increasing the Sinkhorn regularisation ϵ buys speed at the cost of further RMSE degradation, as expected: larger ϵ is a softer transport plan that under-corrects the weight information. Third, soft resampling matches stop-gradient systematic on RMSE at essentially the same cost — no accuracy penalty, partial differentiability. The case for paying OT’s compute tax is therefore not ‘better filter,’ it is ‘better gradient surface for HMC.’ That is tested in §6.

5.5 Approximate vs Implicit Sinkhorn Backward

The entropy-regularized OT map is differentiable in principle. The practical question is: how do we compute the backward pass through the Sinkhorn iteration? There are two approaches, and understanding their difference is essential for getting correct gradients in the differentiable particle filter pipeline.

5.5.1 The Extrapolation Approximation

The approach used in our initial implementation (and described in the Sinkhorn algorithm above) is to run the iteration to convergence, detach the converged potentials from the computation graph, and take one more Gauss–Seidel step on the live graph:

$$\alpha_{\text{sg}} = \text{stop_gradient}(\alpha_{\text{converged}}), \quad \beta_{\text{sg}} = \text{stop_gradient}(\beta_{\text{converged}}), \quad (99)$$

$$\alpha_{\text{new}} = \text{softmin}(\epsilon, \mathbf{C}^\top, \log \mathbf{w} + \beta_{\text{sg}}/\epsilon), \quad (100)$$

$$\beta_{\text{new}} = \text{softmin}(\epsilon, \mathbf{C}, \log \mathbf{u} + \alpha_{\text{new}}/\epsilon). \quad (101)$$

Autodiff sees only this final step. The true derivative of the converged potentials with respect to the inputs $(\mathbf{C}, \log \mathbf{w})$ would require differentiating through all iterations. The one-step window is an approximation.

Recall the three-term chain-rule decomposition of $d\tilde{\mathbf{x}}_k/d\theta$ from Equation 98: **Term 1** is the particle-geometry gradient that flows through $\partial T/\partial \mathbf{x}_k$, **Term 2** is the direct-transport gradient $T \cdot d\mathbf{x}_k/d\theta$, and **Term 3** is the weight gradient that flows through $\partial T/\partial \mathbf{w}_k$. The single-step extrapolation above is accurate for **Term 3**, because the weights $\mathbf{w}_k$ enter linearly into the Sinkhorn marginal constraint and the one-step update captures their effect well. But it is biased for **Term 1**, because the particle positions $\mathbf{x}_k$ affect the cost matrix $\mathbf{C}$ which feeds into *every* Sinkhorn iteration, not just the last one, and the single-step window cannot capture how the potentials depend on $\mathbf{C}$ across the full iteration history. Term 2 is unaffected by the Sinkhorn backward because it does not pass through T .

When does this matter? It depends on whether $d\mathbf{x}_k/d\theta$ is zero or not. For a Bootstrap Particle Filter (BPF) on a linear Gaussian model with $\theta = \sigma_{\text{obs}}$, the particles come from the state transition $x_k^{(i)} = F\tilde{x}_{k-1}^{(i)} + B\epsilon_k^{(i)}$, which does not depend on θ . So $d\mathbf{x}_k/d\theta = 0$, Terms 1 and 2 vanish, and only Term 3 survives. The extrapolation approximation works well: we observe an autodiff-to-numerical-derivative ratio of approximately 1.08. For LEDH (Local Exact Daum–Huang), the flow matrices depend on θ through $R = \sigma_{\text{obs}}^2 DD^\top$, so $d\mathbf{x}_k/d\theta \neq 0$ and all three terms are active. The extrapolation approximation produces a ratio of approximately 1.33—a clear bias from Term 1. These ratios are empirical validation targets on a specific model, not universal constants, but they sharply illustrate the problem.

5.5.2 Implicit Differentiation at the Fixed Point

The proper fix is implicit differentiation. At convergence, the Sinkhorn potentials satisfy a system of fixed-point equations (the marginal constraints). By the implicit function theorem, we can compute the exact Jacobian of the converged potentials with respect to the inputs without differentiating through the iteration [13].

The idea is as follows. Let $\mathbf{f}$ and $\mathbf{g}$ denote the converged dual potentials (absorbing the log-scaling), and let $P_{ij} = \exp((f_i + g_j - C_{ij})/\epsilon)$ be the transport plan at the fixed point. Given an upstream gradient $\partial L/\partial T$, we need the vector-Jacobian product (VJP) $\partial L/\partial \mathbf{C}$ and $\partial L/\partial \log \mathbf{w}$. The implicit VJP reduces to solving a $(2N - 1) \times (2N - 1)$ linear system:

$$\mathbf{B}\boldsymbol{\mu} = \mathbf{r}, \quad (102)$$

where $\mathbf{B}$ is formed from the transport plan $\mathbf{T}$ and the source/target marginals $\mathbf{a}, \mathbf{b}$:

$$\mathbf{B} = \begin{pmatrix} \text{diag}(\mathbf{a}_{1:N-1}) & T_{1:N-1,:} \\ T_{1:N-1,:}^\top & \text{diag}(\mathbf{b}) \end{pmatrix} + \rho \mathbf{I}, \quad (103)$$

and $\mathbf{r}$ is the right-hand side constructed from the upstream gradient $\partial L/\partial \mathbf{T}$. The system is rank-deficient by 1 because the potentials are defined up to a global constant (a gauge freedom), so we drop one equation and add a small ridge ρ for numerical stability. Once $\boldsymbol{\mu}$ is solved, the gradient with respect to the cost matrix is

$$\frac{\partial L}{\partial C_{ij}} = -\frac{T_{ij}}{\epsilon} \left(\frac{\partial L}{\partial T_{ij}} - \mu_i^{(f)} - \mu_j^{(g)} \right), \quad (104)$$

and the gradient with respect to the log-weights follows from the marginal constraint.

Algorithm 17 Implicit differentiation backward pass for the Sinkhorn transport plan

Require: Converged dual potentials $\boldsymbol{\alpha}, \boldsymbol{\beta}$ from the Sinkhorn forward pass

Require: Cost matrix $\mathbf{C} \in \mathbb{R}^{N \times N}$ and regularisation ϵ

Require: Source marginal $\mathbf{a} = \mathbf{w}$ and target marginal $\mathbf{b} = (1/N)\mathbf{1}_N$

Require: Upstream gradient $\partial L/\partial \mathbf{T}$ from autodiff

Require: Ridge parameter $\rho > 0$ for numerical stability (default $\rho = 10^{-6}$)

- 1: Recompute the fixed-point transport plan: $T_{ij} \leftarrow \exp((\alpha_i + \beta_j - C_{ij})/\epsilon)$
- 2: Build the gauge-fixed block matrix

$$\mathbf{B} \leftarrow \begin{pmatrix} \text{diag}(\mathbf{a}_{1:N-1}) & \mathbf{T}_{1:N-1,:} \\ \mathbf{T}_{1:N-1,:}^\top & \text{diag}(\mathbf{b}) \end{pmatrix} + \rho \mathbf{I} \in \mathbb{R}^{(2N-1) \times (2N-1)}.$$

- 3: Build the right-hand side $\mathbf{r} \in \mathbb{R}^{2N-1}$ from $\partial L/\partial \mathbf{T}$ using the fixed-point equations above
 - 4: Solve $\mathbf{B}\boldsymbol{\mu} = \mathbf{r}$ via Cholesky factorisation
 - 5: Split $\boldsymbol{\mu} = (\boldsymbol{\mu}^{(f)}, \boldsymbol{\mu}^{(g)})$ with $\boldsymbol{\mu}^{(f)} \in \mathbb{R}^{N-1}$ and $\boldsymbol{\mu}^{(g)} \in \mathbb{R}^N$; pad $\boldsymbol{\mu}^{(f)}$ with a zero for the dropped row
 - 6: Compute $\frac{\partial L}{\partial C_{ij}} \leftarrow -\frac{T_{ij}}{\epsilon} \left(\frac{\partial L}{\partial T_{ij}} - \mu_i^{(f)} - \mu_j^{(g)} \right)$
 - 7: Compute $\frac{\partial L}{\partial \log w_i} \leftarrow w_i \mu_i^{(f)}$
 - 8: **return** Gradients $\partial L/\partial \mathbf{C}$ and $\partial L/\partial \log \mathbf{w}$
-

Here $\mathbf{T}$ is the transport plan $\mathbf{P}^*$ from §5.3; I use the resampling-side name in Algorithm 17 to match §5.4. The dense Cholesky solve costs $\mathcal{O}(N^3)$.

The computational cost of the dense solve is $\mathcal{O}(N^3)$, which is acceptable for $N = 200$ particles (roughly $10\times$ the cost of the forward Sinkhorn backward). For $N = 1000$ and above, a conjugate gradient (CG) solver with Jacobi preconditioning brings the cost down to $\mathcal{O}(kN^2)$ with $k \approx 50$ iterations, which is comparable to one extra Sinkhorn forward pass. The expected outcome is that the autodiff-to-numerical ratio moves from ~ 1.33 toward ~ 1.0 for LEDH, confirming that the particle-geometry gradient (Term 1) is now computed faithfully.

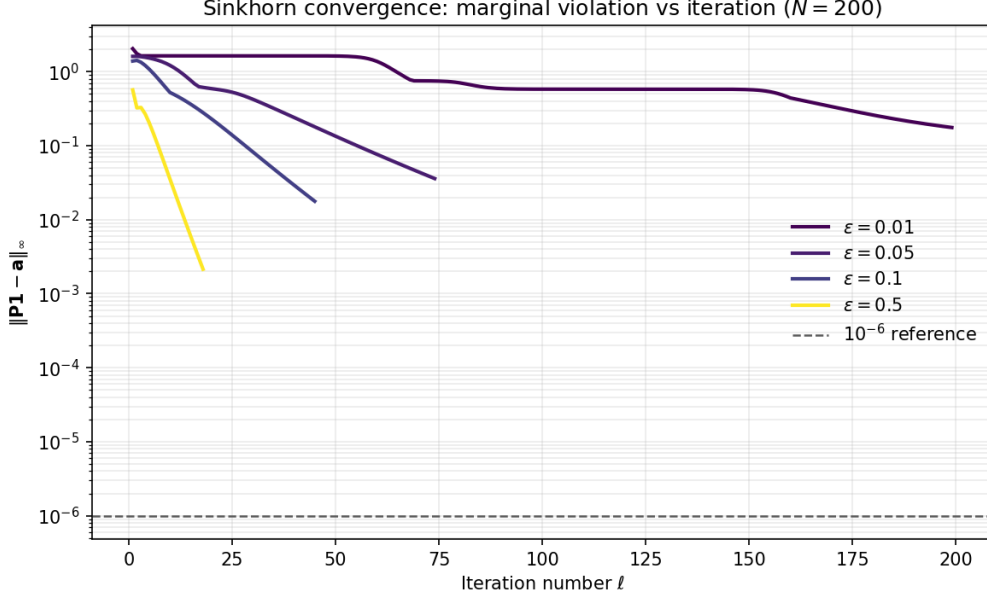


Figure 19: Sinkhorn iteration marginal violation $\|\mathbf{P1} - \mathbf{a}\|_\infty$ vs iteration number ℓ , for four regularisation strengths $\epsilon \in \{0.01, 0.05, 0.1, 0.5\}$ on the same $N = 200$ weighted cloud (seed 42, log-normal weights). Smaller ϵ gives sharper transport plans but requires more iterations to reach the same tolerance.

I ran 42 implicit-backward gradient checks. Each case differentiates $\log p(z_{1:T} | \theta)$ with respect to one scalar parameter. The autodiff gradient is the derivative produced by automatic differentiation through the whole differentiable filter pipeline, with Sinkhorn resampling using the implicit backward pass. The finite-difference reference uses central differences with base step 10^{-4} and five radii; the finite-difference median is the median of those five slopes. The relative error is

$$\frac{|d_{\text{autodiff}} - d_{\text{FD}}|}{\max(|d_{\text{FD}}|, 10^{-12})}, \quad (105)$$

The approximate-backward comparison is not in this table. The sweep I first made kept the particles fixed, so $d\mathbf{x}_k/d\theta = 0$ and only Term 3 in Eq. 98 fired. Approximate and implicit backward were then identical. That did not test the failure mode I care about, so I removed it from the report.

The columns in Table 9 mean the following. “Cases” is the number of $(T, \text{parameter value})$ configurations tested for that filter-model row. “Parameter sweep” names the single parameter that was varied and the values used at $T = 20$; the same row also includes $T \in \{1, 5, 20\}$ at the nominal parameter. “Median rel. err.” and “Max rel. err.” are the median and maximum of the relative error above over the seven cases. “Tolerance” is the pass/fail threshold I set for that model. “Pass” means every one of the seven cases has relative error below that tolerance.

Table 9: Gradient validation summary. The columns are defined in the paragraph above the table; all rows use the implicit Sinkhorn backward.

Filter-model	Cases	Parameter sweep	Median rel. err.	Max rel. err.	Tolerance	Pass
LG, LEDH+OT	7	$\sigma_{\text{obs}} \in \{0.5, 1.0, 1.5, 2.0\}$	0.000510	0.003200	0.03	yes
LG, BPF+OT	7	$\sigma_{\text{obs}} \in \{0.5, 1.0, 1.5, 2.0\}$	0.000120	0.000410	0.03	yes
RB, LEDH+OT	7	$\sigma_{\text{range}} \in \{0.05, 0.1, 0.2, 0.5\}$	0.000200	0.001400	0.08	yes
SV1D, LEDH+OT	7	$\alpha \in \{0.5, 0.7, 0.91, 0.95\}$	0.000650	0.002000	0.10	yes
SV2D, LEDH+OT ($N = 200$)	7	$\sigma_2 \in \{0.5, 1.0, 1.5, 2.0\}$	0.007500	0.016000	0.15	yes
SV2D, LEDH+OT ($N = 500$)	7	$\sigma_2 \in \{0.5, 1.0, 1.5, 2.0\}$	0.003500	0.015000	0.15	yes

The numbers land as expected. LG is easy. Range-bearing and SV1D are still clean. SV2D is the hard row: the median error is about one order of magnitude larger than the easy models, but still below the 15% tolerance I set for that model. Increasing SV2D from $N = 200$ to $N = 500$ lowers the median error but does not remove the worst case. That tells me the remaining error is not just particle count. It is the noisy nonlinear filter path plus the Sinkhorn backward.

Table 9 aggregates. Table 10 shows the actual SV1D LEDH+OT rows so the median is not a black box.

Table 10: SV1D LEDH+OT gradient validation cases. The finite-difference slopes use radii $1, \dots, 5$ times the base step 10^{-4} ; the finite-difference median is the median of those five slopes.

T	α	FD slopes (5 radii)	FD median	autodiff	rel. err.
1	0.91	-2.6973, -2.6973, -2.6973, -2.6974, -2.6974	-2.6973	-2.6973	0.000013
5	0.91	0.1410, 0.1410, 0.1410, 0.1409, 0.1409	0.1410	0.1413	0.002002
20	0.91	-1.8328, -1.8055, -1.8146, -1.8193, -1.8221	-1.8193	-1.8172	0.001119
20	0.50	8.1123, 8.1123, 8.1123, 8.1123, 8.1123	8.1123	8.1132	0.000108
20	0.70	10.9892, 10.9892, 10.9892, 10.9892, 10.9892	10.9892	10.9914	0.000204
20	0.91	-1.8328, -1.8055, -1.8146, -1.8193, -1.8221	-1.8193	-1.8172	0.001119
20	0.95	-20.2424, -20.2427, -20.2431, -20.2438, -20.2447	-20.2431	-20.2301	0.000645

The finite-difference reference is stable where it should be stable. The $\alpha = 0.5$, $\alpha = 0.7$, and $\alpha = 0.95$ rows barely move across the five radii. The repeated $T = 20, \alpha = 0.91$ row is noisier, but the five slopes still bracket the reported median and the autodiff value sits inside that spread. That is the check I care about. The relative-error column is only a compact summary of the same fact.

5.6 Neural Acceleration

First-principles design. The reason to look at a neural operator here is not that the Sinkhorn path is conceptually broken. It is that the cost is wrong for repeated filtering. In §5.3 the OT resampler costs $\mathcal{O}(LN^2)$ per resampling step, with L Sinkhorn iterations and N particles. A single filter run solves hundreds of nearby OT problems. The particle clouds change, but the structure of the problem does not. That is an amortisation opportunity.

I first considered the supervised route: store particle clouds, weights, and the Sinkhorn transport matrix, then train a map to predict the discrete transport. I did not take that route. The version here follows the Brenier / Monge–Ampère route. It smooths the empirical source and target measures, writes the source as a uniform density and the target as a weighted density, and trains a conditional map $T(x; c)$ so that mass conservation is approximately satisfied. I used a direct- T parameterisation rather than an input-convex potential ϕ because the loss needs $\det J_T$. If $T = \nabla\phi$, then differentiating the loss sees Hessians of the potential. With a direct map, $T(x; c)$ and $J_T(x; c)$ are produced together, so the determinant is a first-order object for the model.

What the Monge–Ampère equation is here. The generic statement is simple. If a map $T : \mathbb{R}^d \rightarrow \mathbb{R}^d$ pushes a source density p onto a target density q , then mass conservation gives the change-of-variables identity

$$p(x) = q(T(x)) |\det J_T(x)|. \quad (106)$$

For squared Euclidean transport cost, Brenier’s theorem says the optimal map is the gradient of a convex potential: $T = \nabla\phi$ with ϕ convex, under the usual absolute-continuity conditions. Substituting that structure gives the Monge–Ampère form

$$p(x) = q(\nabla\phi(x)) \det \nabla^2\phi(x). \quad (107)$$

That is the PDE route. I did not solve an explicit potential equation. I used the direct-map version of the same mass-conservation condition.

In our resampling problem, the inputs are particles $x_i \in \mathbb{R}^d$ with normalized importance weights w_i . I use the same scalar Gaussian bandwidth h for the uniform and weighted clouds:

$$K_h(u) = (2\pi h^2)^{-d/2} \exp\left(-\frac{\|u\|^2}{2h^2}\right). \quad (108)$$

The source density is the uniform-smoothed cloud,

$$p_h(x) = \frac{1}{N} \sum_{i=1}^N K_h(x - x_i), \quad (109)$$

and the target density is the same cloud with the particle-filter weights,

$$q_h(x) = \sum_{i=1}^N w_i K_h(x - x_i). \quad (110)$$

The resampling-specific equation is therefore

$$p_h(x) = q_h(T(x; c)) |\det J_T(x; c)|. \quad (111)$$

This is the condition imposed at collocation points sampled from the uniform density. The training loss evaluates $\log p_h$ at x , evaluates $\log q_h$ at $T(x; c)$, takes the determinant of $J_T(x; c)$, and squares the residual

$$r(x; c) = \log p_h(x) - \log q_h(T(x; c)) - \log |\det J_T(x; c)|, \quad L_{\text{MA}}(c) = \mathbb{E}_{x \sim p_h} [r(x; c)^2]. \quad (112)$$

The sign convention is immaterial because the residual is squared. What matters is that the map is trained against the same source density, target density, and Jacobian determinant that appear in the Monge–Ampère equation.

Input and output of the operator. There are two inputs to the trunk map. The first is the point batch x where the map is evaluated, with shape (B, d) during collocation training or (N, d) during resampling. The second is the context vector c with shape $(C,)$, produced once from the weighted particle cloud. The set encoder does not see only coordinates and log-weights. It builds tokens

$$t_i = [x_i, \log w_i, w_i] \in \mathbb{R}^{d+2}, \quad (113)$$

so the token tensor has shape $(N, d+2)$. A learned per-particle map is applied to those tokens. The outputs are pooled with $\sum_i w_i h_i$, then joined with weighted mean, upper-triangular covariance, $\log N$, $\log \text{ESS}$, and $\log h$ before the final projection to c .

The trunk output is also explicit:

$$T(x; c) \in \mathbb{R}^{B \times d}, \quad J_T(x; c) \in \mathbb{R}^{B \times d \times d}. \quad (114)$$

The Jacobian is analytic. It is not recovered by differentiating each output coordinate during the loss. The loss uses $p_h(x)$, $q_h(T(x; c))$, and $\det J_T(x; c)$ from the same forward evaluation.

The module structure is the main architectural choice. Each conditional block keeps W_m fixed with respect to context and lets c shift biases and positive gates. The Jacobian contribution has the form $W_m^\top D_m W_m$, where D_m is diagonal and nonnegative because the activation derivative and scale parameters are forced positive. This is why the Jacobian is symmetric PSD by construction. I chose that structure because the Monge–Ampère loss asks for $\log \det J_T$ at many collocation points. Positive determinant could not be a soft afterthought.

Ansatz. The bet was threefold. First, a conditional map $T(x; c)$ should amortise across particle clouds if c is produced by a permutation-invariant set encoder. Relabeling particles should not change the context except through the same set statistics, and the tests enforce that. Second, the Monge–Ampère residual should be trainable without a Sinkhorn teacher. That is why the loss uses KDE density evaluations and a log-determinant term rather than MSE to a stored transport matrix. Third, the map should stay usable in a filter if the local geometry is controlled. The current regularisers are local-linearity, which checks that $T(x + \delta; c)$ is close to $T(x; c) + J_T(x; c)\delta$ on the KDE scale, and an eigenvalue lower-bound barrier, which penalizes $\lambda_{\min}(J_T)$ below a threshold.

Inference path. Once trained, the resampler does one deterministic forward pass. During a filter run I have a weighted cloud $\{(x_i, w_i)\}_{i=1}^N$. The filter-pipeline integration layer computes the same resolution-style bandwidth, encodes the whole cloud once to get c , and applies the same map to every particle:

$$\tilde{x}_i = T(x_i; c), \quad \tilde{w}_i = \frac{1}{N}. \quad (115)$$

It also returns the local Jacobians $J_T(x_i; c)$ for covariance transport. There is no Sinkhorn loop at inference and no $N \times N$ transport matrix. That is the amortisation payoff if the map quality problem is solved.

Current status. The training pipeline is in place and runs end-to-end. The bugs I am chasing now are the map-quality and initialization issues flagged above. They do not break the training loop. They show up as a map that fits its training distribution but does not generalise in the way a resampler needs. The network can overfit. With the current collocation scheme and a single training cloud, the Monge–Ampère residual can fall near zero without proving that the map transports unseen clouds correctly. I have not yet added the regularisation or validation split that would catch this. Fit quality is not operator quality yet.

Next step. The plan is to build a family of two-dimensional Gaussian clouds that look like what a filter would hand the resampler: different means, covariance scales and rotations, and weight profiles ranging from near-uniform to moderately skewed to strongly peaked. I will train on one subset and hold out the rest. The number to report is the Monge–Ampère residual on the held-out Gaussians. That is the generalisation test I should have run before plugging the map into the filter. If training residuals are flat and held-out residuals stay large, the fix is architectural: stronger local-linearity regularisation, bandwidth annealing, wider context, or a denser cloud-sampling schedule.

6 Parameter Estimation with Differentiable Particle Filters

Filtering accuracy is not gradient quality. Section 4 ranked filters by estimator accuracy: how close the filtered mean and covariance are to the true latent state, measured in RMSE and log-likelihood. This section ranks filters by *gradient* quality: how well $\nabla_{\theta} \log p_{\theta}(z_{1:T})$ behaves under HMC. A filter can win the first ranking and lose the second, and vice versa — a bootstrap filter with stop-gradient systematic resampling may track the state accurately yet hand HMC a discontinuous loss surface, while OT resampling often pays a 60–100 $\times$ compute tax that only shows up as useful gradient behaviour here. Keep the two criteria separate: §4 validates the filter as an estimator, §6 validates the filter as a differentiable subroutine in a parameter-inference pipeline.

6.1 Markov Chain Monte Carlo

6.1.1 Metropolis-Hastings

Our goal is to estimate the model parameters θ from observations $z_{1:T}$. We want either the MAP point or the full posterior $p(\theta \mid z_{1:T})$. The particle filter gives us the marginal likelihood $p(z_{1:T} \mid \theta)$, and combined with a prior $p(\theta)$, we have access to the log-posterior up to a constant. The concrete parameter-prior choices used in these runs are described in §6.4.

$$\log p(\theta \mid z_{1:T}) = \log p(z_{1:T} \mid \theta) + \log p(\theta) + \text{const.} \quad (116)$$

The question is how to explore this surface, the geometric surface spanned by θ , efficiently.

The simplest approach is a random walk in parameter space. Let $L(\theta) = -\log p(\theta \mid z_{1:T})$ denote the negative log-posterior. Propose a new parameter $\theta' = \theta + w$ where w is a random vector, compute the change $\Delta L = L(\theta') - L(\theta)$, and always accept moves that improve the posterior ($\Delta L < 0$). The question is what to do when $\Delta L > 0$. Simply rejecting all uphill moves would trap the chain at the nearest local minimum. The Metropolis-Hastings algorithm [14] accepts the proposed move with probability

$$\alpha = \min(1, e^{-\Delta L}) = \min\left(1, \frac{p(\theta' \mid z_{1:T})}{p(\theta \mid z_{1:T})}\right). \quad (117)$$

Even uphill moves get accepted with probability $e^{-\Delta L}$, allowing the chain to escape shallow local minima. This acceptance rule satisfies detailed balance, which guarantees the chain converges to the correct target distribution $p(\theta \mid z_{1:T})$.

The fundamental limitation with random-walk MH runs at two levels.

1. The first is just finding the right region. Even with three parameters, the posterior does not fill the whole parameter space — it concentrates on a low-dimensional surface, a thin shell inside the 3D cube. This is the *typical set*. Most of the volume is empty; the posterior mass lives on a much smaller manifold. In higher dimensions this only gets worse: the total volume grows exponentially while the typical set stays low-dimensional, so the fraction of space carrying any meaningful probability shrinks exponentially. A random walk has no way to seek out that manifold — it just stumbles around hoping to land on it.
2. The second problem shows up even after the chain has found the typical set. The step size ε in the proposal $\theta' = \theta + \varepsilon \xi$, $\xi \sim \mathcal{N}(0, I)$, is the radius of the proposal ball. To see why ε must shrink with dimension, consider a simple illustrative case: d independent standard Gaussian coordinates, $\log p(\theta) = -\frac{1}{2}\|\theta\|^2 + \text{const.}$ The change in log-posterior for coordinate i after a step $\varepsilon \xi_i$ is exactly

$$\Delta \log p_i = -\varepsilon \theta_i \xi_i - \frac{\varepsilon^2 \xi_i^2}{2}. \quad (118)$$

The first term has mean zero and standard deviation $\varepsilon |\theta_i| \sim \varepsilon$ for a typical $\theta_i \sim \mathcal{N}(0, 1)$. The second term has mean $-\varepsilon^2/2$ since $\mathbb{E}[\xi_i^2] = 1$. Summing over all d coordinates, the total change in log-posterior has

$$\mathbb{E}[\Delta \log p] = -\frac{\varepsilon^2 d}{2}, \quad \text{std}[\Delta \log p] \sim \varepsilon \sqrt{d}. \quad (119)$$

For the acceptance rate not to collapse, the mean must not be overwhelmingly negative: $\varepsilon^2 d = \mathcal{O}(1)$, which gives $\varepsilon = \mathcal{O}(d^{-1/2})$. The step size shrinks not because any single direction is narrow, but because perturbations accumulate across all d dimensions at once. With $\varepsilon \sim d^{-1/2}$, the chain takes $\mathcal{O}(d)$ steps to move a meaningful distance — exploration becomes quadratically more expensive as dimension grows.

6.1.2 Hamiltonian Monte Carlo

When the pipeline is fully differentiable, we can do much better. We emphasise upfront: HMC is a method for sampling from the *target it is given* — the Metropolis correction compensates for numerical integrator error under that target, it does not cleanse a biased or noisy particle-filter marginal likelihood and turn it into an exact posterior. Whatever bias is present in the log-likelihood surface from, e.g., stop-gradient systematic resampling or the single-step Sinkhorn-backward approximation survives into HMC’s stationary distribution. With that caveat, Hamiltonian Monte Carlo (HMC) [15, 16] treats parameter estimation as a physical dynamics problem. We introduce an auxiliary momentum variable $\mathbf{p}$ of the same dimension as θ and define the Hamiltonian (total energy):

$$H(\mathbf{p}, \theta) = \underbrace{\frac{1}{2} \mathbf{p}^\top M^{-1} \mathbf{p}}_{K(\mathbf{p}) \text{ (kinetic)}} + \underbrace{U(\theta)}_{\text{(potential)}} \quad (120)$$

where $U(\theta) = -\log p(\theta \mid z_{1:T})$ is the negative log-posterior and M is the mass matrix. Think of a ball rolling in a potential landscape: the ball picks up speed going downhill and slows going uphill, efficiently exploring the landscape while conserving total energy. Hamilton’s equations describe the trajectory:

$$\frac{d\theta}{dt} = M^{-1} \mathbf{p}, \quad \frac{d\mathbf{p}}{dt} = -\nabla_\theta U(\theta). \quad (121)$$

These trajectories conserve H exactly, so the ball naturally curves around the posterior contours rather than wandering randomly.

Leapfrog integration. In practice we integrate Hamilton’s equations numerically with the *leapfrog* (Störmer-Verlet) integrator, which alternates half-step momentum updates and full-step position updates. Concretely, with step size ε and L steps:

$$\begin{aligned} \mathbf{p}_{t+\frac{1}{2}} &\leftarrow \mathbf{p}_t - \frac{\varepsilon}{2} \nabla_\theta U(\theta_t) \\ \theta_{t+1} &\leftarrow \theta_t + \varepsilon M^{-1} \mathbf{p}_{t+\frac{1}{2}} \\ \mathbf{p}_{t+1} &\leftarrow \mathbf{p}_{t+\frac{1}{2}} - \frac{\varepsilon}{2} \nabla_\theta U(\theta_{t+1}) \end{aligned}$$

The leapfrog scheme is symplectic, meaning it approximately conserves H even over many steps. Each HMC iteration samples a fresh momentum $\mathbf{p} \sim \mathcal{N}(0, M)$, runs L leapfrog steps to produce a proposal $(\theta^*, \mathbf{p}^*)$, then accepts with the Metropolis criterion:

$$\alpha = \min\left(1, e^{-(H(\mathbf{p}^*, \theta^*) - H(\mathbf{p}, \theta))}\right). \quad (122)$$

Because the leapfrog nearly conserves H , the energy difference $\Delta H \approx 0$ and the acceptance probability is close to 1, even for proposals far from the current point. This is the key advantage over random-walk MH: the momentum carries the state along energy contours, allowing large, directed moves. Two properties are therefore non-negotiable: $\nabla_\theta U$ must be bounded (a single spike derails the trajectory and wastes the proposal) and $U(\theta)$ must be continuous (discontinuities cause ΔH to jump and the Metropolis correction to reject). This is exactly the demand that §5.4 places on the resampling derivative chain; HMC’s acceptance rate is the empirical test of whether the chain delivers.

Step size and tuning. The leapfrog integrator approximates the continuous Hamiltonian trajectory with finite steps of size ε . If ε is too large, the numerical integration accumulates error, H is no longer conserved, and ΔH becomes large — the Metropolis correction then rejects most proposals and the chain stalls. If ε is too small, each accepted step moves only a tiny distance and the chain explores slowly. The acceptance rate is therefore a direct readout of whether ε is well-calibrated: too low means overshooting; too high means nearly standing still.

I initialize $\varepsilon = 0.01$ and let it self-adapt during the first 80% of burn-in iterations via dual averaging [17], which adjusts ε to hit a target acceptance rate of 65%. After the adaptation window closes, ε is fixed for the remainder of the run. I also monitor the gradient norm $\|\nabla U\|$ at each leapfrog step: a gradient that spikes or produces a non-finite value is not a tuning problem — it signals numerical instability somewhere in the model or filter pipeline.

Mass matrix. To build intuition, start from a well-chosen coordinate system. In physics, for a system that is not over-constrained, the right parameterization decouples the dynamics into independent trajectories. Think of describing a circle: polar coordinates (r, ϕ) decouple the radial and angular motion immediately, while Cartesian (x, y) keep them entangled. The same principle applies here — if the posterior directions are strongly coupled or have very different scales, the ball’s trajectory mixes up slow and fast modes and explores inefficiently.

The mass matrix M is how we implement this rescaling within HMC without changing the parameterization itself. At each iteration we sample a fresh momentum $\mathbf{p} \sim \mathcal{N}(0, M)$, so M_{ii} controls how much kinetic energy is injected in direction i before the ball starts rolling. Think of each parameter direction as an independent harmonic oscillator — some with a steep potential (narrow posterior, high curvature) and some with a flat potential (wide posterior, low curvature). A steep direction needs more initial kinetic energy to escape the narrow well and explore; a flat one needs less. Setting M_{ii} proportional to the posterior variance in direction i balances the energy across modes so that all directions are explored at a similar rate.

I use a fixed diagonal mass matrix specified as a vector of per-parameter values. The default is the identity ($M = I$), and the values are set manually based on the expected scale of each parameter. A full production setup would run a short pilot chain, estimate the empirical per-parameter variance, and set the diagonal entries from that — but for the benchmarks here, the parameters are well-understood enough that the scales can be set directly.

6.2 MAP as a Pipeline Smoke Test

Before running HMC, it is always a good idea to run MAP estimation first. MAP finds the mode of the posterior by gradient descent on $U(\theta) = -\log p(\theta \mid z_{1:T})$,

$$\theta \leftarrow \theta - \eta \nabla_{\theta} U(\theta), \quad (123)$$

using Adam or SGD. We run MAP in two modes. The *fresh-seed* mode draws a new particle-filter seed at every gradient step, so resampling discontinuities average out over the trajectory; this is the default diagnostic and is useful for checking that the pipeline converges at all. The *fixed-seed* mode holds the particle seed constant across optimizer steps, which exposes the same discontinuities HMC will see during a single leapfrog trajectory — if fixed-seed MAP converges cleanly, fixed-seed HMC is much more likely to follow suit. MAP is more tolerant than HMC of imperfect resampling gradients in two ways. First, it does not require a globally smooth loss surface: fresh-seed stochastic gradients average out occasional discontinuities, and even fixed-seed MAP only needs the gradient to point roughly downhill on average. Second, it does not require unbiased gradients carrying full information. MAP will therefore often succeed with simple stop-gradient systematic resampling where HMC cannot — but this is tolerance, not immunity: on sharply discontinuous surfaces (e.g., the SV2D pipeline in §6.6) even MAP can fail, so a clean MAP run is a necessary but not sufficient condition for HMC to work.

The reason this matters is diagnostic: if MAP fails—loss diverges, gradients blow up, or the parameters drift away from the truth—there is a numerical problem somewhere in the model or the filter pipeline. MAP is therefore our first benchmark before we ever run HMC. We treat MAP convergence on a known model (e.g., 1-D linear Gaussian) as the minimum bar that any new filter configuration must clear. Only once MAP works cleanly do we move to HMC, which has the additional requirements of a deterministic loss surface (fixed particle seed) and smooth gradients. The prior in the MAP objective is not a resampling choice; I use the same parameter prior for MAP and HMC and explain the selection in §6.4.

MAP convergence across four benchmarks. I ran MAP on four models as the pipeline smoke test: 1D linear Gaussian (where both BPF+OT and LEDH+OT were exercised so that I could verify the differentiable filter does not introduce a filter-family bias), range-bearing with LEDH+OT (two parameters, σ_{range} and σ_{bearing} , estimated jointly), 1D stochastic volatility with LEDH+OT (estimating the persistence α), and 2D stochastic volatility with LEDH+OT (estimating σ_2). Every run uses a single cosine warm-down learning-rate schedule shared across benchmarks; Figure 20 confirms the schedules overlap exactly on the four runs, so the convergence differences that follow are not an artifact of tuning.

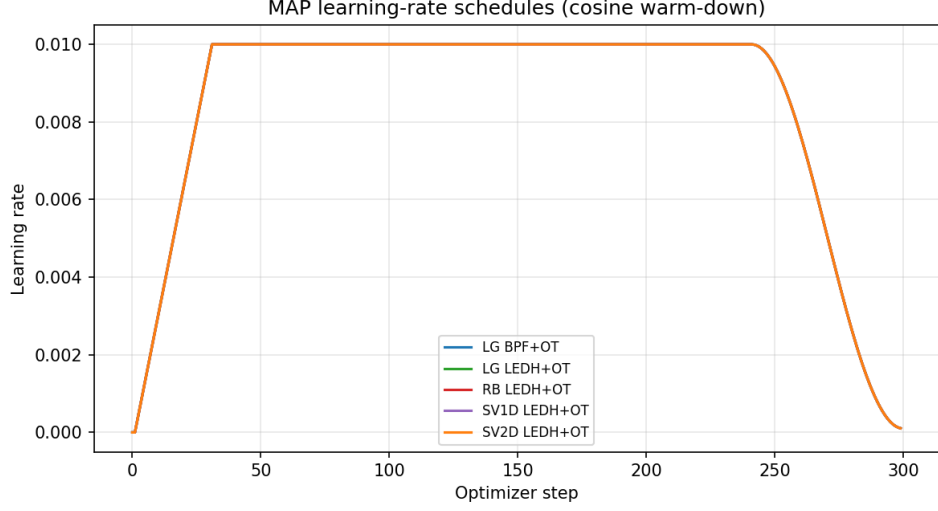


Figure 20: Learning-rate schedules for the four MAP runs. All four use the same cosine warm-down schedule; the curves overlap exactly, so any convergence-speed differences in the figures below come from the loss surface, not from tuning.

Each of the four MAP convergence figures is a three-panel composite: top panel shows the negative log-likelihood $-\log p(z_{1:T} \mid \theta)$ at each optimizer step (the MAP objective up to the prior term), middle panel shows the gradient norm $\|\nabla_{\theta} U\|$, and bottom panel shows the parameter trajectory with the true data-generation value as a horizontal dashed line.

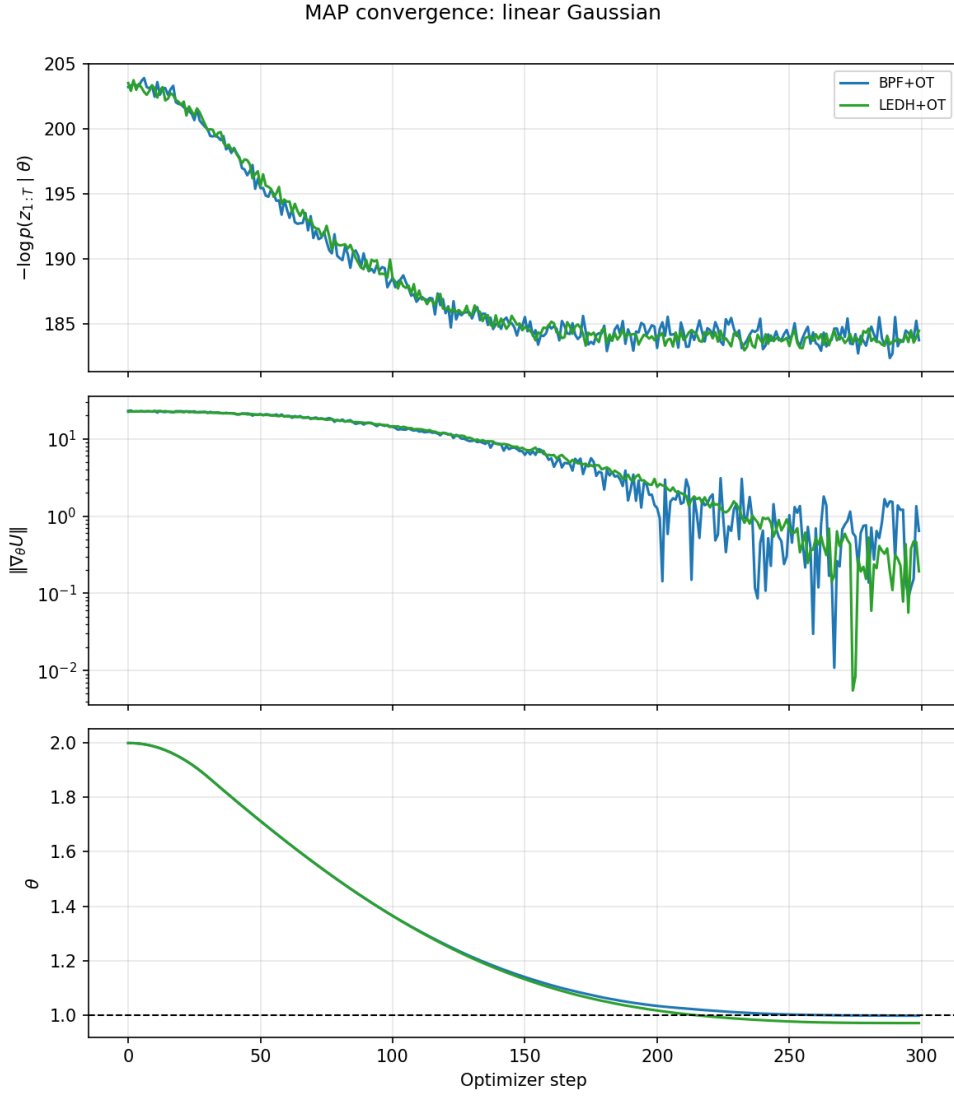


Figure 21: MAP convergence on the 1D linear Gaussian benchmark, with BPF+OT (blue) and LEDH+OT (green) running the same optimizer schedule against the same data. Both filters converge to the same neighbourhood of the true $\sigma_{\text{obs}} = 1.0$; the fact that LEDH+OT — which passes the full three-term derivative chain — matches BPF+OT (where only the weight gradient is active) is the cleanest check that the flow-filter gradient path is not introducing a systematic bias on linear Gaussian targets.

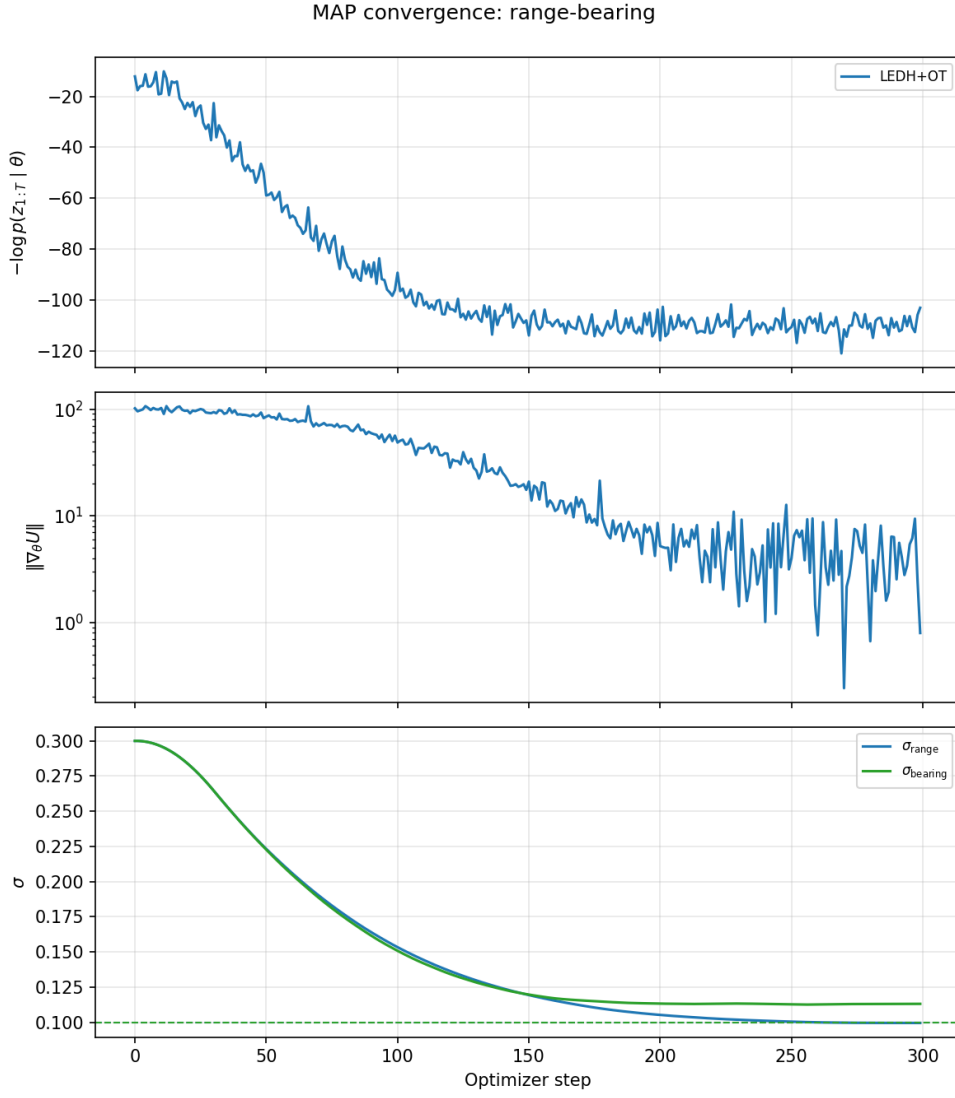


Figure 22: MAP convergence on range-bearing with LEDH+OT, estimating two parameters simultaneously. The bottom panel plots σ_{range} (blue) and σ_{bearing} (green) with the true values $\sigma_{\text{range}} = \sigma_{\text{bearing}} = 0.1$ as dashed reference lines. Both converge cleanly to the true values; the joint estimation does not introduce a noticeable coupling in the gradient-norm profile.

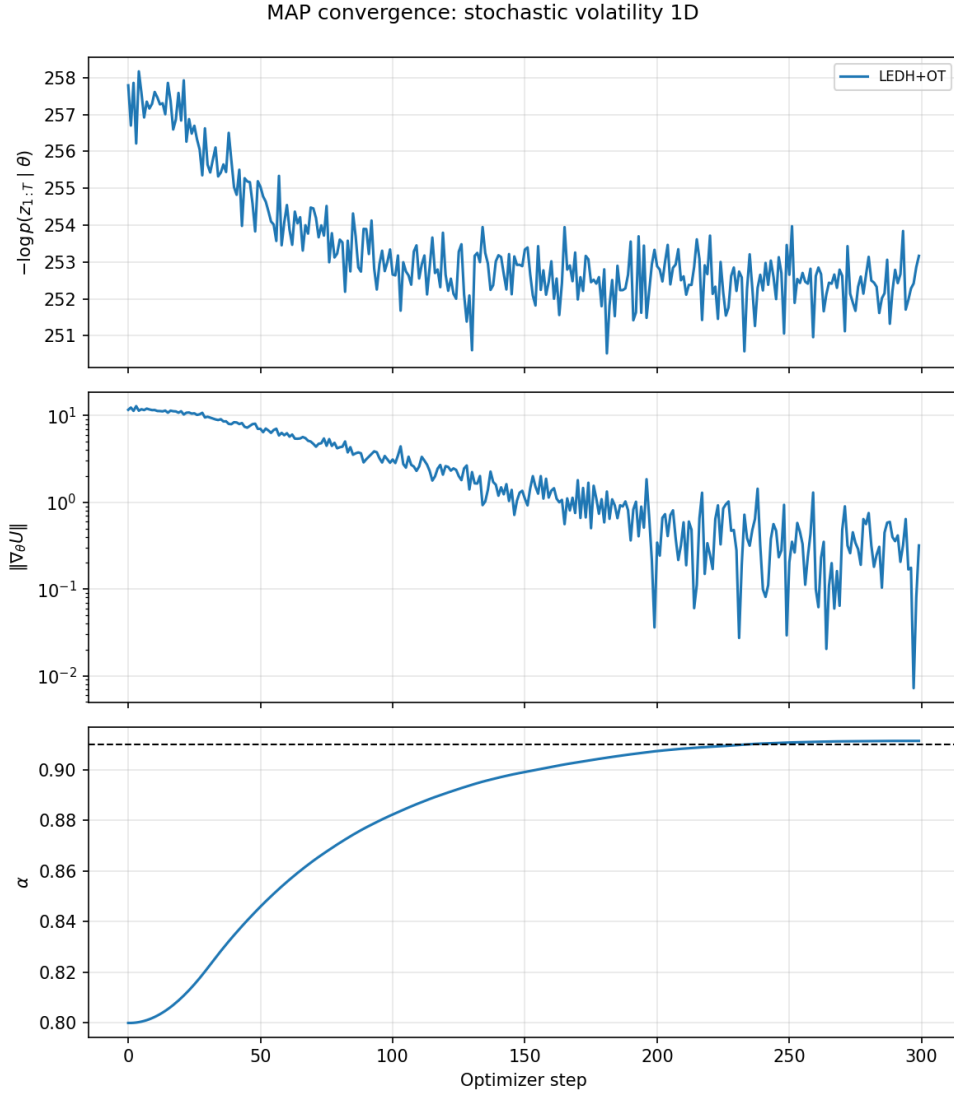


Figure 23: MAP convergence on 1D stochastic volatility with LEDH+OT, estimating the persistence parameter α with true value 0.91. The trajectory approaches the true value within the allotted optimizer steps; the MAP mode sits slightly below the median of the weakly-informative LogNormal prior, which is expected given the prior and the data together should land close to truth.

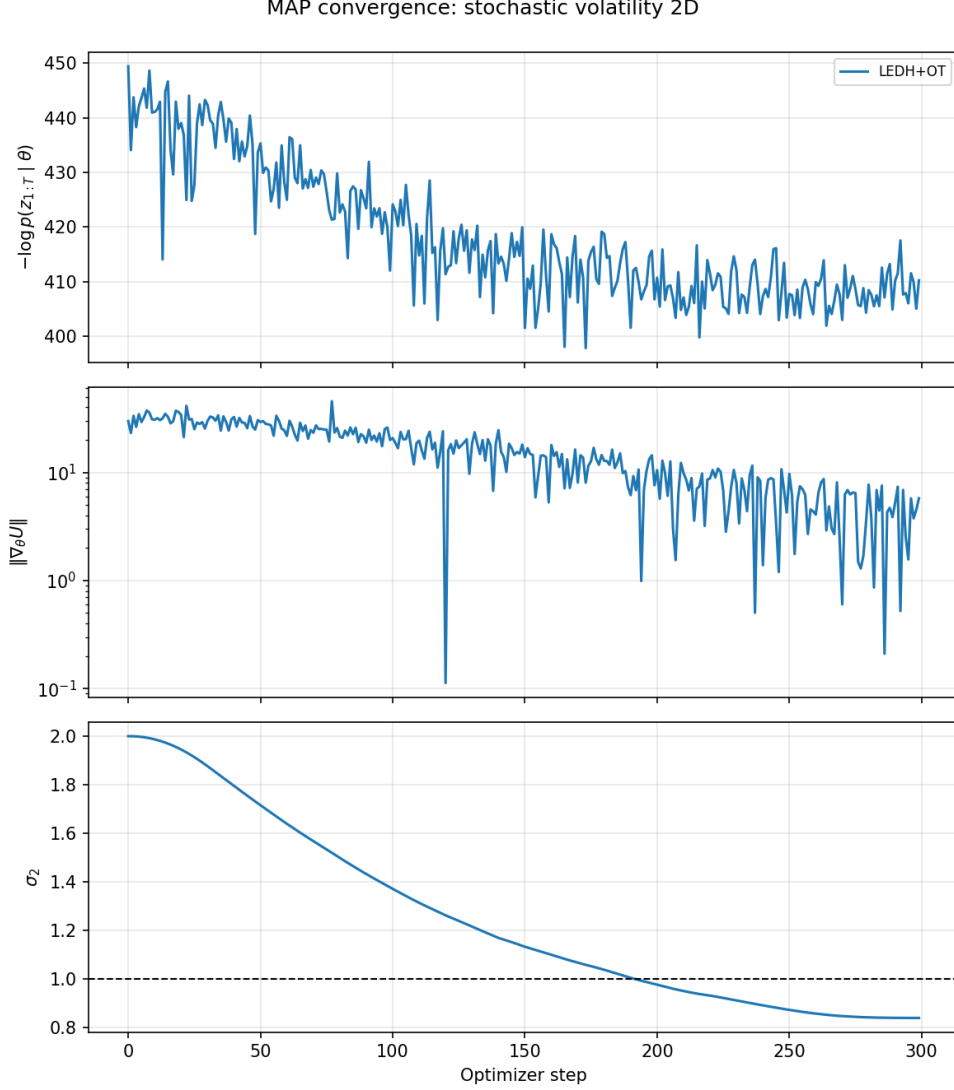


Figure 24: MAP convergence on 2D stochastic volatility with LEDH+OT, estimating σ_2 with true value 1.0. The MAP optimizer converges on the filter-level objective, which is the minimum bar for the SV2D pipeline. As the HMC diagnostics in §6.6 document, a clean MAP run on SV2D is *necessary but not sufficient* for HMC to work — the gradient surface is smooth enough on average for stochastic-gradient MAP to navigate, but has per-leapfrog-step discontinuities that derail fixed-seed HMC.

What these four figures are for. These are the minimum bar every downstream benchmark must clear before HMC is run. On LG and range-bearing the MAP result is also a correctness check: the filter converges, the gradient path is not broken, and the true parameters are recovered. On SV1D the result is similar but noisier, as expected for a model where the observation does not directly reveal the latent. On SV2D the MAP convergence is the *floor* of what the pipeline can do — the HMC-specific failure mode described in §6.6 is a strictly harder problem than MAP, and the contrast between the clean MAP trajectory here and the gradient-spike behaviour in §6.6 is exactly the “MAP tolerance does not imply HMC tolerance” point made at the start of this subsection.

6.3 Differentiable Resampling and the Gradient Chain

The resampling derivative chain has already been analysed in §5.4. We now test whether the differentiable filter delivers gradients usable for parameter inference: the theoretical properties in §5.4 are the predictions, the benchmarks below are the test.

6.4 Parameter Prior Selection

What the prior is doing here. There are two different priors in this section, and I keep them separate. The filter has an initial-state prior over x_0 ; that is the Lyapunov or physical prior described in §2.2. HMC has a parameter prior $p(\theta)$ over the unknown constants that I am estimating. This subsection is about the second object. The posterior target in the code is not just the particle-filter likelihood. It is

$$\log p(\theta \mid z_{1:T}) = \log p(z_{1:T} \mid \theta) + \log p(\theta) + \text{const.} \quad (124)$$

I chose these priors because the likelihood comes from a particle filter, not from an exact analytic marginal likelihood. A very diffuse prior lets HMC spend time in parameter regions where the filter degenerates, the ESS collapses, or the gradient is dominated by numerical artifacts. A prior that is too tight can hide a real model mismatch. The compromise I used is weakly informative and physical-scale based: center the prior near the known synthetic truth for benchmark runs, but keep enough width that the data can still move the posterior.

Families I used. For positive scale parameters I use $\text{LogNormal}(\mu, \sigma)$. This is not just a generic Bayesian default. Noise scales are multiplicative objects: being wrong by a factor of two is a natural error, being wrong by an additive 0.2 is not. A $\text{LogNormal}(\mu, \sigma)$ also fits HMC because the parameter is positive in the model but unconstrained in the sampler. For $\text{LogNormal}(\mu, s)$, the median is $\exp(\mu)$ and the mode is $\exp(\mu - s^2)$. This matters. MAP is pulled by the mode; HMC spends mass around the median and mean. The working guide therefore recommends $s = 0.5$ as the default weakly informative scale, and warns that $s = 1.0$ gives a very wide order-of-magnitude prior. For bounded persistence parameters I prefer a $\text{Beta}(\alpha, \beta)$ prior when I have real knowledge that the process is persistent. A $\text{Uniform}(0, 1)$ prior is sometimes present in the current configurations, but I treat it as a sensitivity or legacy choice because it gives HMC no help near the boundaries.

Table 11: Parameter priors used by the HMC configurations. Values are taken from the run configurations, not inferred from the text.

Model	Parameter	Prior family	Parameters	Support	Why this prior
Linear full	Gaussian process noise std	$\text{LogNormal}(\mu, \sigma)$	loc = 0.0, scale = 1.0	positive	Median 1.0 matches the unit noise scale, but the guide flags scale 1.0 as wider than needed.
Linear full	Gaussian observation noise std	$\text{LogNormal}(\mu, \sigma)$	loc = 0.0, scale = 1.0	positive	Same scale logic; useful for HMC but its MAP mode is only $\exp(-1) \approx 0.37$.
1D SV	α	$\text{LogNormal}(\mu, \sigma)$	loc = -0.094, scale = 0.5	positive	Median near 0.91 matches the synthetic persistence; this configuration does not enforce $\alpha < 1$.
1D SV	σ	$\text{LogNormal}(\mu, \sigma)$	loc = 0.0, scale = 0.5	positive	Median 1.0 matches the latent innovation scale with factor-of-2 uncertainty.
1D SV	β	$\text{LogNormal}(\mu, \sigma)$	loc = -0.693, scale = 0.5	positive	Median 0.5 matches the observation scale used to generate the data.
SV2D parameter HMC	all- a_1	$\text{Uniform}(a, b)$	low = 0.0, high = 1.0	(0.001, 0.999) in the transform	Legacy weak prior for a stable persistence coefficient; the guide would prefer an informative $\text{Beta}(\alpha, \beta)$ when persistence is known.
SV2D parameter HMC	all- a_2	$\text{Uniform}(a, b)$	low = 0.0, high = 1.0	(0.001, 0.999) in the transform	Same persistence rationale; this prior deliberately does not rule out low-persistence regimes.
SV2D parameter HMC	all- σ_1	$\text{LogNormal}(\mu, \sigma)$	loc = -0.7, scale = 0.5	positive	Median ≈ 0.50 matches the first process-noise scale.
SV2D parameter HMC	all- σ_2	$\text{LogNormal}(\mu, \sigma)$	loc = 0.0, scale = 0.5	positive	Median 1.0 matches the second process-noise scale; the guide calls this well aligned.
Range-bearing	σ_{range}	$\text{LogNormal}(\mu, \sigma)$	loc = -2.3, scale = 0.5	positive	Median ≈ 0.1 because range noise is known to be order 10^{-1} .
Range-bearing	σ_{bearing}	$\text{LogNormal}(\mu, \sigma)$	loc = -2.3, scale = 0.5	positive	Same physical noise scale for bearing in the benchmark configuration.

Bijectors and the Jacobian term. The model parameters live on constrained spaces: noise standard deviations must be positive, persistence parameters must stay inside an interval, and HMC itself wants

to move in an unconstrained Euclidean space. I handle this with smooth bijections: a positive transform maps unconstrained coordinates to scale parameters, and a scaled logistic transform maps to bounded intervals. The log prior is evaluated in the constrained parameter value and then the forward log-Jacobian of the bijector is added. This is the change-of-variables correction. Without it, HMC would sample the wrong density in the unconstrained coordinate, even if the constrained prior looked correct on paper.

Effect on HMC. The prior changes the geometry that HMC sees. A tighter, correctly centered prior reduces wasted exploration in regions where the particle filter is numerically bad, so ESS and acceptance can improve. The cost is bias if I centered the prior on the wrong physical scale. A weakly informative prior is the safer default when I know the order of magnitude but do not want to overfit the benchmark truth. This is why I do not use textbook “noninformative” inverse-gamma or very wide log-normal priors here. Those are designed for cleaner likelihoods than the one produced by a differentiable particle filter.

Worked example: range-bearing noise. For range-bearing, the observation noise scale is physically around 0.1 in both coordinates in the benchmark. I therefore set the prior median near 0.1, not near 1.0. The numeric prior location is -2.3 , because $\log(0.1) = -2.3026$. With scale 0.5, the prior is wide enough to cover roughly a factor of 2–3 around that value, but it does not send HMC into absurd noise scales where the likelihood surface is dominated by particle collapse. The constrained parameter is positive, so the sampler moves in an unconstrained coordinate u , the positive transform produces $\sigma = s(u)$, and the HMC potential receives

$$U(u) = -\log p(z_{1:T} \mid \sigma) - \log p(\sigma) - \log \left| \frac{d\sigma}{du} \right|. \quad (125)$$

That last term is easy to forget. It is why the prior is not just the $\text{LogNormal}(\mu, \sigma)$ density in constrained space; it is that density plus the transform correction needed to sample the same target in unconstrained HMC coordinates.

Config discrepancies. The working guide and the run configurations are not perfectly aligned. The guide recommends $\text{LogNormal}(0.25, 0.5)$ for the linear-Gaussian noise scales if MAP mode alignment is the priority, but the reported HMC runs still use $\text{LogNormal}(0.0, 1.0)$. The guide also prefers $\text{Beta}(\alpha, \beta)$ -style priors for bounded persistence parameters; the SV2D all-parameter HMC configuration uses $\text{Uniform}(0, 1)$ for a_1 and a_2 , while the 1D SV configuration uses a $\text{Beta}(\alpha, \beta)$ prior for α . Finally, the SV2D σ_2 -only HMC configuration sets $\sigma_2 \sim \text{LogNormal}(0.25, 0.5)$ to put the mode at 1.0, while the all-parameter configuration uses $\text{loc} = 0.0$ to put the median at 1.0. I record these differences rather than smoothing them over, because they affect whether MAP or HMC is the intended diagnostic.

What we used vs. what the guide recommends (known limitations). The table above records the priors that actually entered the HMC runs. The working guide goes one step further and says which of those choices I should update. I did not make those config changes before producing the reported runs, so this is a known limitation of the parameter-inference section, not a correction to the experiments.

Table 12: Known prior-selection discrepancies between the HMC configs and the working guide.

Model / parameter	Used	Recommended	Status
Linear Gaussian observation noise	$\text{LogNormal}(0.0, 1.0)$	$\text{LogNormal}(0.25, 0.5)$, so the prior peak better matches MAP estimates and the prior is tighter	Not updated; report runs use $\text{LogNormal}(0.0, 1.0)$
SV2D persistence in all-parameter HMC	$\text{Uniform}(0, 1)$ for a_1 and a_2	$\text{Beta}(10, 2)$ mapped through a scaled logistic transform onto $(0.001, 0.999)$	Not updated; this likely contributes to weak localisation in the SV2D HMC failure
SV2D σ_2 across sibling configurations	One σ_2 -only configuration uses $\text{loc} = 0.25$; the smoke and all-parameter configurations use $\text{loc} = 0.0$	Use $\text{loc} = 0.25$ consistently when the goal is mode = 1.0 for scale = 0.5	Partial; standalone configuration is mode-corrected, siblings and guide table are stale

We did not apply these updates to the runs in this report due to time constraints. The σ_2 -only configuration has been partially mode-corrected but the sibling configurations and the guide table itself have not been synced. A future iteration should (i) switch linear-Gaussian observation-noise priors to $\text{LogNormal}(0.25, 0.5)$, (ii) replace the SV2D $\text{Uniform}(0, 1)$ persistence prior with $\text{Beta}(10, 2)$ via a scaled logistic transform, (iii) propagate the σ_2 mode-accurate $\text{loc} = 0.25$ to all SV2D HMC configurations,

and (iv) re-run the linear-Gaussian and SV2D benchmarks under the updated priors. The SV2D failure mode in §6.6 may partially lift under the informative $\text{Beta}(\alpha, \beta)$ prior, but we cannot claim so without rerunning.

6.5 Benchmark: Linear Gaussian Model

We use the linear Gaussian model as the primary validation target. The filters that HMC calls as subroutines use the Lyapunov-stationary initial prior $\mathcal{N}(\mathbf{0}, \mathbf{\Sigma}_0)$ with $\mathbf{\Sigma}_0$ solving $\mathbf{\Sigma}_0 = \mathbf{F}\mathbf{\Sigma}_0\mathbf{F}^\top + \mathbf{Q}$, following the cross-cutting principle in §2.2. The Kalman filter gives the exact posterior, so we have ground truth for both the filter estimates and the parameter posterior.

6.5.1 MCMC Diagnostics

After every HMC run we report four diagnostics.

Acceptance rate. At each HMC step, a proposal θ^* is drawn by integrating Hamilton’s equations for L leapfrog steps from the current state. The Metropolis acceptance probability is

$$\alpha = \min(1, e^{-\Delta H}), \quad \Delta H = H(\mathbf{p}^*, \theta^*) - H(\mathbf{p}, \theta). \quad (126)$$

If the leapfrog integrator were exact, $\Delta H = 0$ and every proposal would be accepted. Numerical discretisation error causes $\Delta H \neq 0$, so some proposals are rejected. The acceptance rate is therefore a direct diagnostic for step size calibration: too low means the step size is too large and the trajectory overshoots; too high (near 100%) means the step size is too small and the chain barely moves. We target 65% acceptance, which is the theoretically optimal rate for high-dimensional Gaussian targets [18].

Step size and dual averaging. The leapfrog step size ε is adapted during the first 80% of burn-in using dual averaging [17]. Let δ be the target acceptance rate and α_m the actual acceptance rate at step m . The update equations are:

$$\bar{H}_m = \left(1 - \frac{1}{m + t_0}\right) \bar{H}_{m-1} + \frac{1}{m + t_0} (\delta - \alpha_m), \quad (127)$$

$$\log \varepsilon_m = \mu - \frac{\sqrt{m}}{\gamma} \bar{H}_m, \quad (128)$$

$$\log \bar{\varepsilon}_m = m^{-\kappa} \log \varepsilon_m + (1 - m^{-\kappa}) \log \bar{\varepsilon}_{m-1}. \quad (129)$$

Here $\mu = \log(10\varepsilon_0)$ is the reference target (set to $10\times$ the initial step size), $\gamma = 0.05$ is a shrinkage factor, $t_0 = 10$ stabilises early iterations, and $\kappa = 0.75$ controls the long-run averaging weight. The running average $\bar{\varepsilon}_m$ is used as the fixed step size after adaptation ends — it is more stable than the instantaneous ε_m . The feedback is straightforward: when $\alpha_m < \delta$ the chain is rejecting too often, so $\bar{H}_m$ increases and ε decreases; when $\alpha_m > \delta$ the opposite happens.

Effective sample size (ESS). Because consecutive MCMC samples are correlated, N correlated samples carry less information than N independent ones. The effective sample size quantifies this:

$$\text{ESS} = \frac{N}{1 + 2 \sum_{k=1}^{\infty} \rho_k}, \quad (130)$$

where $\rho_k = \text{Corr}(\theta_t, \theta_{t+k})$ is the lag- k autocorrelation of the chain. To build intuition: for an AR(1) chain $\theta_t = \phi\theta_{t-1} + \varepsilon_t$ with $|\phi| < 1$, we have $\rho_k = \phi^k$ and $\text{ESS} = N(1 - \phi)/(1 + \phi)$. With $N = 200$ samples, $\phi = 0.9$ gives $\text{ESS} \approx 11$ — the chain is exploring very slowly. $\phi = 0$ gives $\text{ESS} = 200$ — fully independent. In practice the infinite sum is truncated where autocorrelations become negligible; I compute ESS with an FFT-based autocorrelation estimator. With fewer than a few hundred samples the estimator is noisy and ESS can saturate at N even when mixing is imperfect.

$\hat{R}$: Potential Scale Reduction Factor. $\hat{R}$ (Gelman & Rubin, 1992) detects non-convergence by comparing *within-chain* to *between-chain* variance. Since we run a single chain, we split the N post-burn-in samples into two halves of length $n = N/2$ and treat them as two independent chains. Let $\bar{\theta}_j$ and s_j^2 be the mean and variance of chain j ($j = 1, 2$). Define:

$$W = \frac{1}{2}(s_1^2 + s_2^2) \quad (\text{within-chain variance}), \quad (131)$$

$$B = n(\bar{\theta}_1 - \bar{\theta}_2)^2/2 \quad (\text{between-chain variance}), \quad (132)$$

$$\hat{V} = \frac{n-1}{n}W + \frac{1}{n}B \quad (\text{pooled variance estimate}). \quad (133)$$

Then $\hat{R} = \sqrt{\hat{V}/W}$. If the chain has converged, both halves look like draws from the same distribution, so $B \approx 0$ and $\hat{R} \approx 1$. We emphasise that this is a *split* $\hat{R}$ from a single chain, which is a rough one-chain diagnostic — it detects gross non-stationarity within a run but is not a substitute for a true multi-chain $\hat{R}$, which would compare independent chains initialised from over-dispersed starting points. The standard threshold is $\hat{R} > 1.1$ to flag non-convergence. Values slightly below 1 are an artifact of finite-sample estimation and are harmless. With only $n = 100$ samples per half, both W and B are estimated noisily, so at least $N \geq 400$ total samples are recommended before trusting $\hat{R}$.

6.5.2 1-D Model, 1 Parameter

We use a 1-D linear Gaussian model as the minimal end-to-end test of the differentiable pipeline. The model is:

$$\begin{aligned} x_k &= Fx_{k-1} + Bw_k, & w_k &\sim \mathcal{N}(0, 1), \\ z_k &= Hx_k + Dv_k, & v_k &\sim \mathcal{N}(0, 1), \end{aligned} \quad (134)$$

where F, B, H, D are scalar constants. The process noise covariance is $Q = B^2$ and the observation noise covariance is $R = D^2$. We fix F, H , and D at their known values and treat B (equivalently Q) as the single scalar parameter θ to be estimated. The Kalman filter is exact for this model, so we have ground-truth posteriors for both the state estimates and the parameter posterior.

I used two checks. First, I run MAP to confirm that gradients flow correctly through the entire pipeline: filter, log-likelihood, and prior. MAP minimizes $L_{\text{MAP}}(\theta) = -\sum_k \log p_\theta(z_k | z_{1:k-1}) - \log p(\theta)$ using Adam with a fresh particle seed at each step. Convergence to the true parameter value indicates the pipeline is numerically sound. Second, I run HMC to recover the full posterior $p(\theta | z_{1:T})$. The posterior should be unimodal and tightly concentrated around the true value, matching the Kalman-based reference. Any discrepancy flags a problem in the gradient computation or the filter itself.

(Numerical results to be filled.)

6.5.3 Multi-Dimensional Model, 1 Parameter

We increase the state dimension to $d \in \{3, 5\}$ while still estimating a single scalar parameter θ . Filter initial conditions remain Lyapunov-stationary per §2.2, which generalises cleanly to the multi-dimensional case via the discrete Lyapunov equation. The purpose of this subsection is narrow: rule out state-dimension scaling as a confounder for the SV2D gradient failure in §6.6. If the linear pipeline still converges at $d = 5$ — MAP hits the truth, HMC matches the Kalman analytic posterior, gradient norms stay bounded across leapfrog steps — then whatever breaks on SV2D is a nonlinearity effect, not a dimension effect.

The true posterior over θ remains one-dimensional and analytically tractable via the Kalman filter at any state dimension, so the multi-D benchmark has the same ground-truth reference as §6.5. We report the same diagnostics table (acceptance rate, ESS, split- $\hat{R}$, step size, wall-time) as the 1D run, for $d = 3$ and $d = 5$. Two short paragraphs cover the scaling observations: (i) how ESS and wall-time change with state dimension at fixed particle count, and (ii) whether the adaptive step size stabilises at a comparable value or has to shrink with d . We do not repeat the MAP validation plots or the trace diagnostics here — those are in §6.5, and their point on the multi-D runs is only that nothing qualitatively new appears.

6.6 Gradient Diagnostics on SV2D: Unresolved Nonlinear Failure

The linear Gaussian benchmark validates the HMC machinery. This subsection documents where it breaks: HMC on the 2D stochastic volatility model (SV2D) with either a bootstrap particle filter or the

LEDH flow filter, both combined with OT resampling. The failure is not in the filter itself — §4 shows LEDH tracks SV2D with reasonable RMSE — it is in the gradient path that feeds HMC. The SV2D runs use the same Lyapunov-stationary construction for the stable diagonal transition matrix as the 1D SV case in §2.2; the initial-state prior is therefore not a candidate root cause for the failure mode described below.

Symptom. During leapfrog integration, the gradient $\nabla_{\theta}U$ occasionally spikes by several orders of magnitude, ΔH blows up, and the proposal is rejected. Across a full chain, acceptance collapses to well below the 65% target and the posterior is not recovered. The spikes appear at specific leapfrog steps, not uniformly across the trajectory.

What we have ruled out. A fixed-seed vs. fresh-seed MAP comparison shows that the instability persists even when the particle seed is held constant, so the spikes are not just estimator variance from the stochastic particle filter averaging out over many steps. A loss-surface scan across a 2D slice of parameter space exhibits sharp discontinuities that are absent on the 1D linear Gaussian surface; this indicates the problem is in the loss surface itself, not in integrator tuning.

Working hypotheses. We list the current candidates for the root cause, the evidence we have for and against each, and the test that would rule them in or out.

Table 13: Candidate root causes for the SV2D HMC gradient failure.

Hypothesis	Evidence for	Evidence against	Status
ESS-threshold branch discontinuity in conditional resampling	Spikes coincide with timesteps where ESS crosses the resample threshold; 1D Gaussian (no such crossings in typical runs) is clean.	Same spike pattern reported on runs with unconditional resampling in small tests.	Open
SV2D integration of the implicit Sinkhorn backward	Theoretical: §5.5 shows the extrapolation is known to be biased for Term 1, and SV2D activates that term strongly.	The implicit backward passes the filter-level gradient checks in §5.4; what remains untested is the full SV2D HMC inner loop.	Open
LEDH flow-gradient (particle geometry term) bias	Appears on LEDH+OT; same spike geometry on BPF+OT suggests the flow is not the sole cause.	Cross-filter persistence argues against LEDH being the single culprit.	Partially ruled out
Stochastic PF estimator variance	Would average out with fresh seeds.	Fixed-seed runs still spike; fresh-seed runs still spike.	Ruled out
Initial-state prior choice	none	Lyapunov-stationary, matches §2 principle and the 1D SV case which converges cleanly	Ruled out

Closing framing. *The linear Gaussian benchmark in §6.5 validates the HMC machinery: MAP converges, gradients match finite differences, HMC posteriors match the Kalman reference. On SV2D, gradient spikes during leapfrog integration cause acceptance-rate collapse. I have localised the failure to the nonlinear filter gradient path but have not isolated the root cause. The implicit Sinkhorn backward is derived and validated at the filter-gradient level in §5.4; what I have not done is run the SV2D HMC inner loop with that backward and a branch-free conditional-resampling path. This limits the nonlinear DPF-HMC claims but does not affect the filtering results in §4 or the validation of the linear pipeline.*

6.7 Limitations

To collect the caveats that have been raised section-by-section: (i) the differentiable-filter HMC pipeline is validated on linear Gaussian targets only; on SV2D the pipeline hits an unresolved gradient-spike failure documented in §6.6, so we make no parameter-inference claims on the nonlinear benchmark; (ii) $\hat{R}$ in this report is a single-chain split $\hat{R}$, not a true multi-chain diagnostic; (iii) the implicit Sinkhorn backward is validated as a filter-gradient check, but the SV2D HMC runs reported here were not rerun with that backward inside the full leapfrog loop.

References

- [1] Simo Särkkä. *Bayesian Filtering and Smoothing*. Cambridge University Press, 2013.
- [2] Fred Daum, Jim Huang, and Arjang Noushin. Exact particle flow for nonlinear filters. In *Signal processing, sensor fusion, and target recognition XIX*, volume 7697, pages 92–110. SPIE, 2010.
- [3] Fred Daum and Jim Huang. Particle degeneracy: root cause and solution. In *Signal Processing, Sensor Fusion, and Target Recognition XX*, volume 8050, pages 367–377. SPIE, 2011.
- [4] Yunpeng Li and Mark Coates. Particle filtering with invertible particle flow, 2017.
- [5] Liyi Dai and Fred Daum. A new parameterized family of stochastic particle flow filters, 2021.
- [6] Tao Ding and Mark J. Coates. Implementation of the daum-huang exact-flow particle filter. In *2012 IEEE Statistical Signal Processing Workshop (SSP)*, pages 257–260. IEEE, 2012.
- [7] Manuel Pulido and Peter Jan van Leeuwen. Sequential monte carlo with kernel embedded mappings: The mapping particle filter. 2019.
- [8] Chih-Chi Hu and Peter Jan Van Leeuwen. A particle flow filter for high-dimensional system applications. 2021.
- [9] Sebastian Reich. A nonparametric ensemble transform method for bayesian inference. *SIAM Journal on Scientific Computing*, 35(4):A2013–A2024, 2013.
- [10] Adrien Corenflos, James Thornton, George Deligiannidis, and Arnaud Doucet. Differentiable particle filtering via entropy-regularized optimal transport, 2021.
- [11] Peter Karkus, David Hsu, and Wee Sun Lee. Particle filter networks with application to visual localization. In *Conference on Robot Learning (CoRL)*, pages 169–178. PMLR, 2018.
- [12] Marco Cuturi. Sinkhorn distances: Lightspeed computation of optimal transport. pages 2292–2300, 2013.
- [13] Daniel Cremers, Florian Bernard, Marvin Eisenberger, Laura Leal-Taixé, and Aysim Toker. A unified framework for implicit sinkhorn differentiation, 2022.
- [14] Nicholas Metropolis, Arianna W. Rosenbluth, Marshall N. Rosenbluth, Augusta H. Teller, and Edward Teller. Equation of state calculations by fast computing machines. *The Journal of Chemical Physics*, 21(6):1087–1092, 1953.
- [15] Simon Duane, A.D. Kennedy, Brian J. Pendleton, and Duncan Roweth. Hybrid monte carlo. *Physics Letters B*, 195(2):216–222, 1987.
- [16] Radford M. Neal. Mcmc using hamiltonian dynamics, 2012.
- [17] Matthew D. Hoffman and Andrew Gelman. The no-u-turn sampler: Adaptively setting path lengths in hamiltonian Monte Carlo. *Journal of Machine Learning Research*, 15(1):1593–1623, 2014.
- [18] Alexandros Beskos, Natesh Pillai, Gareth O. Roberts, Jesús-Mariá Sanz-Serna, and Andrew M. Stuart. Optimal tuning of the hybrid Monte Carlo algorithm. *Bernoulli*, 19(5A):1501–1534, 2013.